Day 01
IC foundations and logic synthesis

VLSI Design Flow RTL to GDS
Lessons 01 to 06

Integration, implementation choices, architectural design, Unix and synthesis.

Contents and bookmarks link to sections; figures and page footers return to contents.

Includes **16 complete handwritten pages**. Find an original page or test your understanding.

Course: Prof. Sneh Saurabh, IIIT Delhi, NPTEL. Unix and Tcl tutorials: Jasmine Kaur. Worked examples state their analytical assumptions.

Contents

Revision route 3

Lesson 01 IC construction and manufacturing foundations 3

- Integration and scaling 3
- Devices and interconnect 5
- Why routing uses multiple layers 7
- Photolithography 8
- Ingot, wafer, die, and chip 12
- Design, fabrication, and the PDK 14
- Points to remember 16
- Definitions and mechanisms in Lesson 01 17
- Application and validation in Lesson 01 17
- Complete original notebook pages for Lesson 01 19

Lesson 02 Implementation choices and design quality 22

- Application-specific and general-purpose chips 22
- Four implementation styles 24
- Economics and the break-even point 27
- PPA and the rest of design quality 29
- Definitions and mechanisms in Lesson 02 31
- Application and validation in Lesson 02 32
- Complete original notebook pages for Lesson 02 33

Lesson 03 Architecture and the design flow 37

- From a product idea to a manufactured chip 37
- Abstraction and turnaround time 39
- Hardware and software partitioning 41

Estimating hardware before it exists . . . 45
Definitions and mechanisms in Lesson 03 . . . 45
Application and validation in Lesson 03 . . . 46
Complete original notebook pages for Lesson 03 . . . 47

Lesson 04 High level synthesis and timing . . . 50
The implementation gap and IP reuse . . . 50
Behavioral synthesis and its cost measures . . . 52
Paths and the clock-period budget . . . 56
Three implementations of a plus b plus c . . . 59
Definitions and mechanisms in Lesson 04 . . . 61
Application and validation in Lesson 04 . . . 62
Complete original notebook pages for Lesson 04 . . . 63

Lesson 05 Unix foundations for EDA . . . 67
A shell is the working interface . . . 67
Navigation and file operations . . . 68
Help, permissions, disk space, and processes . . . 69
Foreground and background jobs . . . 69
Definitions and mechanisms in Lesson 05 . . . 70
Application and validation in Lesson 05 . . . 70

Lesson 06 Logic synthesis and technology mapping . . . 71
RTL plus libraries plus constraints . . . 71
Library pins and instance pins . . . 72
Generic logic and technology mapping . . . 76
Definitions and mechanisms in Lesson 06 . . . 79
Application and validation in Lesson 06 . . . 80
Complete original notebook pages for Lesson 06 . . . 81

Revision checks and explained answers . . . 83
Check 01: Manufacturing yield has a denominator . . . 83
Check 02: Recover the fixed investment . . . 83
Check 03: Check the accelerator's interface . . . 83
Check 04: Separate latency from throughput . . . 83
Check 05: Know what a shell command changes . . . 83
Check 06: Count library and instance pins . . . 84

Handwritten page finder . . . 84

Sources and references . . . 85
Primary references for the technical clarifications . . . 85

Revision route

Begin with the device/interconnect cross-section, then connect implementation style to cost and PPA. Follow the product-to-chip flow before comparing the three adder architectures. Finish by identifying the distinct roles of RTL, cell libraries and constraints. For each handwritten diagram, explain what changes between its abstraction levels; do not treat a block diagram as physical geometry. The checks at the end connect manufacturing cost, interface bandwidth and architecture timing.

Revision checks and explained answers close the day. These are authored practice examples; original handwritten page identities remain attached to their source figures.

Lesson 01 IC construction and manufacturing foundations

Week 1 · Lecture video

This lesson connects the physical construction of a chip to the design files that eventually become manufacturing patterns. Its central idea is that a circuit is designed once, then its geometry is reproduced across many dies and wafers.

Integration and scaling

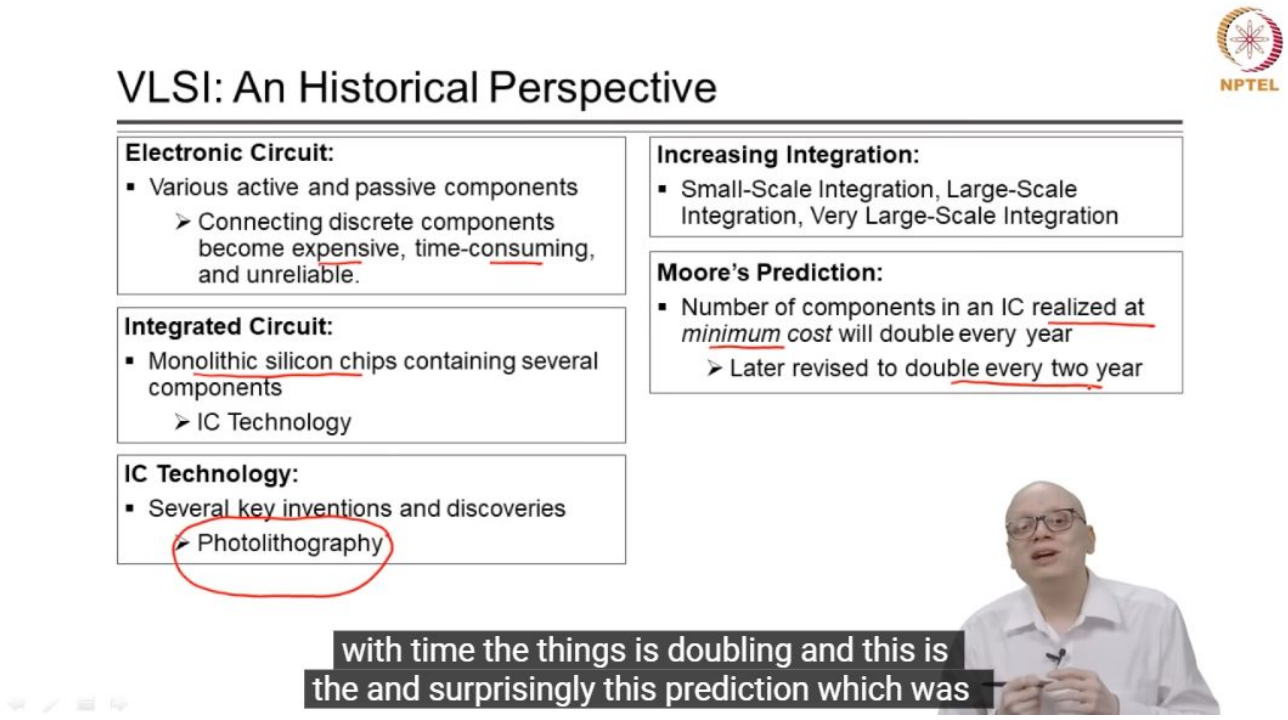

Figure 1.01. *Lecture: 09:15. The slide links monolithic integration, photolithography, and increasing component count.*

An **integrated circuit (IC)** contains interconnected electronic devices fabricated together on a common substrate. In the silicon technology discussed here, transistors and other structures are formed in and above a silicon wafer. A **monolithic** IC is built as one integrated piece; it is not assembled by soldering a collection of separately packaged transistors onto a board. Single-crystal silicon is the usual starting material in this example, but “monolithic” describes integration into one piece and should not be treated as a universal definition of the substrate's crystallinity.

A board containing separate components also implements a circuit. Its assembly, however, requires many separate parts and external connections. Integration makes the structures smaller and allows many copies to be fabricated with a common sequence of process steps. The expensive design and mask preparation can be shared across a large production run. This is the physical basis of the lecture's "copying" argument: the layout is reproduced, rather than every transistor being individually assembled by hand.

Moore's law is a historical observation and projection about economically useful component integration. The lecture distinguishes the original annual doubling estimate from the later estimate of roughly two years. It is not a physical law, a guarantee that every product doubles its transistor count, or a promise that clock frequency doubles. Technology names such as "65 nm" identify process generations; they should not automatically be read as the exact length of every transistor or wire in a modern process.

Shrinking structures can increase density and can improve energy or speed, but the benefits depend on the process and design. More transistors also mean more states, paths, connections, and constraints to verify. The course's design flow is needed because increasing manufacturing capability creates a growing design-management problem.

Integration

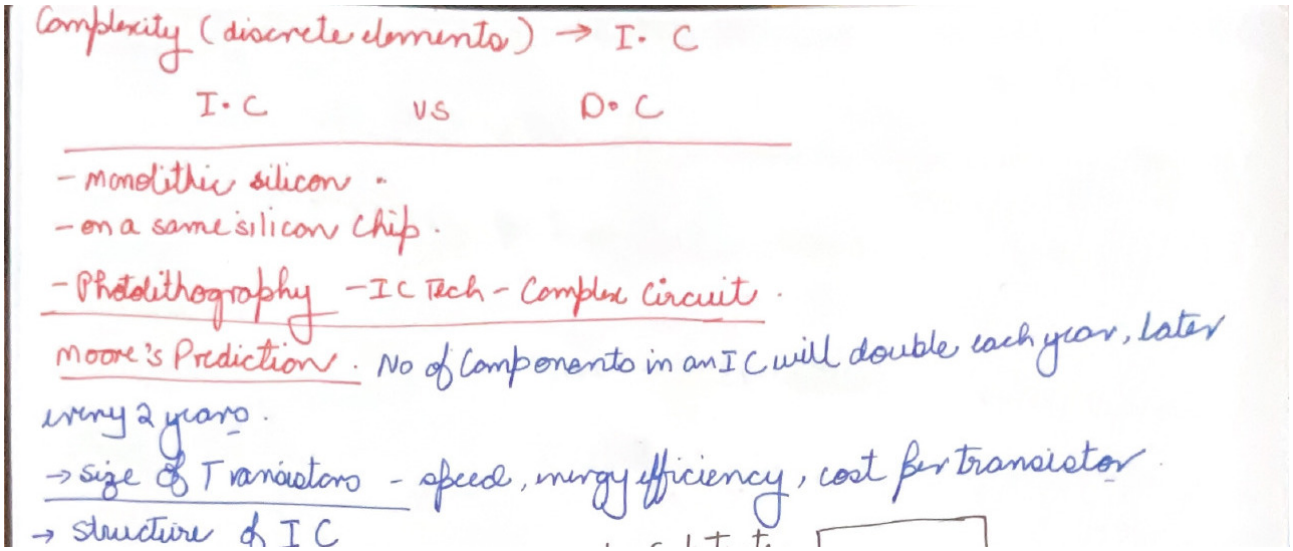

Figure 1.02. Kapil's integration and Moore's-law notes, Part 1 page 1

The comparison between discrete components and an IC captures the key manufacturing change: many devices are formed together and connected by patterned material layers. "Monolithic" means one integrated piece. Photolithography helps reproduce the geometry; it is one operation inside a much longer fabrication sequence. The historical doubling interval is an observation about integration, not a guarantee that all chip properties improve at the same rate.

Devices and interconnect

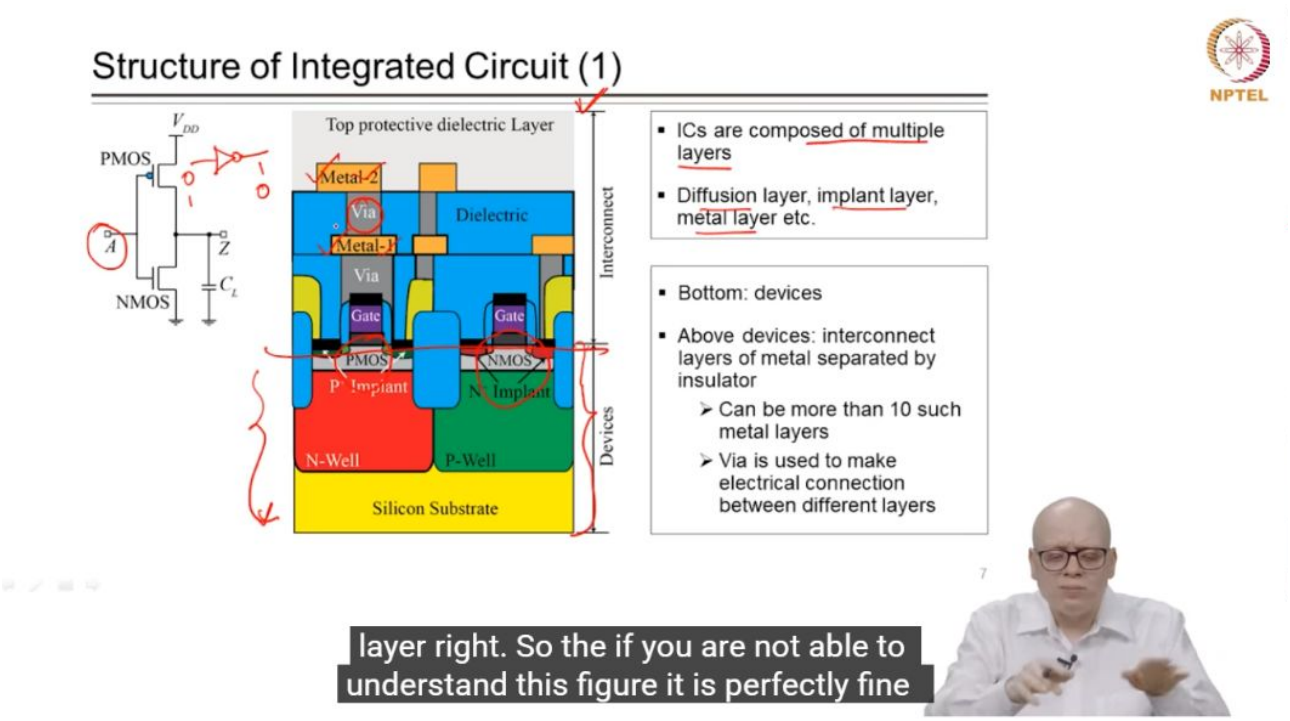

Figure 1.03. *Lecture: 15:38. Read the figure from the silicon substrate upward, then compare it with the inverter schematic on the left.*

The **CMOS inverter** uses a PMOS pull-up device connected toward the positive supply and an NMOS pull-down device connected toward ground. Their gates share the input; their output terminals meet at the output node. In the ideal steady-state switch model:

Input	PMOS	NMOS	Output
Low	Conducting	Off	Pulled high
High	Off	Conducting	Pulled low

This table describes the logical function. It does not mean the transition takes zero time or draws zero current. The output capacitance shown on the schematic must charge or discharge through a finite drive resistance. During switching, both devices can conduct briefly; leakage also prevents a real inverter from having exactly zero static current.

The cross-section explains how that schematic can exist physically. The lower structures form the devices: substrate and wells, doped source/drain regions, and the gate stack. Above them, **interconnect** carries signals and power between devices. The metal conductors are separated by **dielectric**, an electrically insulating material. A conductor on one layer does not automatically contact a conductor on another layer.

A **via** is a deliberately formed conductive connection between interconnect levels. A **contact** connects a device terminal or suitable lower-level structure into the interconnect system; exact naming depends on the process. The gate dielectric is different from a via: its job is insulation while allowing the gate's electric field to control the channel. Confusing the two would change the transistor into a short circuit.

The drawing is a conceptual cross-section, not a complete process recipe or a universal layer stack. Real dimensions, allowed layers, and connection rules come from the chosen process. SKY130's process-stack documentation is an example of such process-specific information.

The layer stack

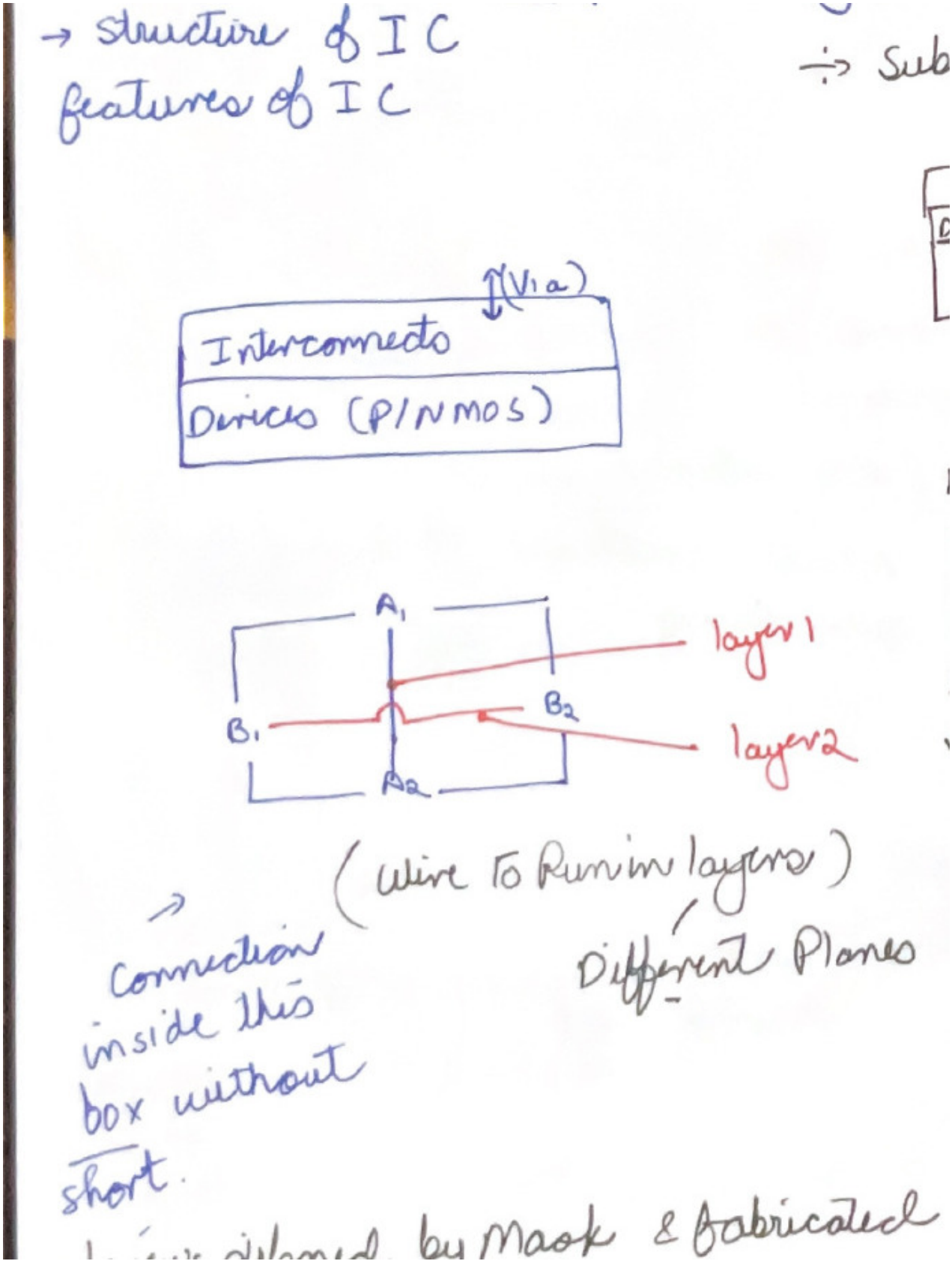

Figure 1.04. Kapil's drawing of device and interconnect layers, Part 1 page 1

Read the cross-section vertically: devices perform switching near the substrate, and metal levels provide connections above them. Dielectric separates conductors. A via deliberately bridges two levels. Read the A/B crossing drawing as a connectivity problem: the two paths may overlap in a top view while occupying different heights, so they need not short together.

Why routing uses multiple layers

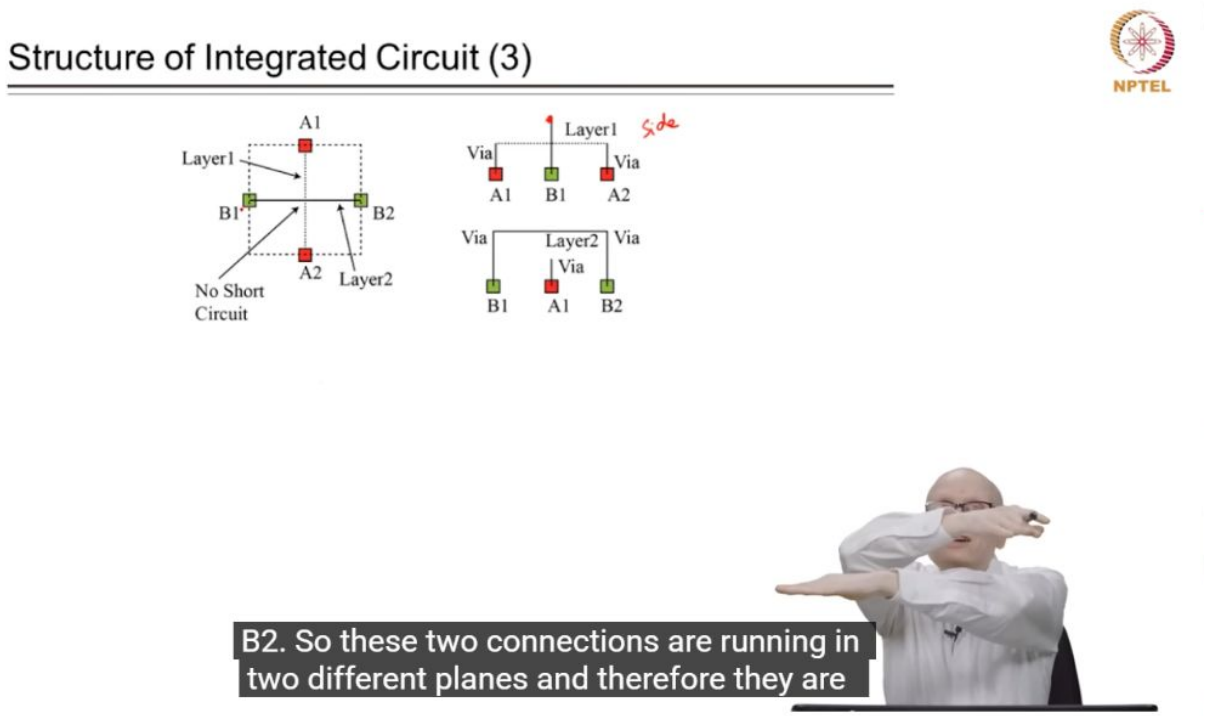

Figure 1.05. *Lecture: 19:35.*

The lecture's routing puzzle places two pairs of terminals around a bounded region. With alternating terminals on the boundary, connecting each pair inside a single plane forces a crossing. If both paths are conductors in the same layer, that crossing joins the nets electrically.

Moving one path into a different metal layer solves this particular problem. The dielectric keeps the crossing conductors separate; vias are added only where a net intentionally changes layers. A top view can therefore show two lines crossing even though no electrical connection exists between them. A netlist records intended connectivity, while layout adds geometry and layer identity.

More layers provide routing freedom but do not remove constraints. Wires and vias still consume area and have resistance and capacitance. A route must satisfy both connectivity and the process's geometric rules. This is why physical routing is more demanding than drawing arbitrary lines between logic symbols.

Photolithography

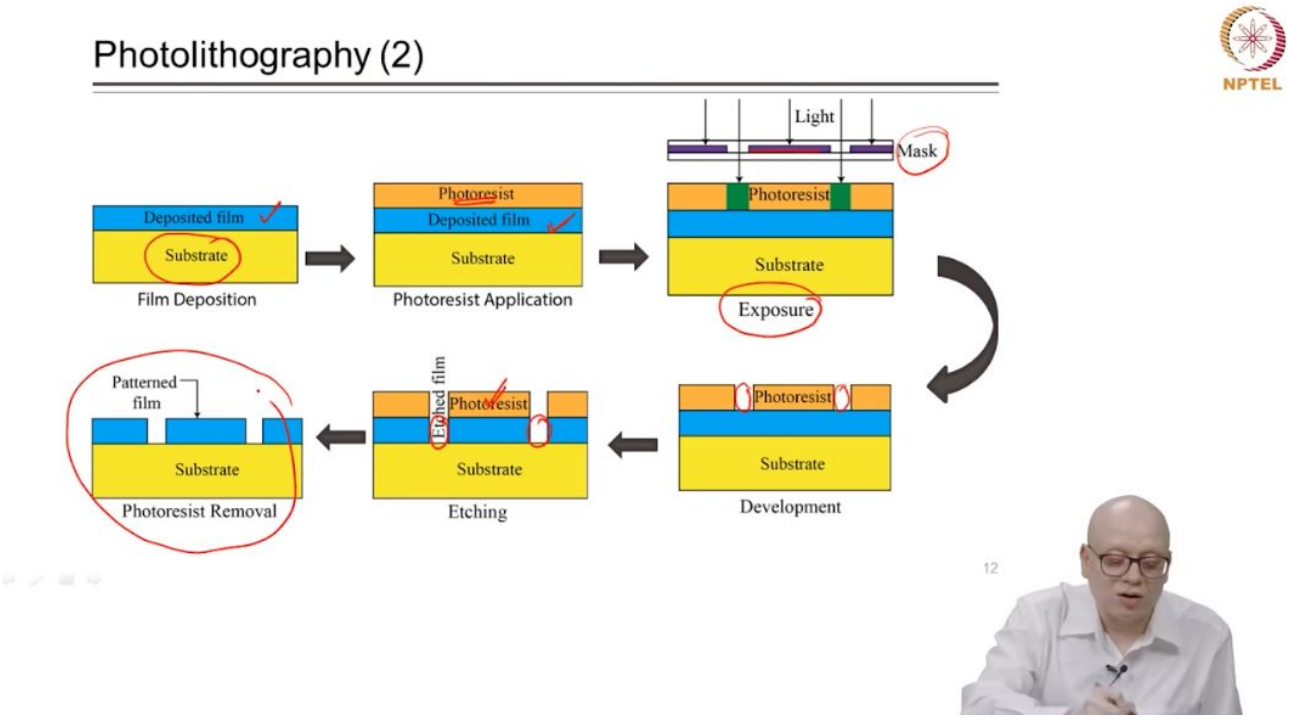

Figure 1.06. *Lecture: 27:58. Follow the arrows across the top row and back across the bottom row. Orange is photoresist, blue is the film being patterned, and yellow is the substrate.*

Photolithography defines a spatial pattern in a light-sensitive resist using optical exposure and development. A subsequent operation, such as etching, transfers that resist pattern into another material. The lecture groups these operations into one explanatory sequence. Keeping the operations separate makes it clear which material is changing at each step.

A **photomask** carries the pattern used during exposure. In the lecture's simplified transmissive-mask example, transparent regions pass light and opaque regions block it. The **photoresist** is a temporary light-sensitive coating on the wafer. It is neither the final interconnect nor the silicon substrate. ASML's lithography explanation describes the projection principle behind this pattern transfer.

1. Prepare the film to be patterned

The starting picture has a continuous deposited film on the substrate. This film might be an insulator or another process material. Its eventual function depends on the layer being fabricated. Deposition gives material coverage; it does not yet define the openings in the slide.

2. Apply and prepare the resist

A liquid resist is dispensed and spread by spinning the wafer. A suitable bake removes solvent and prepares the coating for exposure. Thickness and uniformity matter: if some regions are much thicker than others, exposure and later pattern transfer may behave differently across the wafer. The resist must also adhere well enough to protect the intended regions during processing.

3. Align and expose

The exposure system positions the pattern relative to previously fabricated structures, then exposes selected resist regions. Light changes the resist chemistry and hence its response to the developer. At this stage a chemical difference exists in the resist; the desired openings have not simply been burned through the deposited film.

Alignment matters because a correctly shaped contact opening at the wrong location can miss its intended terminal or touch an unintended conductor. **Overlay** describes the alignment relationship between patterns from different processing steps. **Critical dimension** describes an important patterned feature size, such as a line width or opening width. They are separate requirements: a feature can have the right size and still be misplaced.

4. Develop the resist

The lecture shows **positive resist**: the exposed portions become more soluble in the developer and are removed, leaving windows in the resist. The unexposed regions remain as a protective pattern. In **negative resist**, exposure makes the exposed regions less soluble relative to the unexposed regions, reversing which portions remain after development.

The statement “light-exposed resist is removed” therefore needs the positive-resist qualification. It is not true for every resist process. ASML's manufacturing overview distinguishes the two resist behaviors and places lithography and etching in the wider manufacturing sequence.

5. Etch the exposed film

Etching removes the film in the windows where resist no longer protects it. The patterned resist acts as a temporary mask. **Etch selectivity** describes the relative rates at which different materials are removed. A useful process removes the target material quickly enough while preserving sufficient masking material and controlling attack on the underlying material.

The lecture's statement that the etchant does not react with resist is an idealized explanation. Real resist can erode; what matters is adequate selectivity, thickness, and process control. An isotropic etch can remove material sideways as well as vertically, potentially undercutting the mask. A strongly directional etch helps preserve steep sidewalls. The choice follows the feature and material requirements.

6. Strip the remaining resist

Once the film pattern is transferred, the remaining resist is removed. In the final picture, the blue film carries the pattern and the orange temporary coating is gone. Resist removal is not the same operation as developing the exposed resist earlier: development created the mask pattern, while stripping removes the mask after it has served its purpose.

Trace one opening through the entire sequence

Consider a transparent opening on the lecture's mask. Light reaches the resist underneath it. In a positive-resist process, that exposed resist is removed during development. The deposited film becomes accessible there, so etching removes it. After resist stripping, an opening remains in the film. Under an opaque mask region, the resist was initially retained and protected the film, so the film remains.

The causal chain is:

Mask transmission → local exposure → resist solubility difference → resist opening → film removal.

The pattern is reused to fabricate many dies. A complete chip requires many aligned process steps; one exposure does not construct every transistor and interconnect layer at once. Real flows can also use implantation masks, hard masks, multiple patterning, and other operations. Those are extensions of the pattern-and-transfer principle, not evidence that the simple slide is a complete fabrication recipe.

What is a mask?

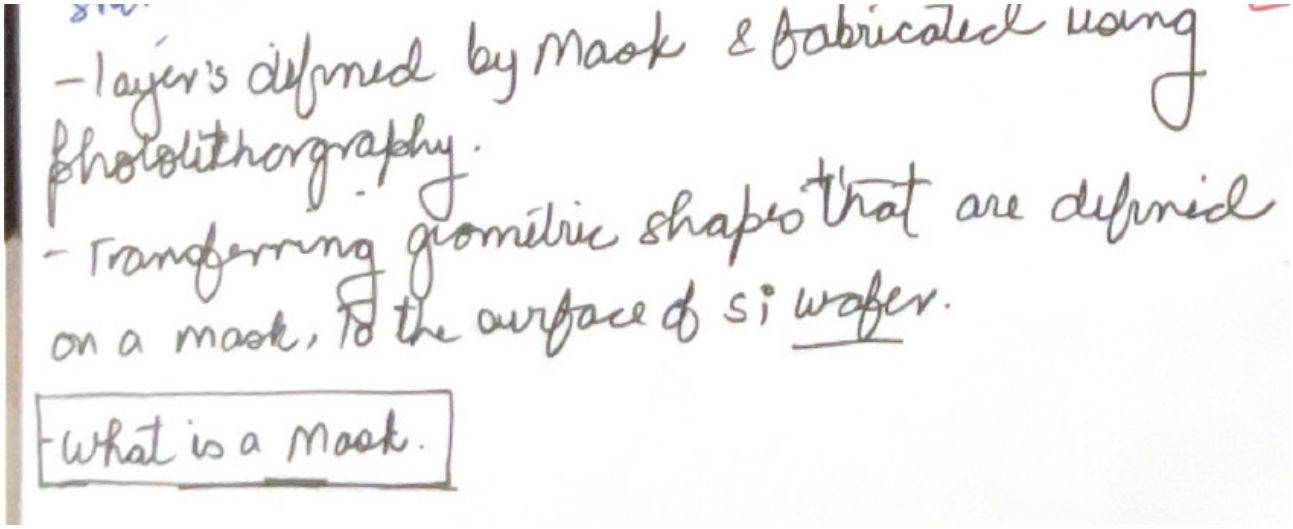

Figure 1.07. *Source: Part 1, PDF page 1, lower-left question.*

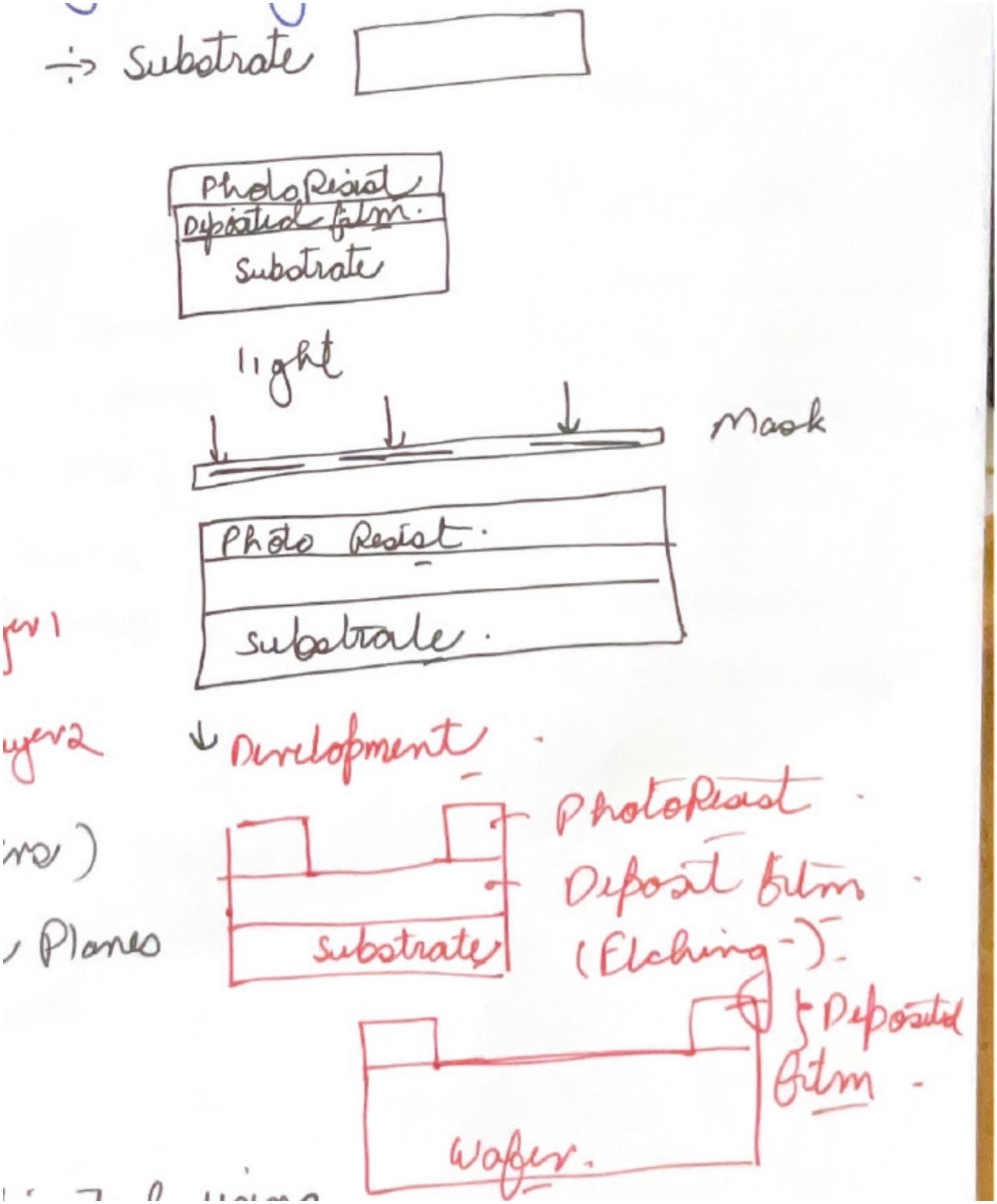

Figure 1.08. Kapil's lithography sequence and mask question, Part 1 page 1

A **photomask** is a patterned optical template used by a lithography system to control which regions of a light-sensitive resist receive exposure. In a conventional transmissive mask, patterned absorbing material on a transparent substrate controls light transmission. The mask is distinct from the resist: the mask supplies the optical pattern; resist is the temporary coating on the wafer. A reticle commonly carries the pattern projected onto one exposure field, which the tool repeats across the wafer. Some lithography technologies use reflective masks, so “a glass plate that blocks light” is an introductory example rather than a universal definition.

The sequence is correctly read as **coat → expose → develop → transfer the pattern → strip**. Exposure changes resist chemistry; development removes selected resist; etching removes exposed underlying material. The mask is not pressed into the silicon like a stamp, and UV light does not directly carve the finished metal line. The optical image and process chemistry together determine the printed feature. ASML explains this pattern-transfer role.

To retain a narrow conductor after subtractive etching, the resist must protect that conductor during the etch. With positive resist, the protected region must remain unexposed in this simplified example. Changing resist tone changes the required mask polarity. This is why "transparent part becomes a wire" is not a general rule: the answer depends on the layer, resist tone, and subsequent process.

The later notes on optical proximity correction and multiple patterning extend this topic in Lesson 09.

Ingot, wafer, die, and chip

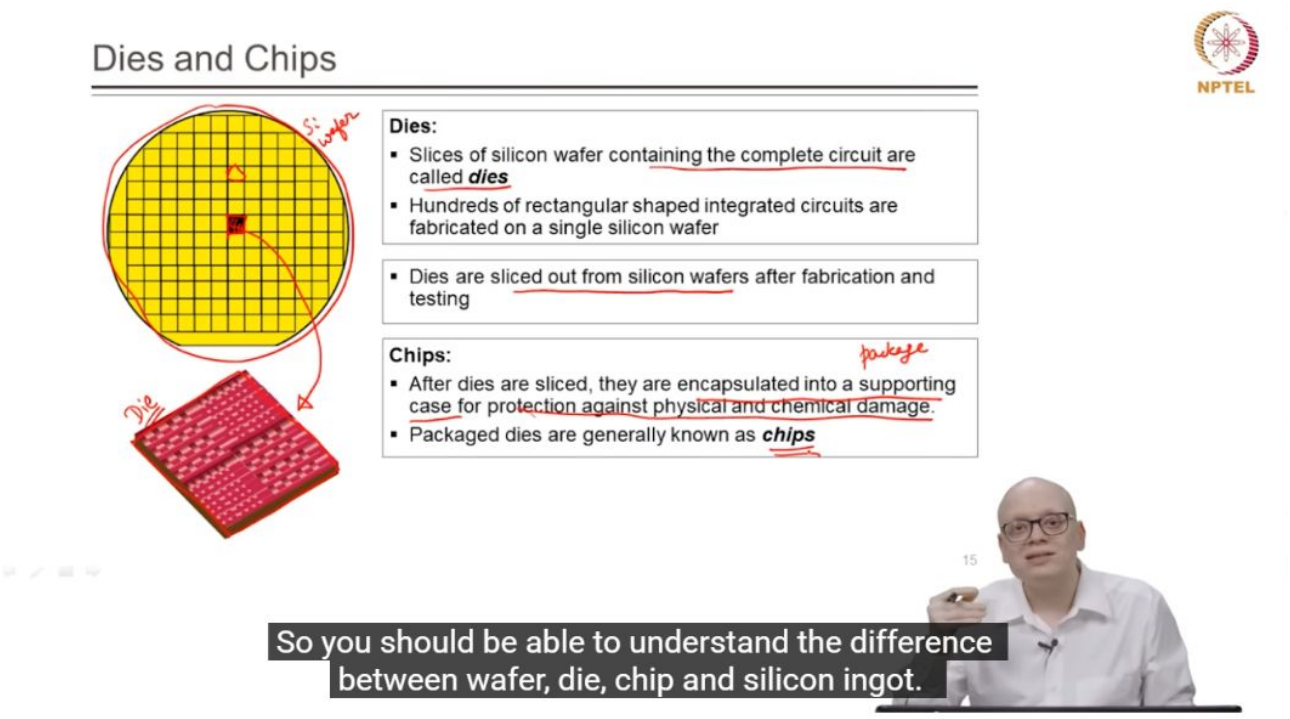

Figure 1.09. *Lecture: 36:46. Each small rectangular region represents a separate die location, not an individual transistor.*

Term	Meaning in this lecture	What happens next
Ingot	A grown bulk crystal of silicon	Sliced and processed into wafers
Wafer	A thin substrate on which many die sites are fabricated	Patterned through many process steps and tested
Die	One individual circuit region cut from a processed wafer	Connected and packaged, depending on the product
Packaged chip	A die or dies with a package providing protection and external connections	Final testing and system use

The lecture introduces the Czochralski growth idea: a seed crystal is pulled from a silicon melt while growth conditions are controlled. This explains why the wafer begins as crystalline material. Slicing alone is not the entire wafer-preparation process; surface finishing and other preparation are also needed before device fabrication.

Yield is the fraction of manufactured items that meet the relevant acceptance criteria at a specified stage. If 500 tested dies contain 450 acceptable dies, the illustrative die yield is

$$Y = \frac{450}{500} = 0.90.$$

This fraction corresponds to 90%. The denominator and test stage must be stated. Wafer-level die yield and final packaged-product yield are not automatically identical. A package protects the die and supplies electrical connections, but it also introduces thermal, mechanical, electrical, and testing considerations. In general usage “chip” can also mean bare die; the lecture uses it primarily for the packaged product.

How can we make good or defect-free dies?

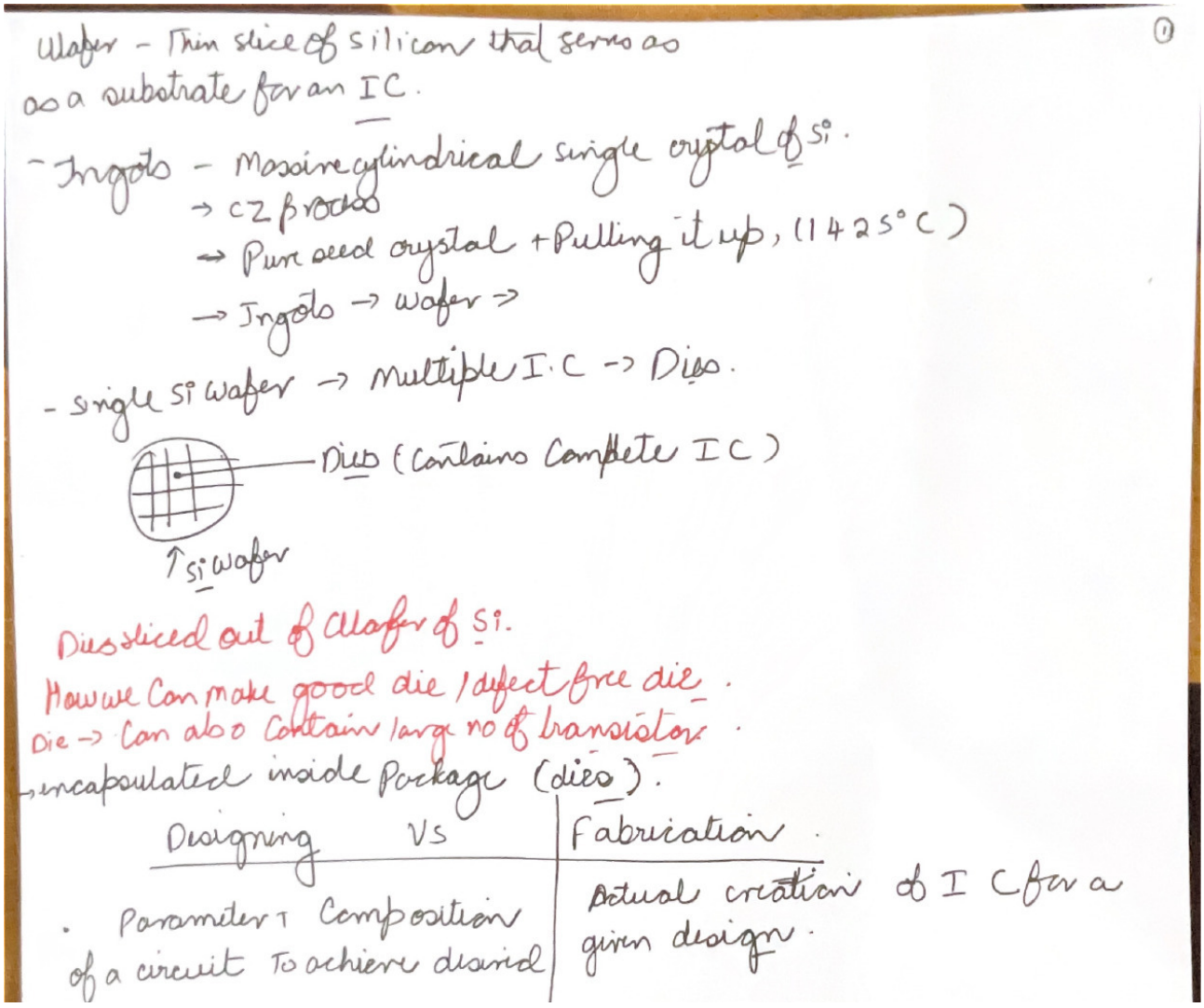

Figure 1.10. Kapil's ingot, wafer, die and yield notes, Part 1 page 2

The ingot-to-wafer-to-die sequence describes three scales of the same manufacturing chain. In Czochralski growth, a seed contacts molten silicon and is withdrawn under controlled conditions to grow a single-crystal ingot. Wafers are sliced, finished, and processed; dies are the individual circuit

regions. The temperature written in the note is an approximate process value, not a setting to memorize as universally exact.

Answer: increase the probability of a good die by controlling contamination, process variation, alignment, deposition, etching, and other fabrication steps, and by designing within the process's validated rules. Cleanrooms reduce particles; inspection and metrology detect process drift; design-for-manufacturability measures avoid fragile patterns. Verification removes design mistakes before fabrication. These measures target different causes of failure.

Testing then identifies dies that meet the specified checks; it does not normally repair a broken transistor or turn a defective die into a good one. Some designs include redundancy or repair mechanisms, especially memories, but that must be designed in. No practical process promises that every die is defect-free. **Yield** measures the fraction passing the chosen acceptance criteria, while test coverage measures how effectively a test targets a stated fault model. Lesson 08 explains the difference with the numerical examples.

Design, fabrication, and the PDK

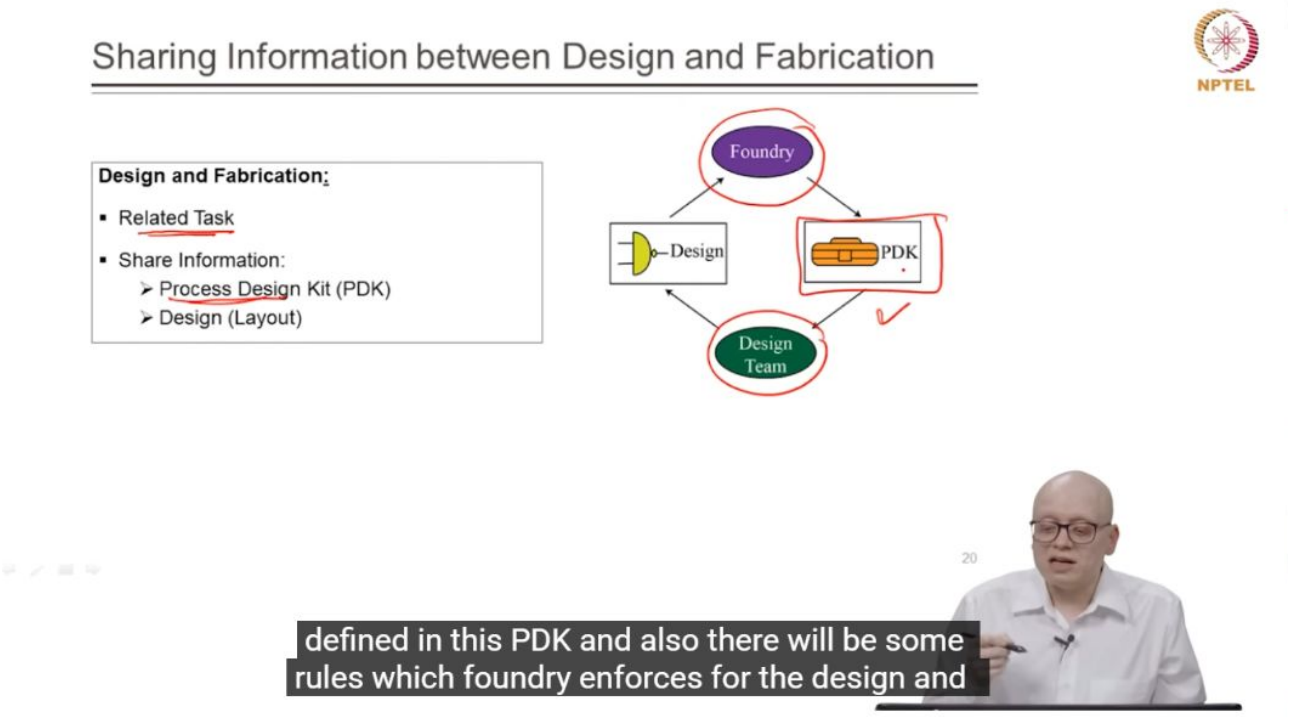

Figure 1.11. *Lecture: 46:21.*

Design chooses a circuit organization and physical implementation that meet the required behavior and constraints. **Fabrication** uses a manufacturing process to realize that implementation in material. Separating the businesses does not make the technical tasks independent: a layout is useful only if the chosen process can manufacture it and the resulting devices behave as assumed.

A **process design kit (PDK)** supplies process-specific information used by design tools and engineers. It can include device models, layer definitions, geometric rules, and verification or extraction support. For example, a designer cannot choose arbitrary metal spacing and assume that the foundry will manufacture it reliably. A process rule defines permitted geometry, while device and parasitic models support predictions about electrical behavior. The SKY130 file-type reference shows concrete examples of model and verification files.

The lecture introduces three business models: a **fabless** company concentrates on design and outsources manufacturing; a **merchant foundry** manufactures for customers; an **integrated device manufacturer (IDM)** combines design and manufacturing activities. Real companies may combine business models. The conceptual distinction matters more here than memorizing a company's current classification.

Following design rules improves manufacturability; it does not by itself prove functional correctness or guarantee a particular yield for every design. A design can be geometrically legal but logically wrong. Likewise, fabrication variability, defects, and electrical conditions affect the final outcome. The physical and functional checks address different failure modes.

Business models and process information

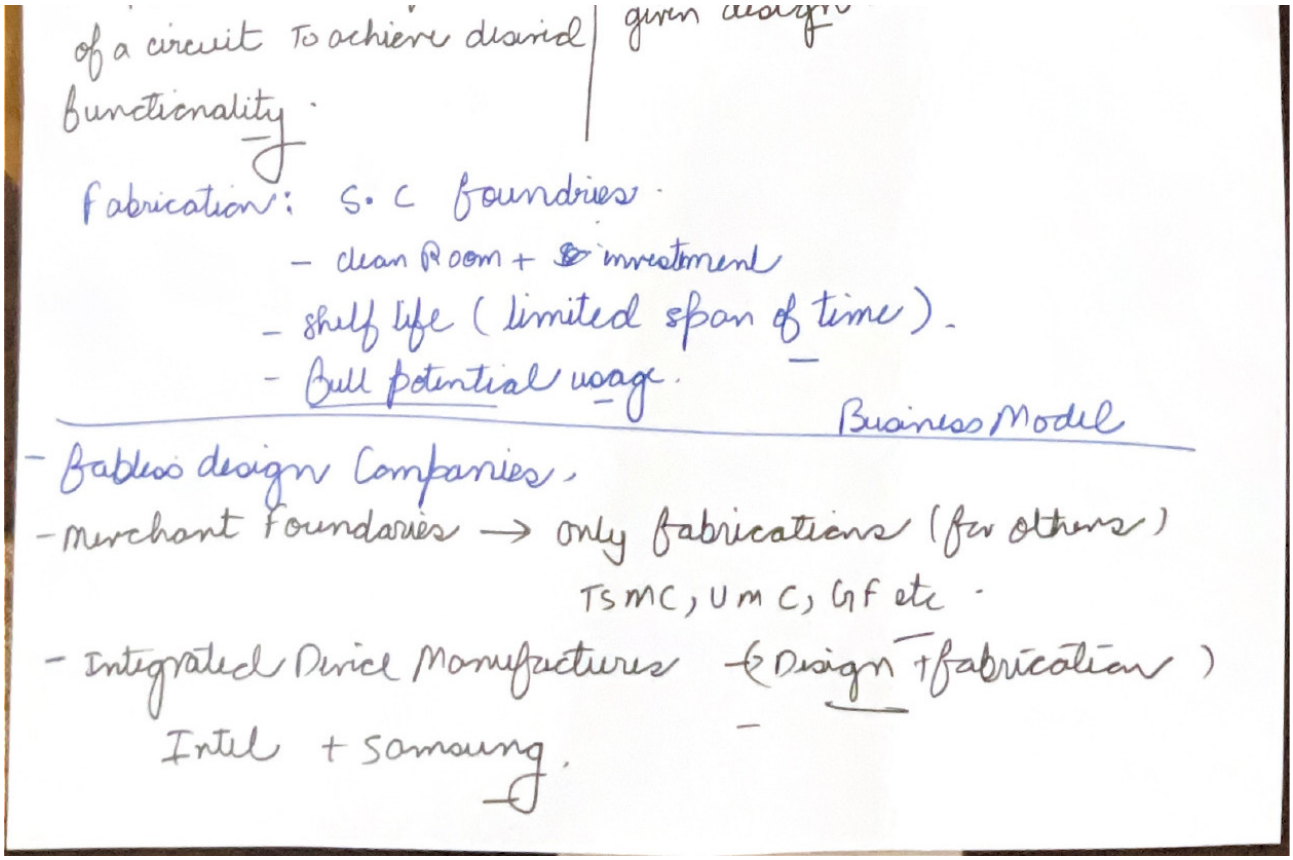

Figure 1.12. Kapil's design-versus-fabrication and business-model notes, Part 1 page 2

The table separates the design investment from the fabrication investment. Design needs engineers, tools, compute, and verification effort; fabrication needs process equipment, facilities, materials, and sustained process control. The useful lifetime and economics of a fab depend on its products and upgrades. A mature process can remain valuable; a newer node does not automatically make an older fab unusable after a fixed number of years.

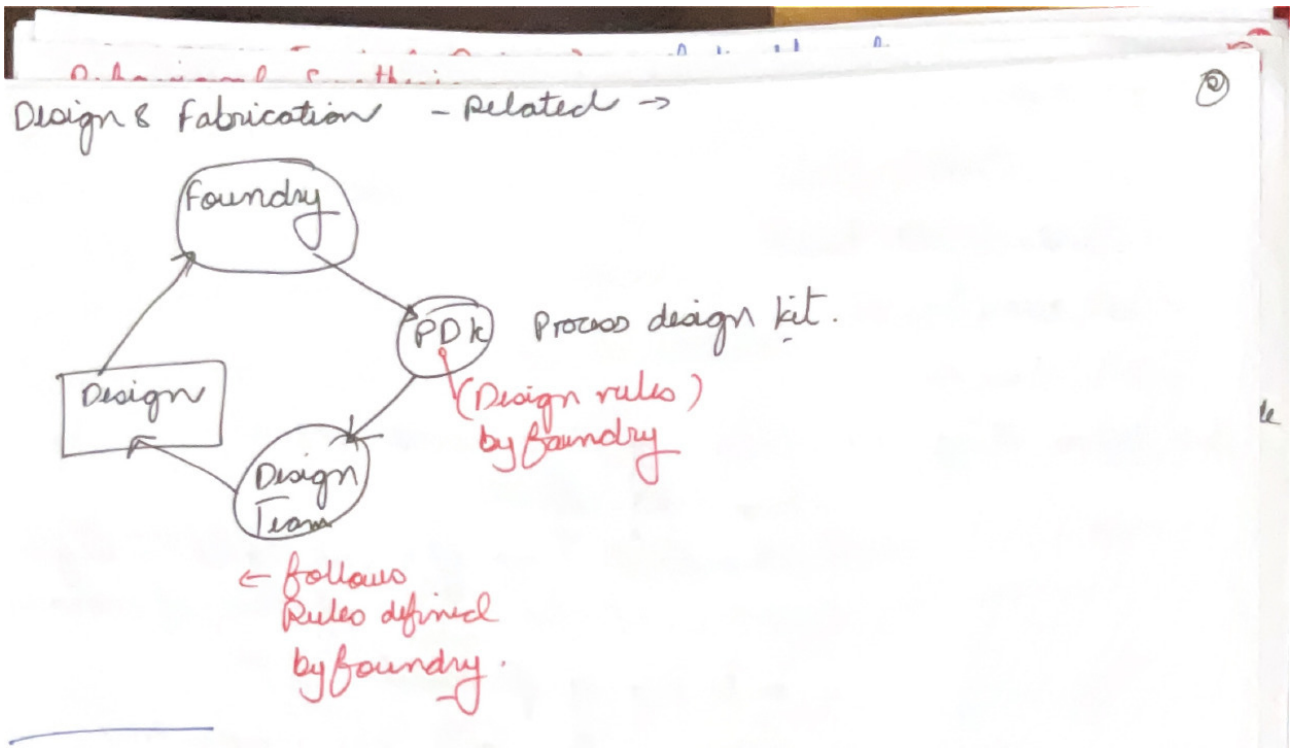

Figure 1.13. Kapil's PDK information loop, Part 1 page 3

The arrows are an information contract. The foundry supplies models and rules; the designer creates a circuit and layout compatible with them; the foundry receives manufacturing data. A **design rule** might specify minimum width, spacing, enclosure, or overlap. A **device model** predicts electrical behavior. A **standard-cell library** provides already designed logic building blocks for a particular technology and library family. These are related resources, but the PDK and the cell library are not interchangeable terms.

Points to remember

- A metal crossing is an electrical junction only if the physical layers and connecting structures make it one.
- Development patterns resist; etching transfers a pattern into another material; stripping removes the remaining resist.
- The positive-resist rule is “exposed regions are removed during development.” Negative resist reverses the retained regions.
- The mask is reused, but each wafer still undergoes physical processing.
- A wafer contains many dies; each die can contain a very large number of devices.
- The PDK connects design assumptions to a specific manufacturing process.

Recall checks

1. Why can two wires cross in a top-view drawing without being shorted? Identify the role of both the dielectric and the via.
2. If a region is opaque on the mask in the positive-resist example, which materials remain there after development, etching, and stripping?
3. Why does a design-rule-clean layout still need functional verification?

4. If yield improves while wafer cost and gross die count stay fixed, why does the approximate manufacturing cost per good die fall?

Definitions and mechanisms in Lesson 01

Integrated circuit and monolithic integration. An integrated circuit is an interconnected collection of electronic devices formed together on a substrate to perform a circuit function. In monolithic integration, those structures belong to one integrated piece rather than an assembly of separately packaged components. The benefit comes from jointly fabricating small devices and their connections using repeatable processing. The word *integrated* therefore describes physical construction, not merely the fact that several components cooperate. A PCB containing packaged transistors is a circuit, but its board assembly is not itself a monolithic IC.

Device, conductor and dielectric. A device controls or otherwise affects electrical behavior; a conductor provides a path for current; a dielectric separates conductors electrically. In the inverter cross-section, transistors perform switching, metal wires connect terminals, and insulating layers prevent unwanted connections. A via creates an intended conductive bridge through an interlayer dielectric. This role-based definition helps read an unfamiliar cross-section: first identify what switches, what connects, and what isolates. A top-view overlap does not supply enough information to decide connectivity without the layer and via information.

Lithography and pattern transfer. Lithography creates a patterned resist by exposure and development. Pattern transfer uses that temporary pattern to control another operation, such as removing the exposed underlying film. The distinction identifies the material that changes: development changes the resist pattern; etching changes the target material; stripping removes the remaining temporary resist. In the positive-tone example, light makes exposed resist removable during development. The mask's bright region therefore determines an opening in resist, while the final device feature also depends on the subsequent process.

Critical dimension and overlay. A critical dimension measures an important feature size, such as the width of a contact opening. Overlay measures the relative registration of patterns from different steps. A contact may have the correct width yet miss the intended terminal because its position is wrong. Conversely, an accurately centered contact may be too narrow. These are independent geometric questions, so a correct explanation names both size control and alignment control rather than saying only that the pattern must be accurate.

PDK and standard-cell library. A PDK provides process-specific models, layer information and rules used to design for a manufacturing process. A standard-cell library provides reusable circuit implementations, such as a particular NAND or flip-flop, together with views for different tools. The PDK constrains and models the technology; the library supplies building blocks designed for that technology. A rule about metal spacing and a table describing NAND delay serve different purposes. A designer needs compatible resources, not an assumption that either file set contains every other kind of information.

Application and validation in Lesson 01

A fabrication sequence is interpreted by tracking the same physical opening through exposure, development, etching and stripping. Each step changes a particular material; the resist opening and the etched film opening are different objects. Connectivity is reconstructed from layer identity and intentional contacts, including vias. A top-view crossing alone does not establish a connection.

Worked polarity check. Consider a metal film patterned by subtractive etching with positive-tone resist. To retain a metal line, its region must remain protected during etching. The resist over that line therefore remains after development, requiring that region to be unexposed in this simplified process. Exposed neighboring regions develop into openings and their metal is removed. Reversing the resist tone changes the exposure choice. The final conductor is determined by the entire sequence, not by transparency alone.

Complete original notebook pages for Lesson 01

Part 1 original page 01

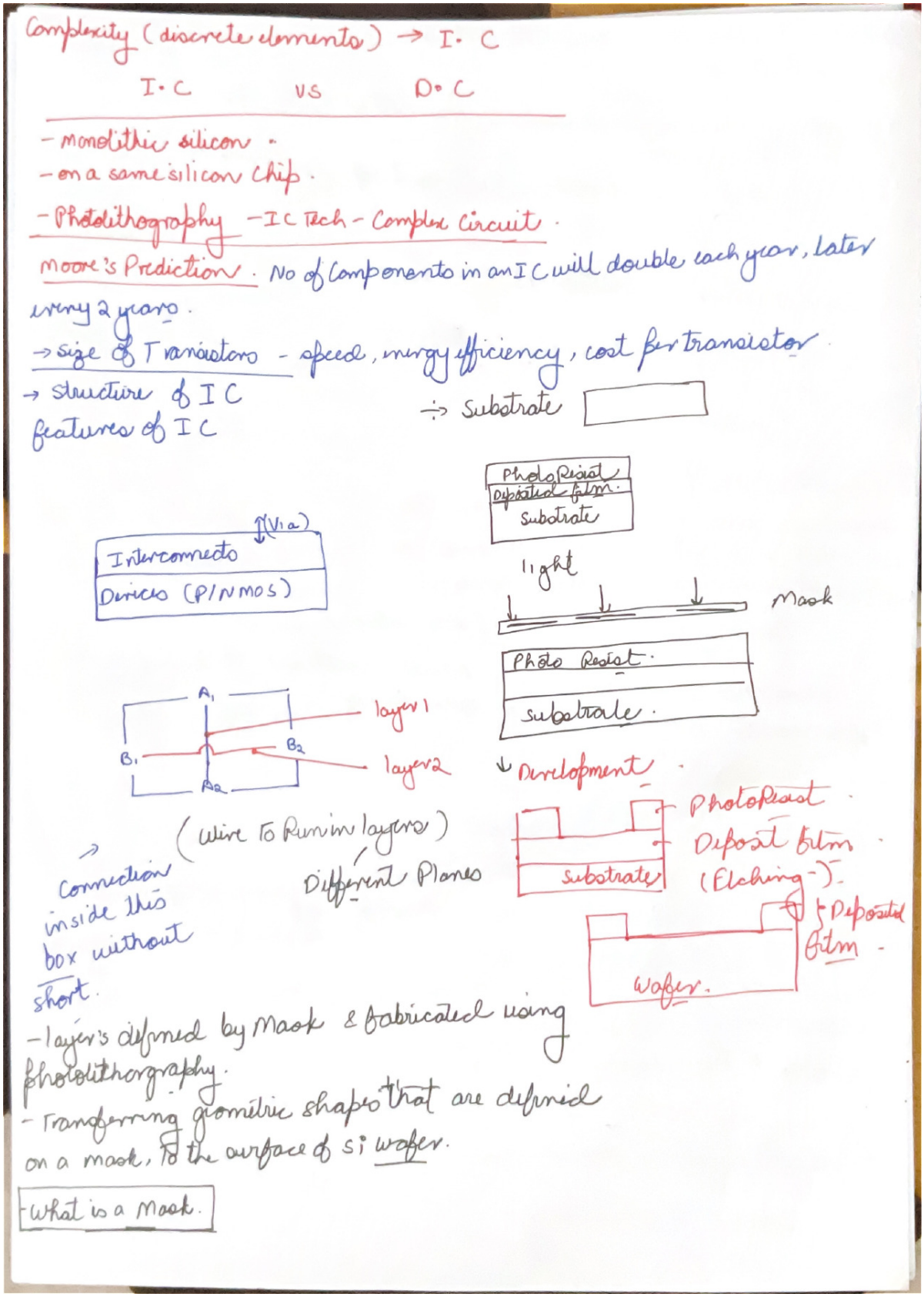

Figure 1.14. *Source: Part 1, PDF page 1. IC integration, layer stack, photolithography; "What is a mask?".*

Explanation and worked examples: Day 1 · Lesson 01.

Part 1 original page 02

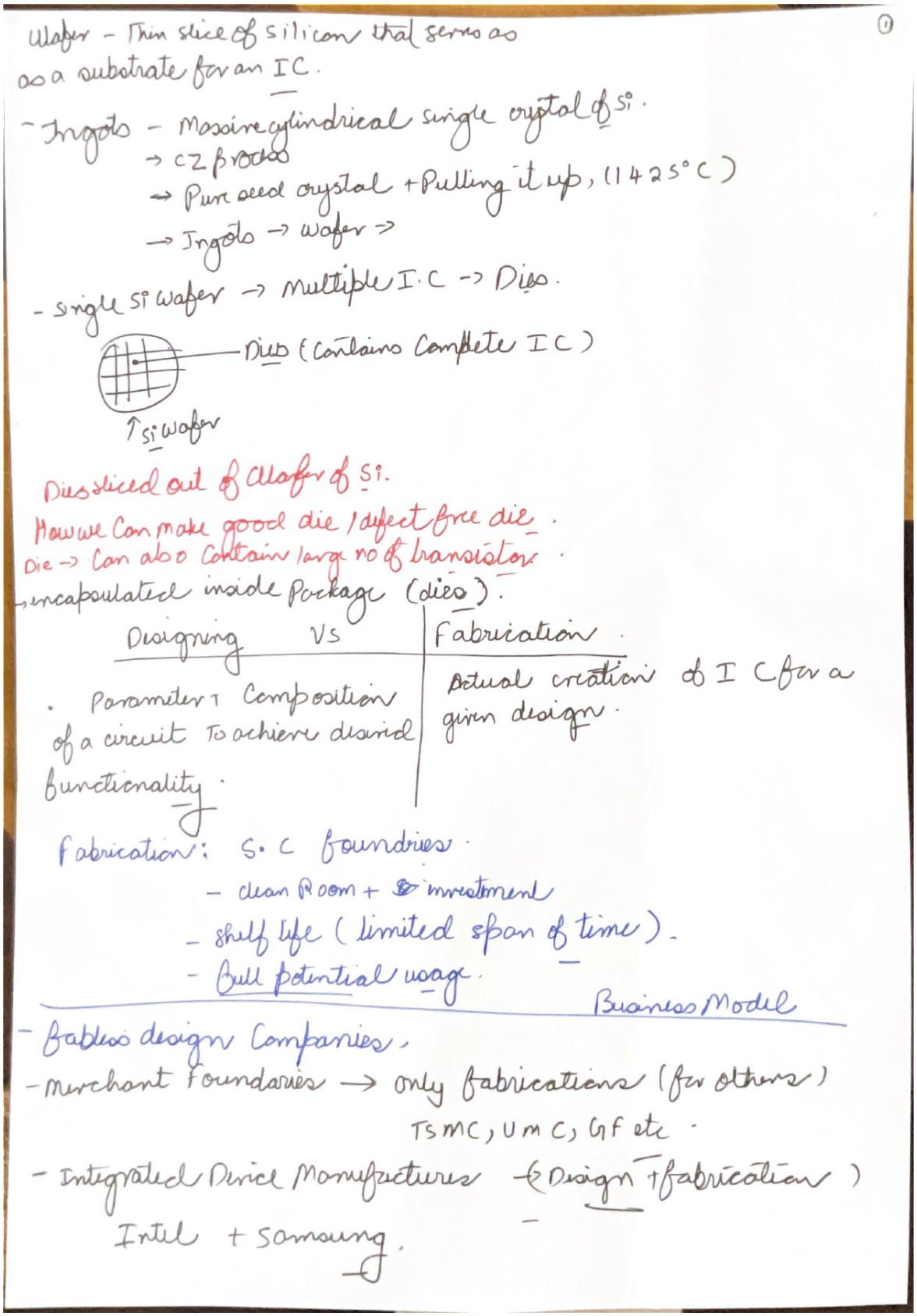

Figure 1.15. *Source: Part 1, PDF page 2. Ingot, wafer, die, yield; good/defect-free dies; industry models.*

Explanation and worked examples: Day 1 · Lesson 01.

Part 1 original page 03

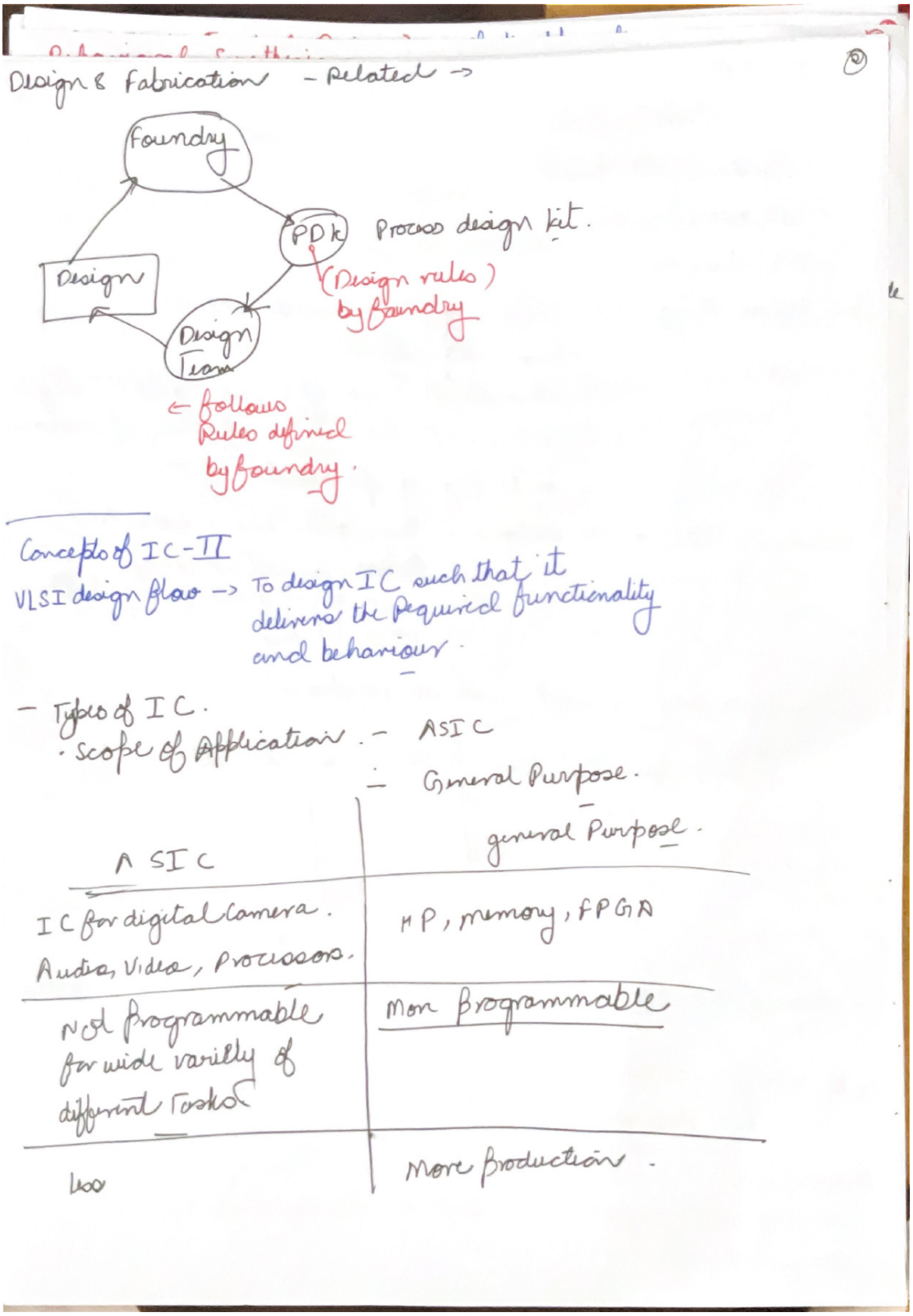

Figure 1.16. *Source: Part 1, PDF page 3. PDK and application-specific/general-purpose ICs.*

Explanation and worked examples: Day 1 · Lesson 01 · Day 1 · Lesson 02.

Lesson 02 Implementation choices and design quality

Week 1 · Lecture video

Application-specific and general-purpose chips

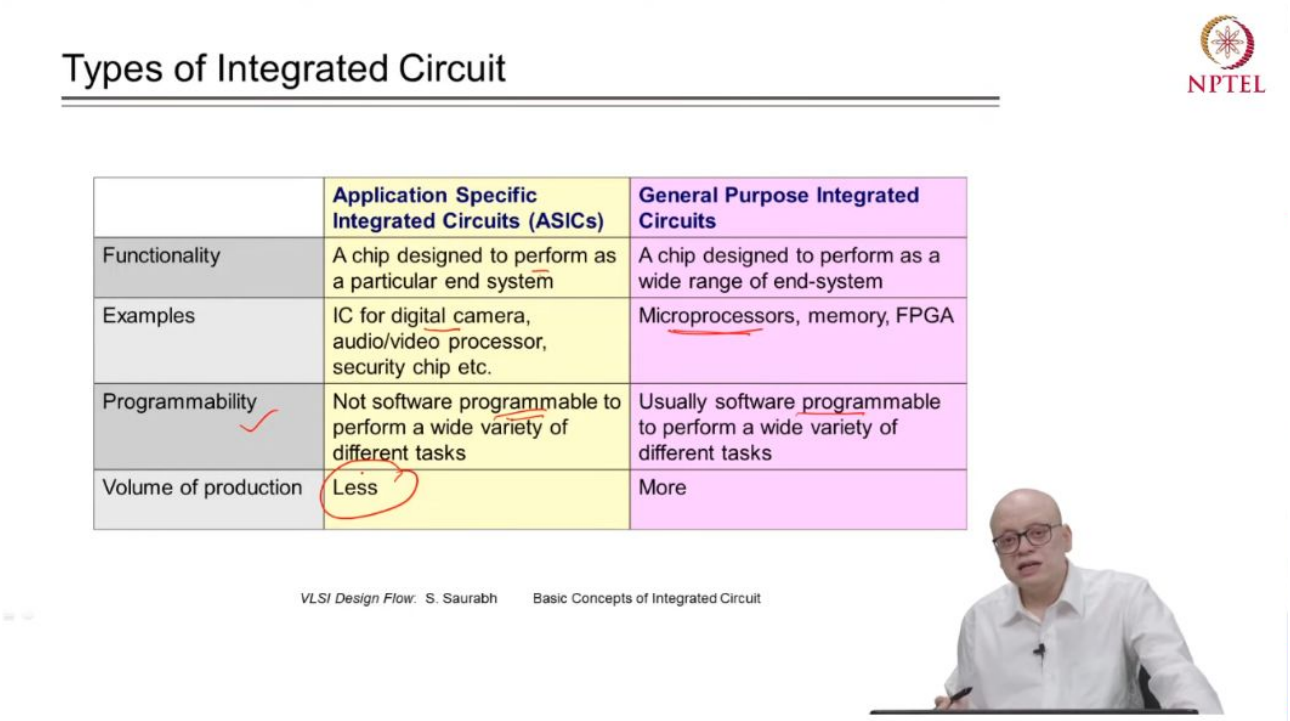

Figure 1.17. *Lecture: 5:16. ASIC and general-purpose IC examples*

An **application-specific integrated circuit (ASIC)** is designed around a particular application or class of applications. A **general-purpose IC** supplies a reusable capability that can serve many systems. A microprocessor executes different programs; a memory stores information for many applications; an FPGA offers configurable logic. This classification describes intended use, whereas full-custom, standard-cell, and FPGA-based design describe implementation styles.

An ASIC can contain a programmable processor. It is still application-specific if the whole chip is built for a particular purpose. Conversely, configuring an FPGA for a camera implements application-specific behavior on an existing programmable fabric; it does not fabricate a custom ASIC. Production volume and programmability are tendencies in the lecture's introductory comparison, not necessary definitions. Some ASICs ship in enormous volumes, and some general-purpose parts serve small markets.

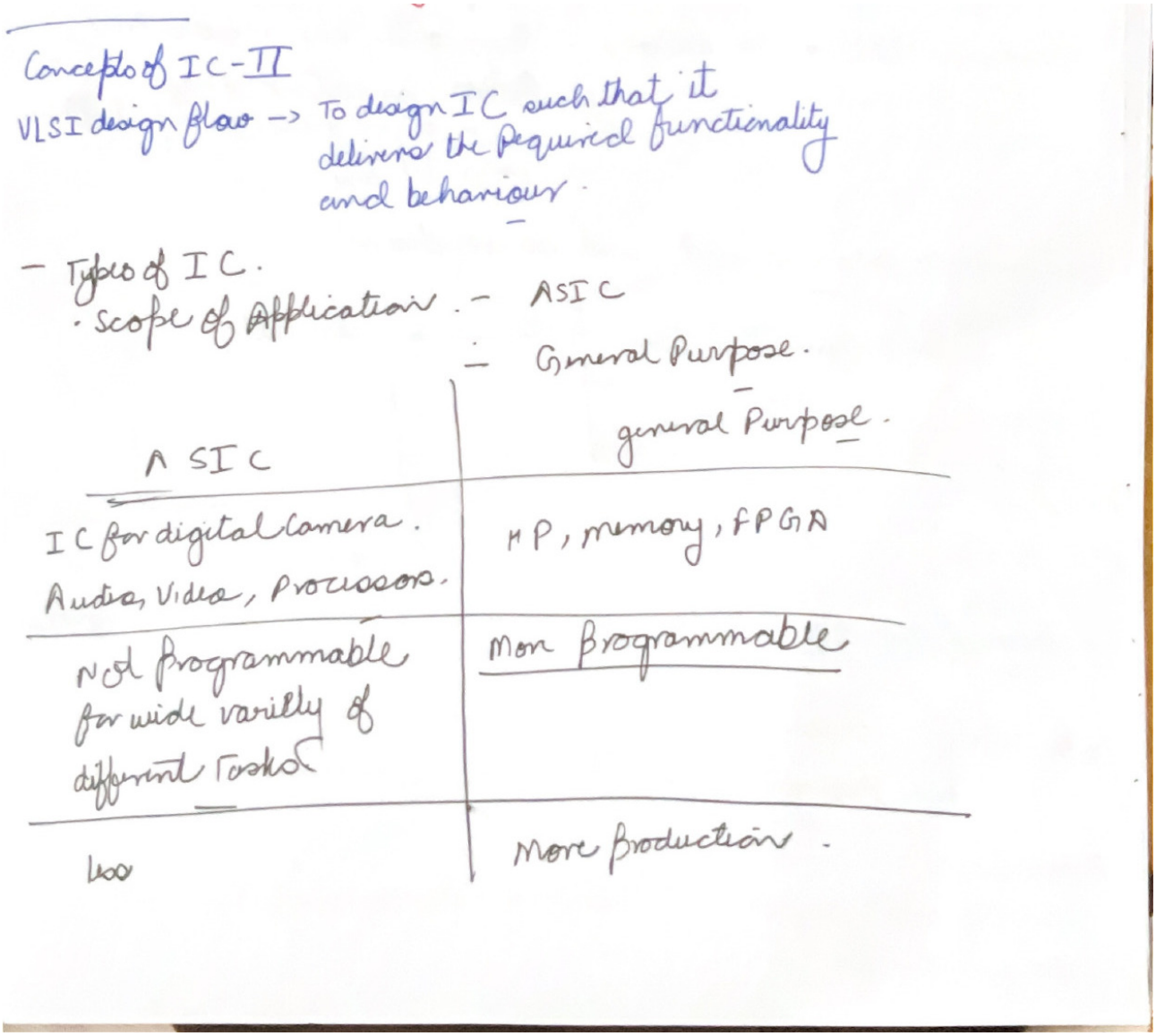

Figure 1.18. *Source: Part 1, PDF page 3, lower portion.*

The camera/audio-processing examples distinguish the product's intended function from the flexibility offered to its user. Read “less programmable” as “more of the function is fixed in hardware,” not “contains no programmable element.” Volume helps choose a cost-effective implementation, but it cannot by itself identify a chip as ASIC or general-purpose.

Four implementation styles

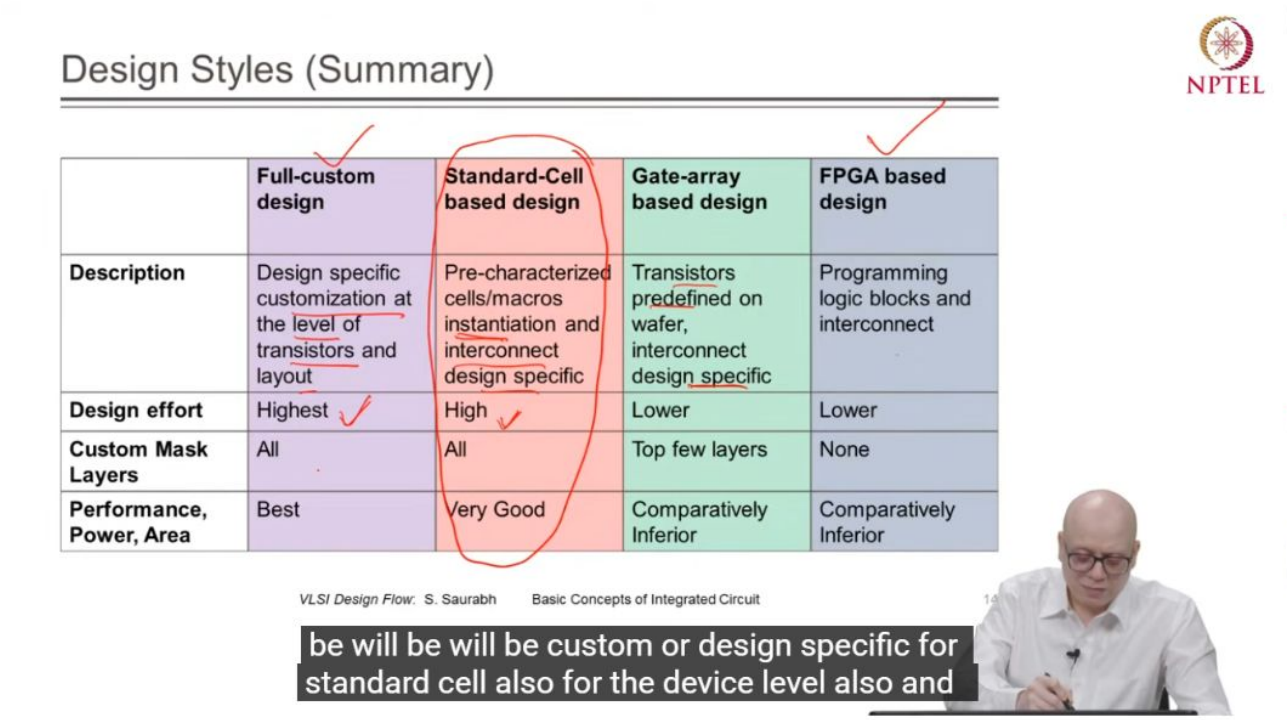

Figure 1.19. *Lecture: 18:53. Comparison of customization, design effort, masks, and PPA*

Style	What the designer customizes	What is reused	Main consequence
Full custom	Transistor circuits, sizes, and layout geometry	Process/device knowledge and selected existing blocks	Greatest freedom and substantial design and verification effort
Standard cell	Cell instances, connectivity, placement, and routing	Pre-designed, characterized logic cells and macros	Automation handles large digital designs efficiently
Gate array	Principally the interconnection of prefabricated devices	A regular base device array	Less device-level freedom; only selected mask layers are customized
FPGA	Configuration bits controlling logic and routing	The already fabricated programmable chip	Rapid reconfiguration; programmable fabric consumes area and delay

A **standard cell** is a library implementation of a small function, such as a NAND gate, multiplexer, buffer, or flip-flop. A family usually has compatible row heights and several widths and drive strengths. A **macro** is a larger block, such as an SRAM, whose physical shape need not match a single standard-cell row. The physical-design tool places instances of these blocks and routes their connections.

Full custom is useful where transistor-level choices matter strongly, including analog circuits and highly optimized blocks. Standard-cell design still fabricates a custom layout: reusing a cell's design does not mean the final wafer already contains the placed cells. An FPGA, by contrast, already exists

as silicon before the customer's design is loaded. Configuration selects functions and routes through that silicon; it does not create new metal layers.

The slide's PPA ranking is a broad comparison for implementing comparable functionality. It is not a promise that any full-custom design is better than any standard-cell design. Process, architecture, engineering quality, hard blocks, and constraints all matter.

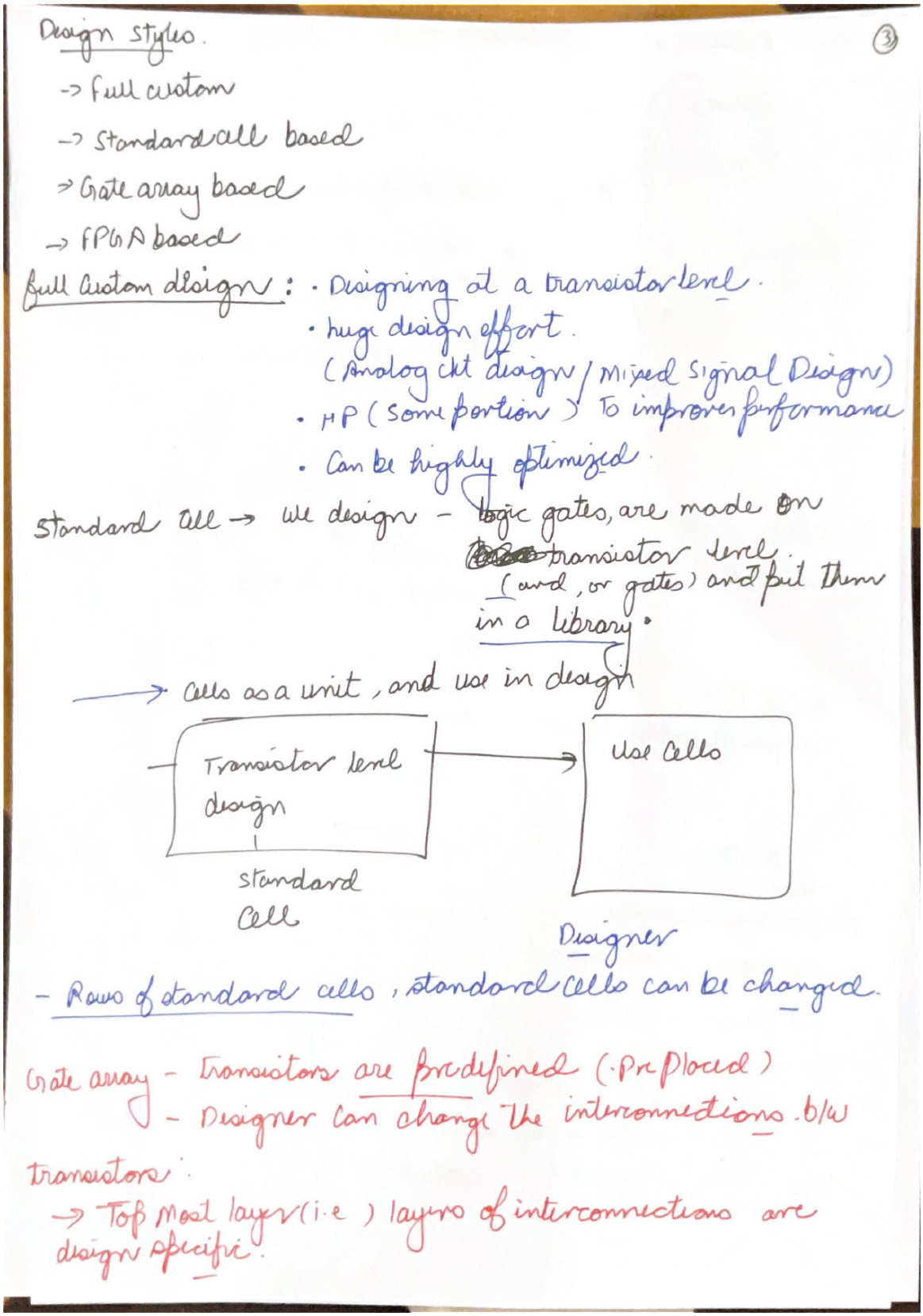

Figure 1.20. *Source: Part 1, PDF page 4.*

The note correctly separates cell creation from cell use. Ordinary standard-cell implementation selects and instantiates characterized cells. Changing a NAND's internal transistor sizes or layout would create a different cell that needs appropriate characterization and physical verification. Selecting `NAND2_X1` versus a stronger library variant is not the same as editing that cell's internal geometry.

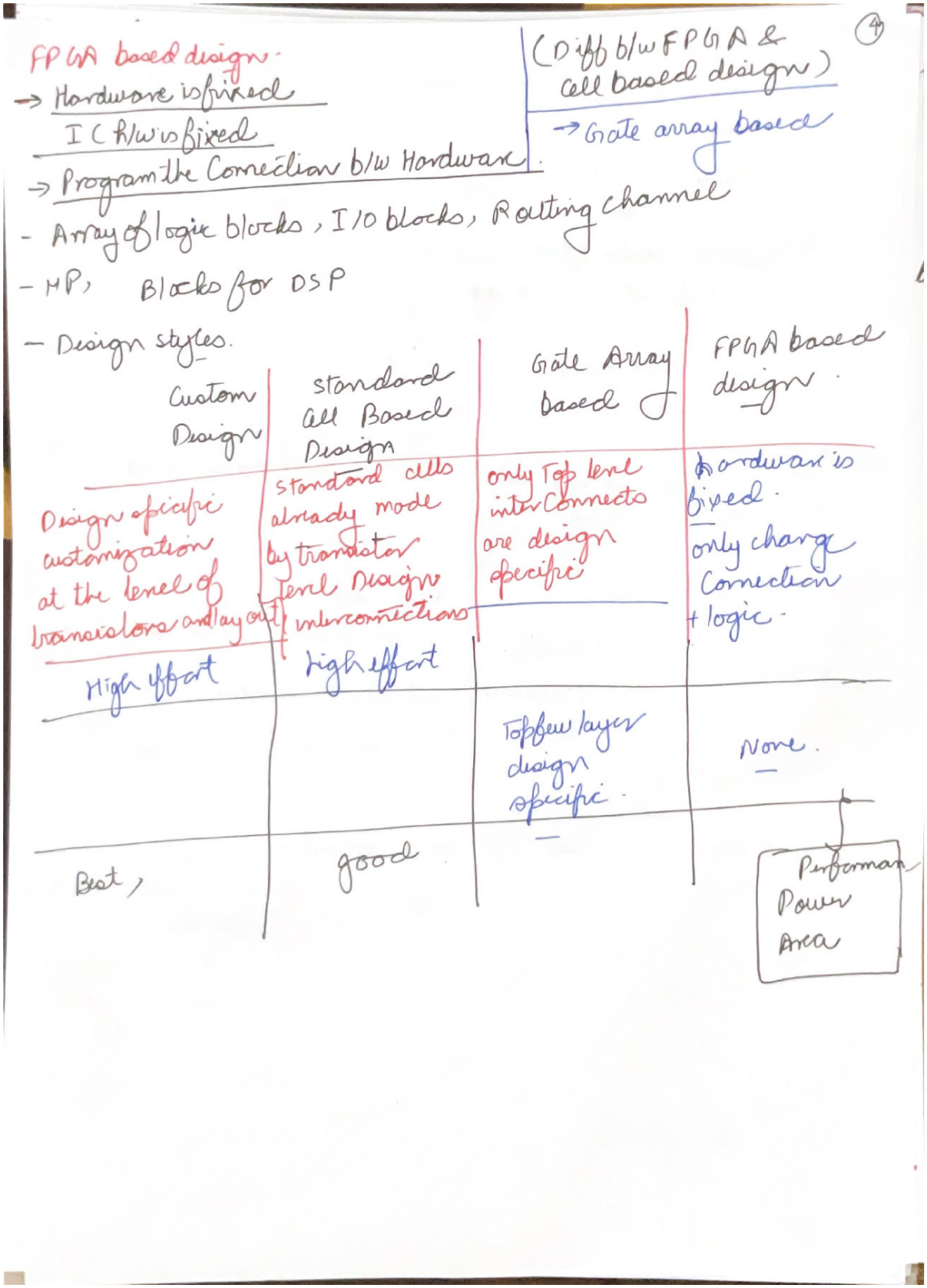

Figure 1.21. *Source: Part 1, PDF page 5.*

FPGA versus cell-based design

Both can start from RTL, but their targets differ. ASIC synthesis maps logic into the selected standard-cell library; FPGA synthesis and implementation map logic into lookup tables, flip-flops, routing switches, memories, DSP blocks, and other resources provided by that FPGA family. A lookup table stores a truth-table result for each input combination; programmable routing connects blocks. The FPGA user pays for available fabric, including unused capacity and configurability overhead. The ASIC designer pays substantial development cost but can tailor the fabricated implementation to the application.

Gate arrays are different again: base devices are fabricated in advance, but customization traditionally occurs through a manufacturing step for selected interconnect layers. FPGA configuration happens electrically after manufacturing. The **custom mask** distinction is all relevant custom layout layers for a conventional standard-cell ASIC, selected upper layers for a traditional gate array, and no customer-specific fabrication masks for FPGA configuration.

Economics and the break-even point

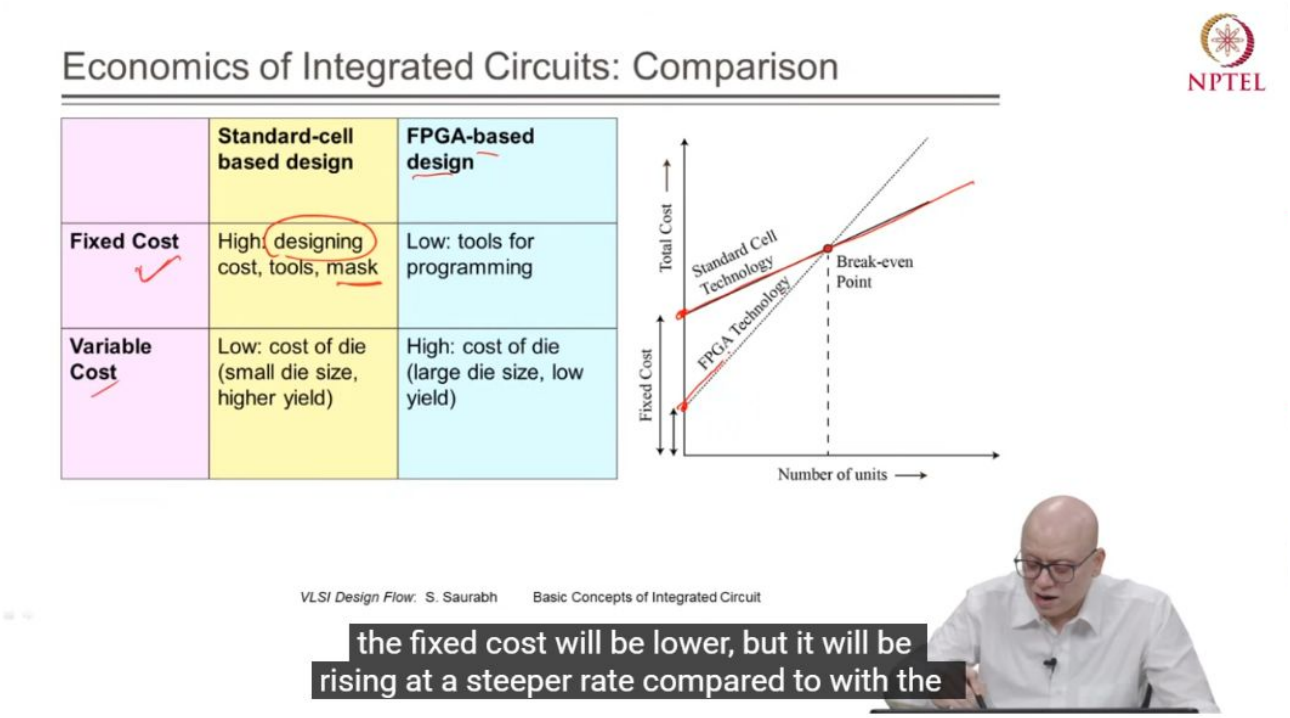

Figure 1.22. *Lecture: 27:52. Fixed cost, per-unit cost, and the break-even graph*

Fixed cost is development expenditure that is approximately independent of how many units are produced in a particular comparison: design work, verification, tools, and masks are examples. **Variable cost** grows with unit count: fabrication, packaging, test, and procurement contribute. Costs can be more complicated in practice, but a useful first model is

$$C_{\text{total}} = C_{\text{fixed}} + NC_{\text{unit}}.$$

The graph's vertical intercept is fixed cost; its slope is variable cost per unit. A standard-cell ASIC often has a larger intercept and smaller slope than an FPGA implementation of the same function. At low volume, avoiding ASIC development cost can dominate. At high volume, saving cost on every unit can repay that investment.

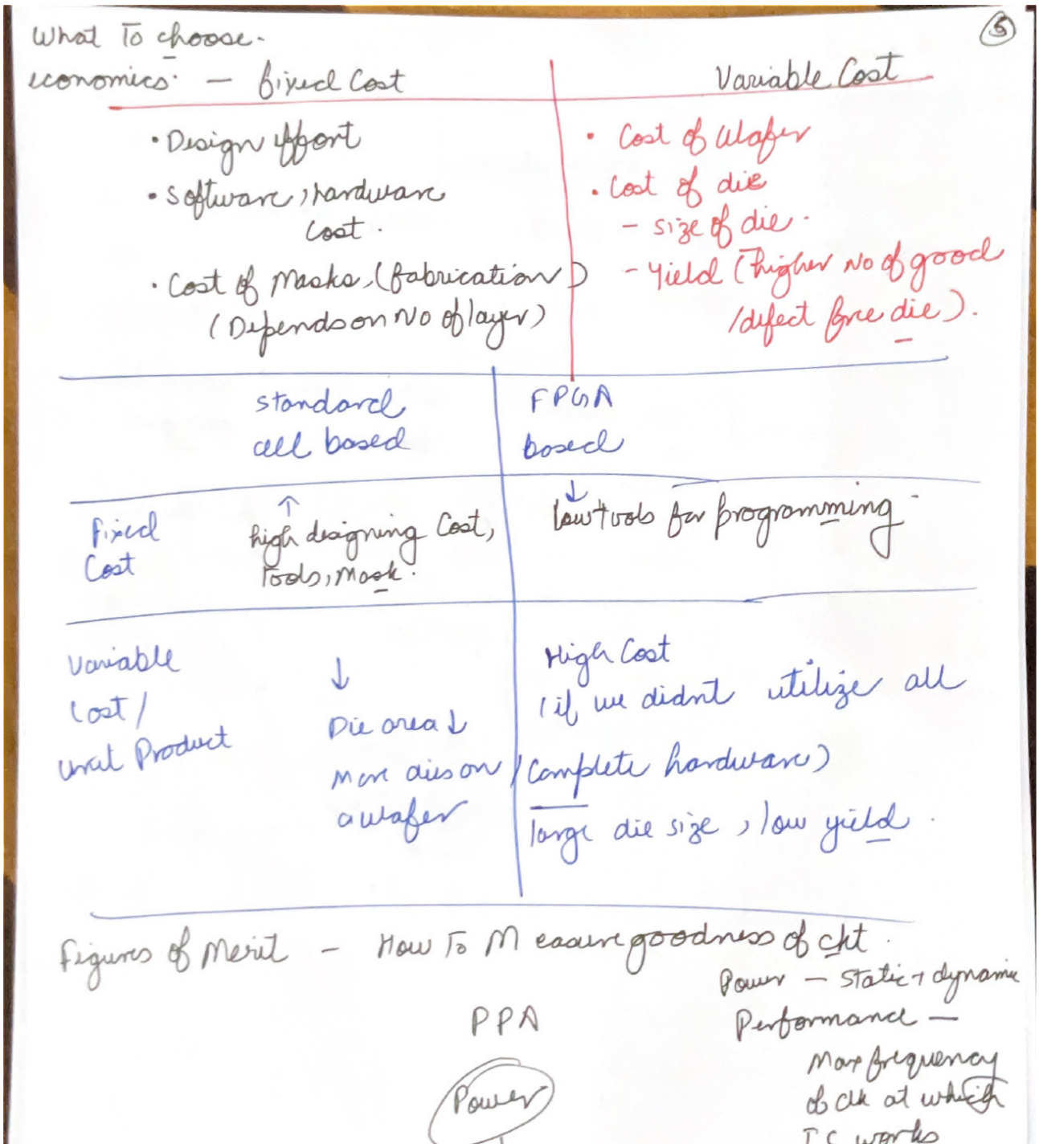

Figure 1.23. *Source: Part 1, PDF page 6, cost comparison.*

What should we choose?

Use required performance, power, volume, schedule, reprogrammability, and total cost together. The cheapest projected manufacturing cost is irrelevant if the design cannot meet the deadline or performance requirement. Conversely, choosing an expensive implementation only for a small speed benefit may be unjustified if both meet the specification.

For an **illustrative calculation**, let ASIC fixed cost be 10,000,000 cost units and unit cost 100; let FPGA fixed cost be 1,000,000 and unit cost 400. Equating total costs gives

$$N_{\text{break-even}} = \frac{10{,}000{,}000 - 1{,}000{,}000}{400 - 100} = 30{,}000.$$

Below 30,000 units, this simplified model favors FPGA cost; above it, ASIC cost. These are teaching numbers, not current price quotations. Mask respins, yield, inventory, engineering changes, and time value can shift the decision. In the slide's manufacturing model, larger die area reduces gross dies per wafer and can reduce yield; both can increase cost per good die. A commercial FPGA purchase price also includes many business factors beyond bare-die cost.

Carry die yield into cost per shipped unit

The handwritten cost graph assumes a known unit cost. To connect it to manufacturing, consider a separate teaching scenario: a wafer costs 6,000 cost units and contains 500 gross dies. Assume failed dies are discarded before packaging; packaging and final test cost 5 per good die, with no further losses. These assumptions deliberately omit edge-die estimation, test escapes and business margins.

Die yield	Good dies per wafer	Wafer cost per good die	Finished-unit cost
80%	400	6,000 / 400 = 15	15 + 5 = 20
50%	250	6,000 / 250 = 24	24 + 5 = 29

If a competing implementation costs 35 per unit and the ASIC needs 9,000,000 more fixed investment, break-even volume is 600,000 units at cost 20, but 1,500,000 at cost 29. The same design investment now takes 2.5 times as many shipments to recover. The denominator is the **saving per shipped unit**, not the gross fabrication cost of one die.

This also explains why reducing die area can help twice: it can increase gross dies per wafer and improve defect-limited yield. Those effects require an area/yield model; neither follows from changing an RTL gate count alone. At an ASIC unit cost of 35 or more, these assumptions provide no positive-volume cost crossover. Recompute the graph when yield or packaging losses change.

PPA and the rest of design quality

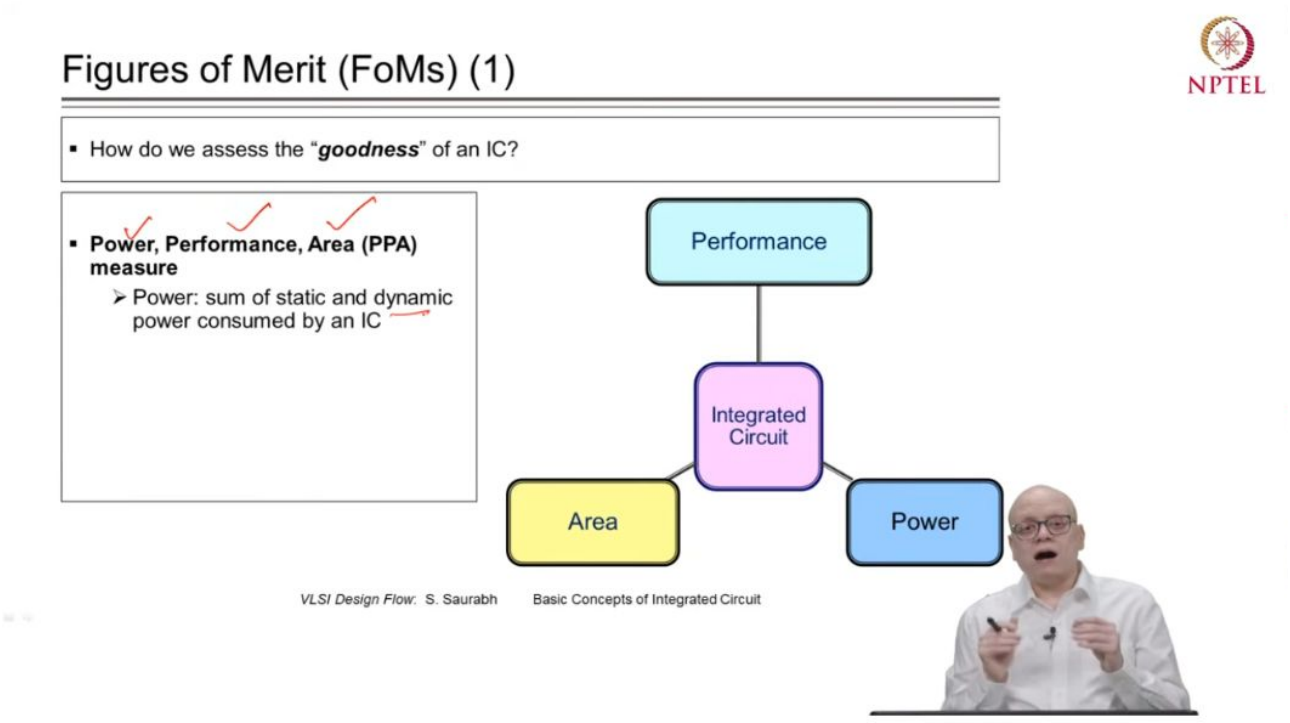

Figure 1.24. *Lecture: 30:13. Power, performance, and area as competing objectives*

Power is the rate of energy consumption. Dynamic power includes switching-related consumption; static power includes leakage while the logical state is unchanged. **Performance** must be specified as a useful metric: clock frequency, operations per second, response latency, or another application measure. **Area** can mean the total cell area, core area, or die area; these are different quantities and must be named.

For example, a higher clock frequency does not guarantee lower end-to-end latency if an implementation requires many more cycles. A small cell-area result may still require a larger core to allow routing. Increasing drive strength may improve timing while increasing input capacitance, area, and power. An improvement in one figure can therefore worsen another.

Worked example: power is not energy per operation

Charging and discharging capacitance consumes energy. A useful model for the capacitive switching component is

$$P_{\text{switch}} = \alpha C V^2 f.$$

Here C is the load capacitance being modeled, V is supply voltage, f is clock frequency, and α counts average **zero-to-one charging events per clock cycle**. This convention avoids an extra factor of two associated with counting both transition directions. For many nodes, sum their contributions. The model excludes leakage and short-circuit current; Intel's microarchitecture white paper, page 5 describes the capacitance, voltage-squared, and switching-frequency dependence.

For an illustrative aggregate load of 10 pF, $\alpha = 0.2$, $V = 1$ V, and $f = 500$ MHz, switching power is 1 mW. Reducing voltage to 0.8 V gives 0.64 mW **if the same frequency remains feasible**. Voltage changes can also change delay.

Energy for a task is $E = P_{\text{average}} t$. At 1 mW, a 1,000-cycle task lasting 2 µs consumes 2 nJ of switching energy. Halving frequency halves this modeled power but doubles runtime, leaving switching energy at 2 nJ under unchanged activity, voltage, and cycle count. Leakage energy can increase with the longer runtime. State both the workload and the metric when comparing designs.

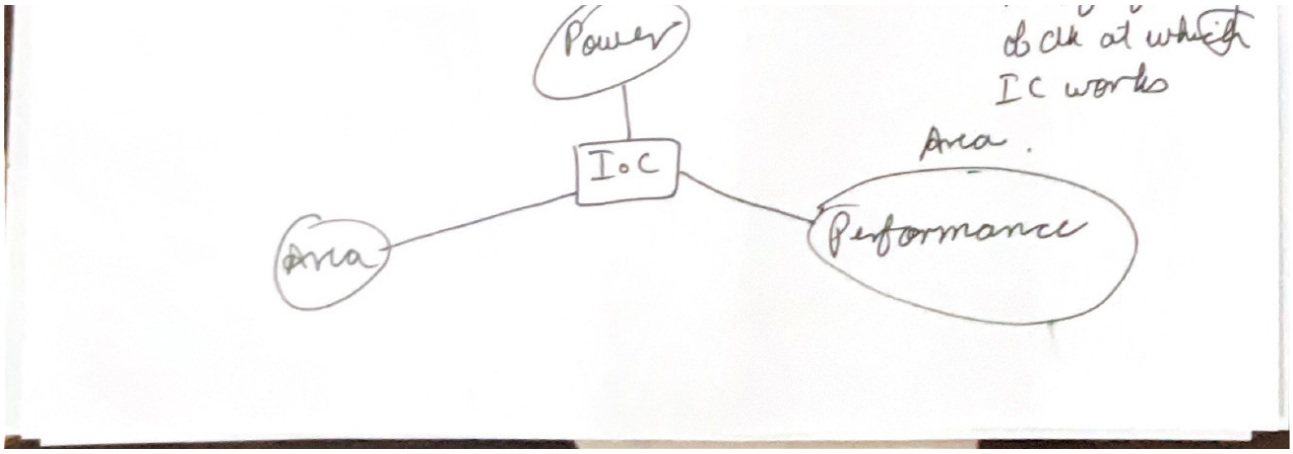

Figure 1.25. *Source: Part 1, PDF page 6, PPA note.*

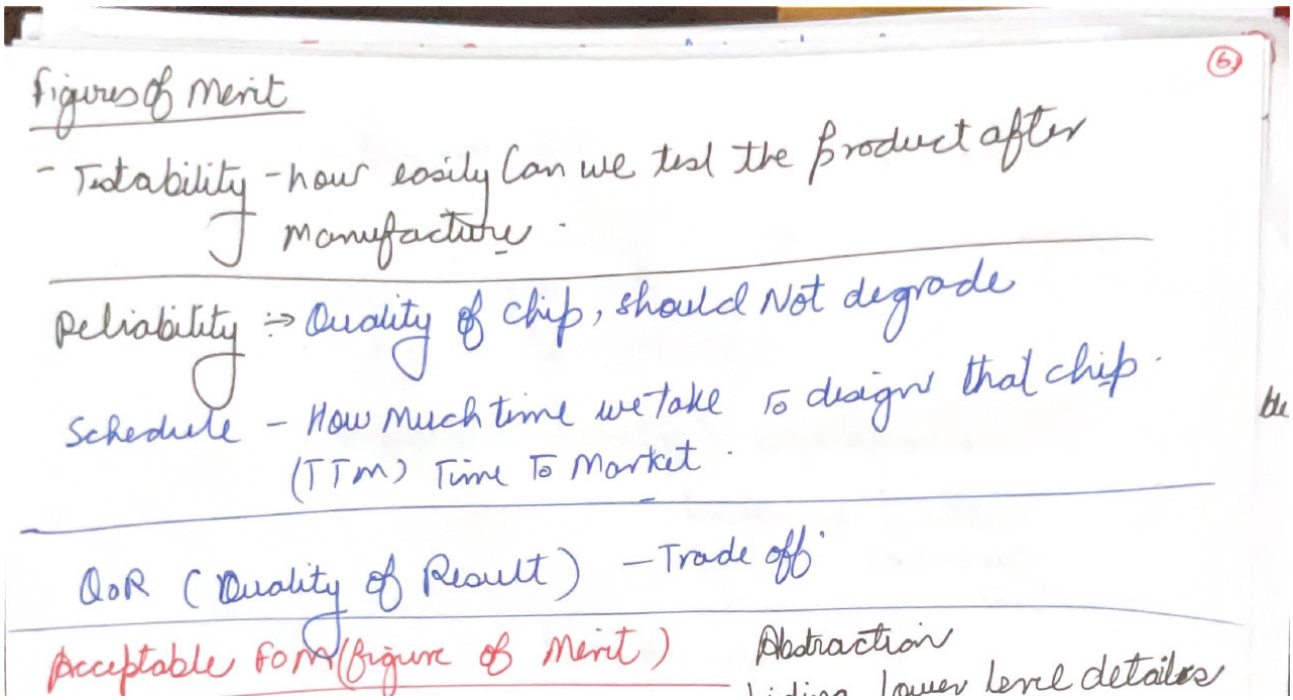

Figure 1.26. *Source: Part 1, PDF page 7, upper portion.*

The added measures are essential: **testability** is how readily manufacturing faults can be controlled and observed; **reliability** concerns correct operation over the required lifetime and conditions; **time to market** is the schedule for delivering a usable product. **Quality of results (QoR)** is judged against the chosen objectives and constraints. A feasible design meets the constraints; an optimal design is best under a precisely defined objective and search space. Large design problems usually use heuristics, so finding an acceptable result does not prove a global optimum.

Recall check: if design A uses less cell area but cannot route within its core, while design B is slightly larger and meets timing after routing, which one actually satisfies the specification? Explain why the answer requires physical implementation evidence, not just the synthesis area report.

Definitions and mechanisms in Lesson 02

Application and implementation style. An application describes the problem a product serves. An implementation style describes how its circuit is realized. “Application-specific” and “general-purpose” classify intended use; “full custom,” “standard cell,” “gate array” and “FPGA” classify construction or configuration choices. An application-specific system may contain a programmable processor. An FPGA may perform one specialized task while remaining a manufactured programmable device. State both classifications when discussing a product, because one does not uniquely determine the other.

Programmability and configurability. A processor changes behavior by executing different instruction sequences on its existing architecture. An FPGA configuration selects functions and interconnections in an existing programmable fabric. Neither operation fabricates new transistors. A standard-cell ASIC can also contain configurable registers and software-controlled blocks, but its fixed physical circuit does not become an FPGA merely because some behavior is programmable. The useful question is which decisions can change after manufacture and what area, timing or power cost supports that flexibility.

Nonrecurring cost and unit cost. Nonrecurring engineering cost is expenditure associated with developing and preparing a design, approximately independent of unit count in a simple model. Unit

cost is the incremental cost associated with supplying another part. Their sum over a production volume produces total cost. Break-even is the volume at which two total-cost models are equal; it is a comparison under stated assumptions, not a universal boundary between ASIC and FPGA. A technically infeasible option cannot become acceptable merely by being cheaper on that graph.

Power and energy. Power measures the rate of energy transfer or consumption. Energy measures the amount consumed during a task or interval. For a constant average power, energy equals power multiplied by duration. A lower-power implementation can consume more task energy if it takes sufficiently longer. Dynamic switching energy depends on transitions and capacitance; leakage continues over time. A meaningful comparison therefore identifies the workload, voltage, activity, duration and the included power components. Saying only “this circuit consumes less” leaves the metric undefined.

Latency, throughput and clock frequency. Latency measures the delay of a particular transaction between defined events. Throughput measures how many transactions complete per unit time over an interval, often in steady state. Frequency measures clock cycles per second. These quantities connect through architecture and protocol, but they are not interchangeable. A pipeline can increase clock frequency and sustained throughput while adding stages that increase the latency of one item. Always name the acceptance and completion events when reporting a latency.

Constraint, objective and quality of results. A constraint sets an acceptable boundary, such as a maximum delay. An objective ranks feasible alternatives, such as minimizing area while meeting that delay. Quality of results describes how the implementation performs on the chosen measures. A design that misses a hard timing requirement is infeasible even if its area is impressive. A result that meets all requirements is feasible, but proving it globally optimal requires a defined search space and stronger evidence than a successful tool run.

Application and validation in Lesson 02

Implementation alternatives are compared with the required function, workload and performance constraints held constant. Development expenditure and per-unit expenditure enter different terms of total cost. In the stated example, equality at 30,000 units follows from both complete cost expressions, rather than from comparing unit prices alone.

Sensitivity to a respin. If an additional ASIC development/mask expense of 3,000,000 cost units is introduced while the FPGA costs and both unit costs remain unchanged, the fixed-cost difference grows from 9,000,000 to 12,000,000. The break-even volume becomes $12{,}000{,}000/300 = 40{,}000$ units. At 35,000 units the original model favors the ASIC, whereas the revised model favors the FPGA. Feasibility must be checked before selecting the lower-cost alternative: a cost advantage does not resolve a missed timing, power or delivery requirement.

Complete original notebook pages for Lesson 02

Part 1 original page 04

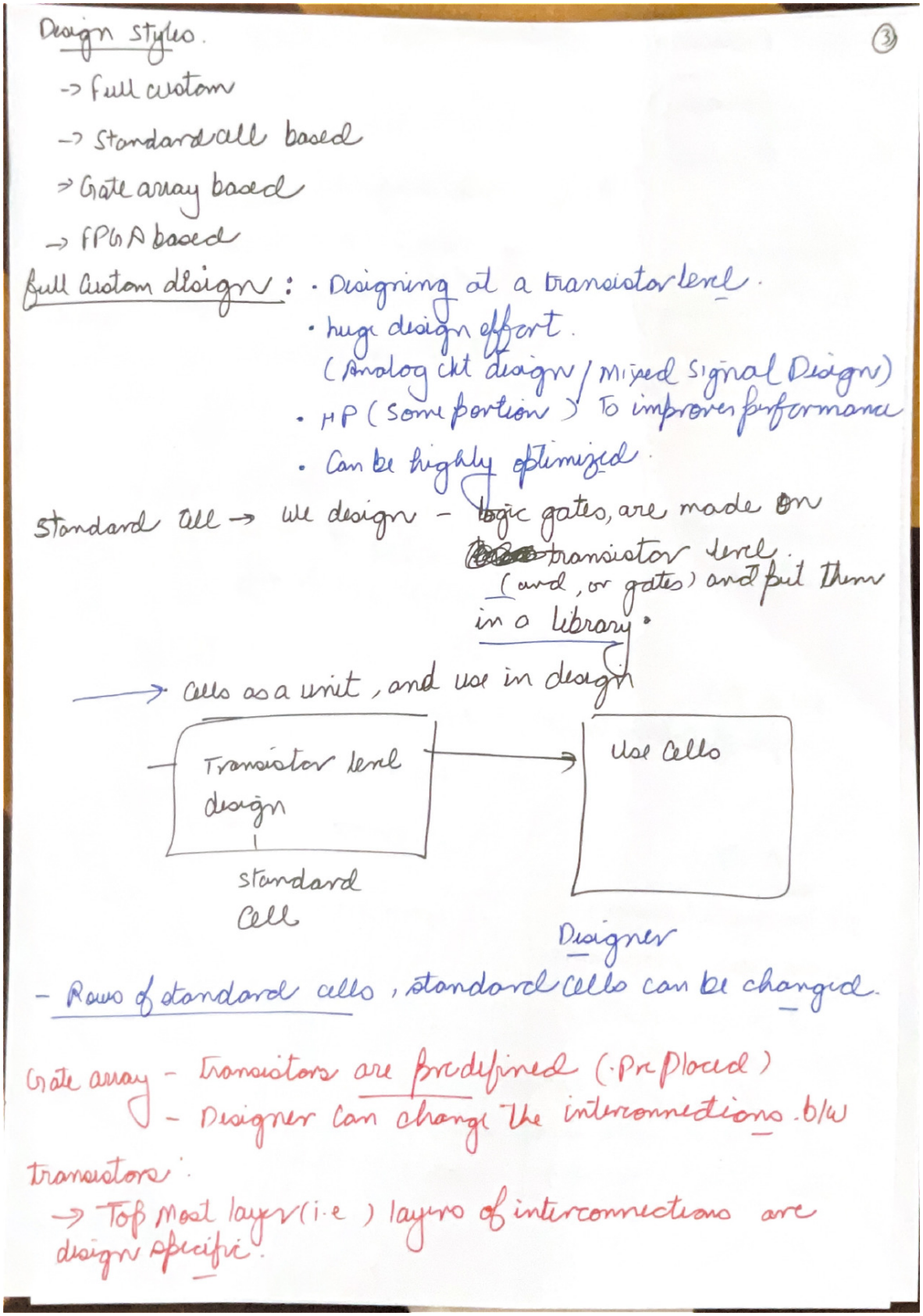

Figure 1.27. *Source: Part 1, PDF page 4. Full custom, standard cells, gate arrays.*

Explanation and worked examples: Day 1 · Lesson 02.

Part 1 original page 05

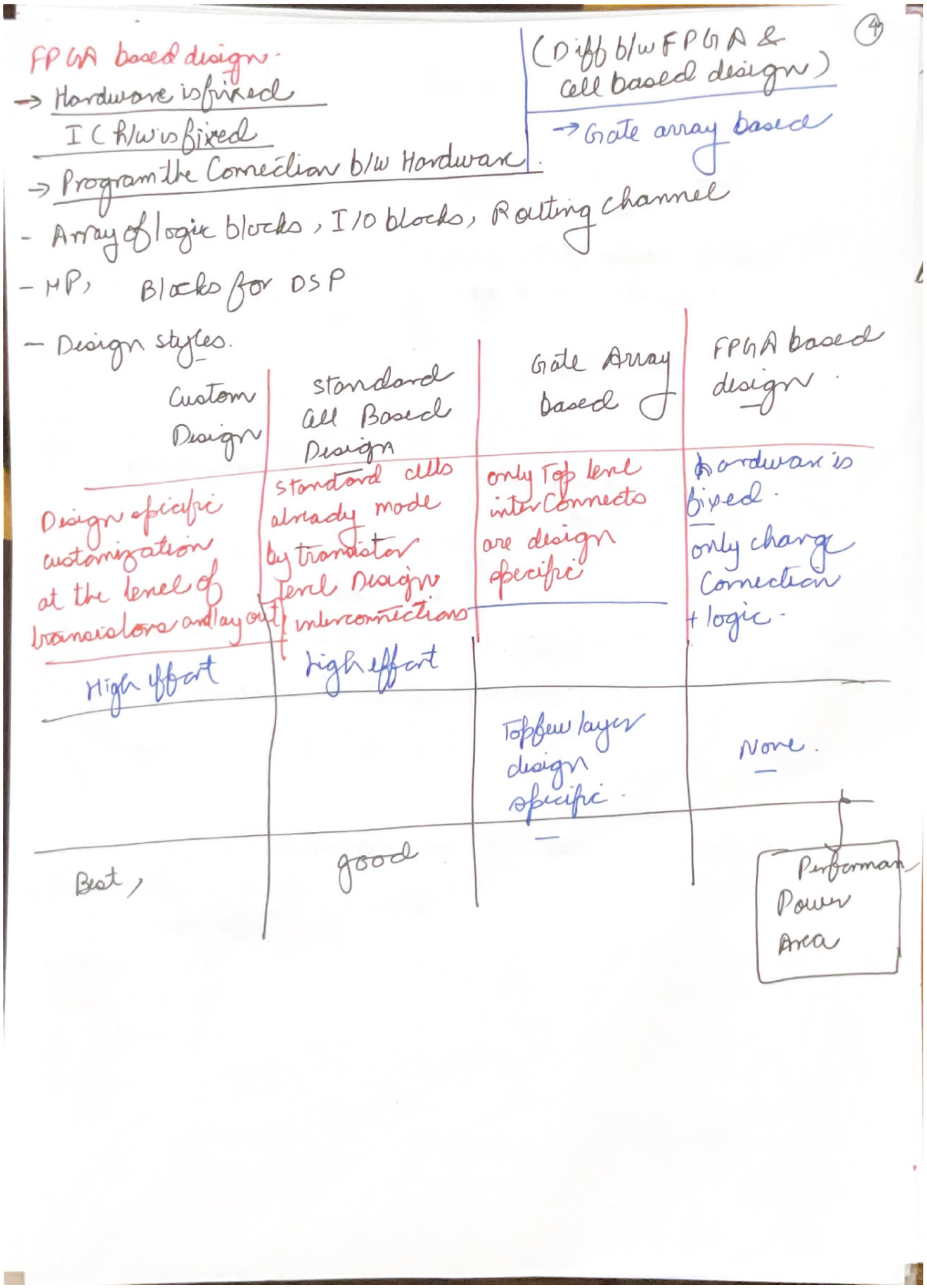

Figure 1.28. *Source: Part 1, PDF page 5. FPGA versus cell-based design and comparison table.*

Explanation and worked examples: Day 1 · Lesson 02.

Part 1 original page 06

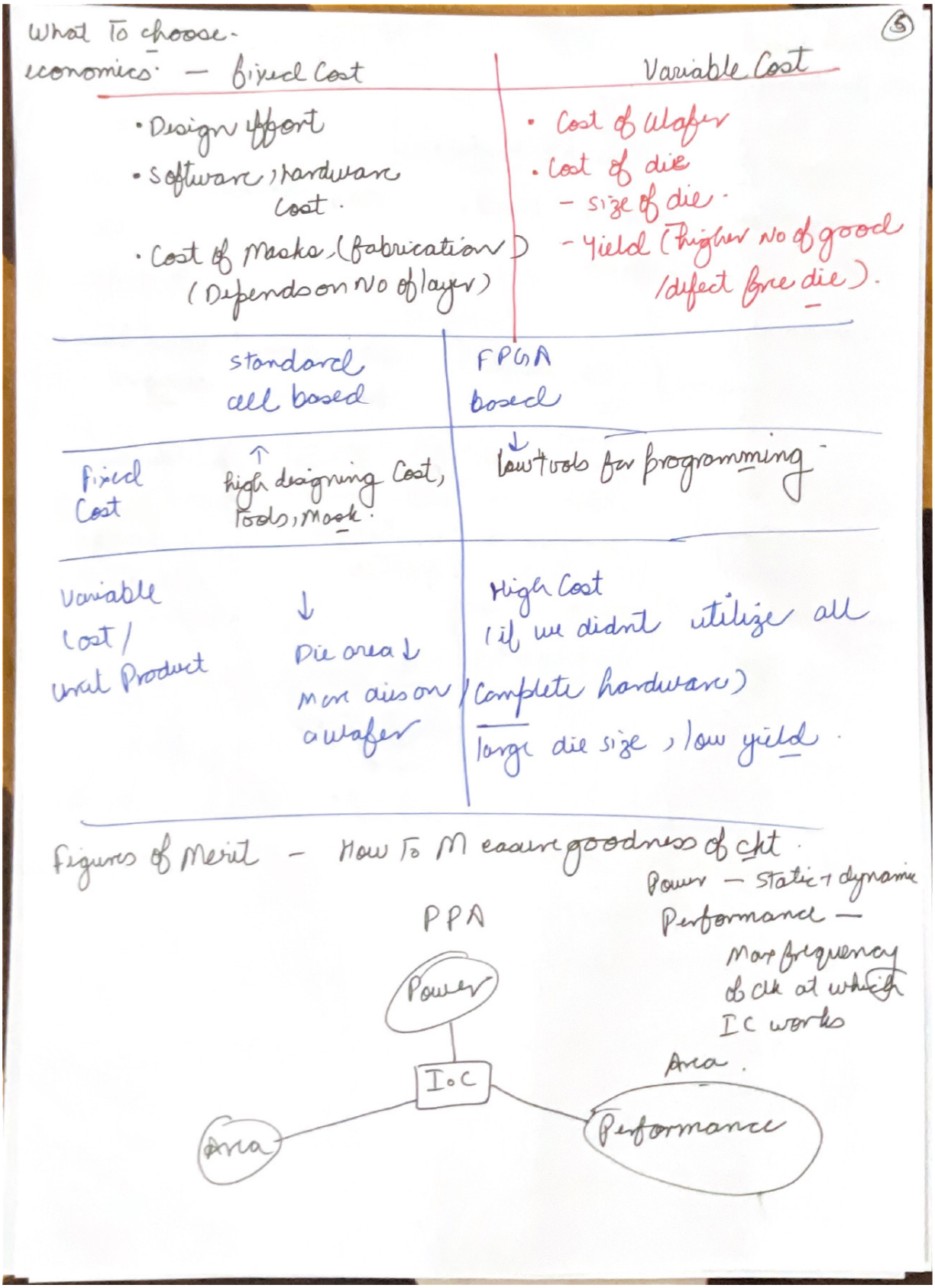

Figure 1.29. *Source: Part 1, PDF page 6. Choosing a design style, fixed/variable cost, PPA.*

Explanation and worked examples: Day 1 · Lesson 02.

Part 1 original page 07

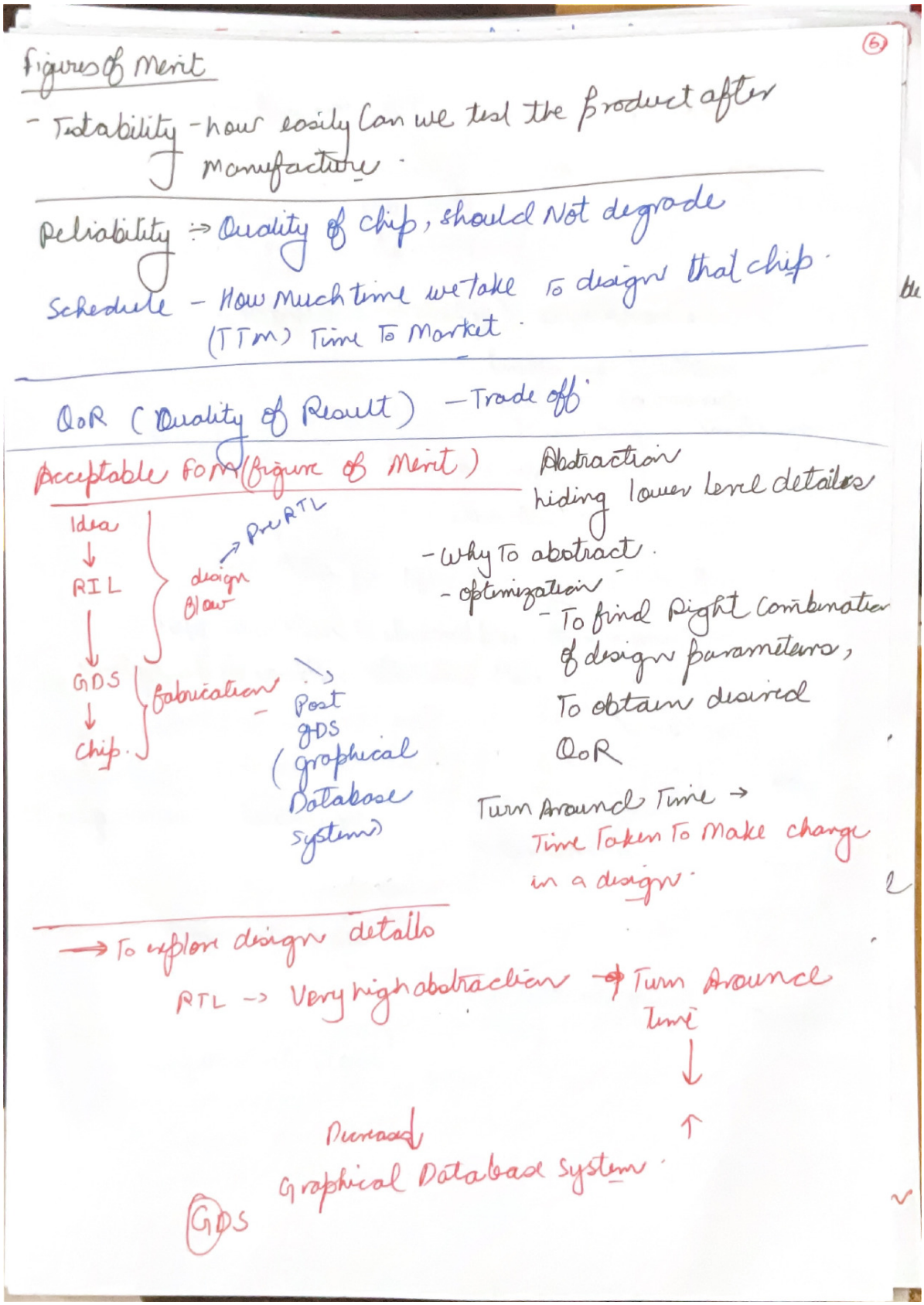

Figure 1.30. *Source: Part 1, PDF page 7. Quality measures and abstraction/design flow.*

Explanation and worked examples: Day 1 · Lesson 02 · Day 1 · Lesson 03.

Lesson 03 Architecture and the design flow

Week 1 · Lecture video

From a product idea to a manufactured chip

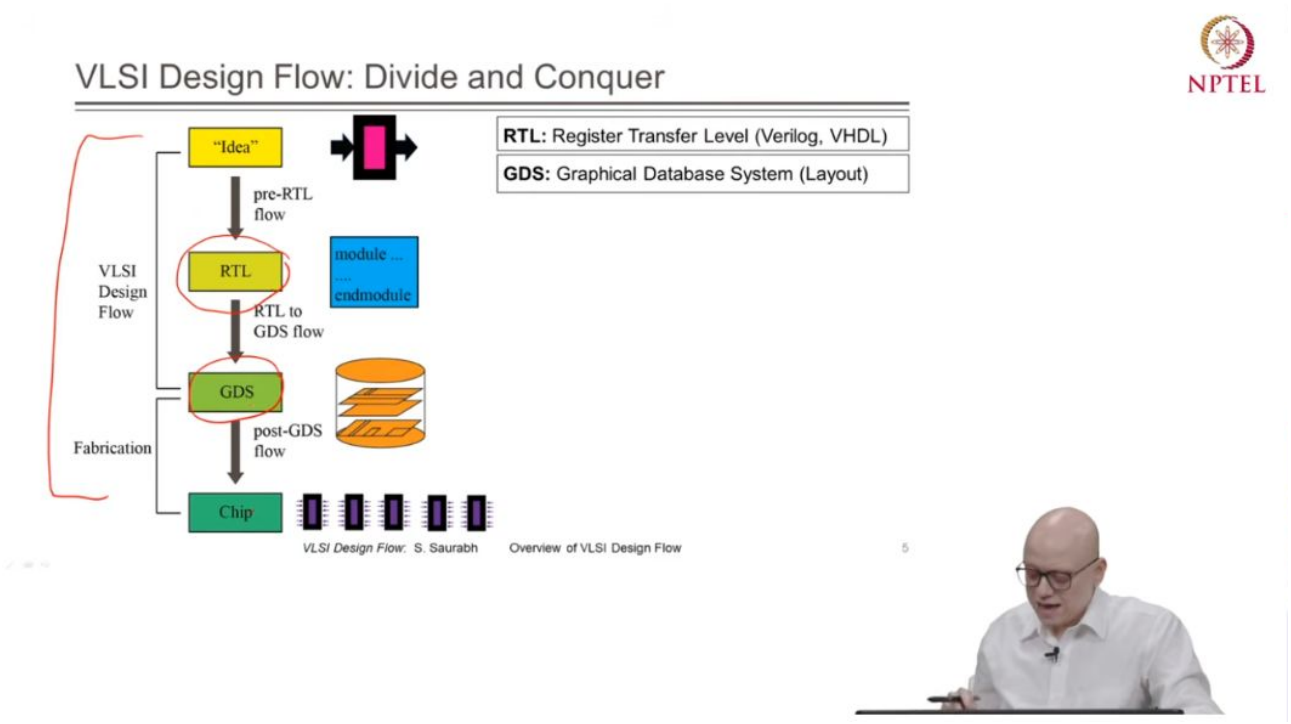

Figure 1.31. *Lecture: 4:43. Pre-RTL, RTL-to-GDS, and post-GDS stages*

The flow translates a desired behavior into successively more concrete representations. **Pre-RTL design** establishes requirements, chooses algorithms and architecture, partitions hardware and software, and defines cycle-level hardware behavior. **RTL-to-GDS** develops a gate-level implementation and then its physical layout. **Post-GDS** prepares manufacturing data, fabricates, tests, and packages the chip.

RTL, register-transfer level, models stored state and the logic that transforms data between storage elements. **GDSII** is a layout-data format describing hierarchical geometry and layers; it is not executable software for a processor. The manufacturing flow uses layout data to prepare patterns and process wafers. These stages are connected by verification and feedback, not a one-way chain that always succeeds on its first attempt.

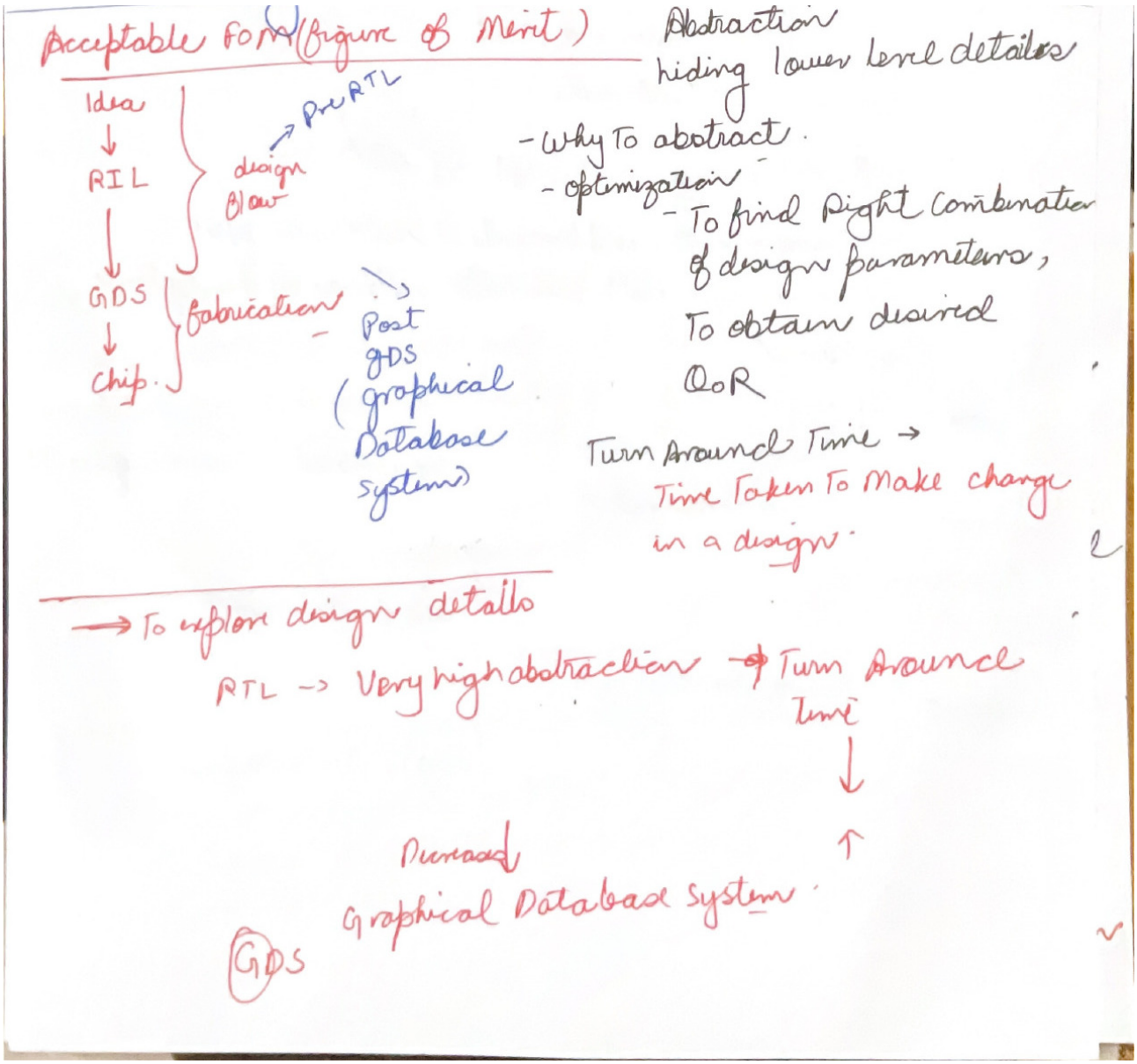

Figure 1.32. *Source: Part 1, PDF page 7, lower portion.*

The idea → RTL → GDS → chip chain is the backbone of the course. The “design” bracket includes decisions before RTL and physical implementation after synthesis. Fabrication begins after design-data handoff, but manufacturability must influence design much earlier through the PDK and physical checks.

Abstraction and turnaround time

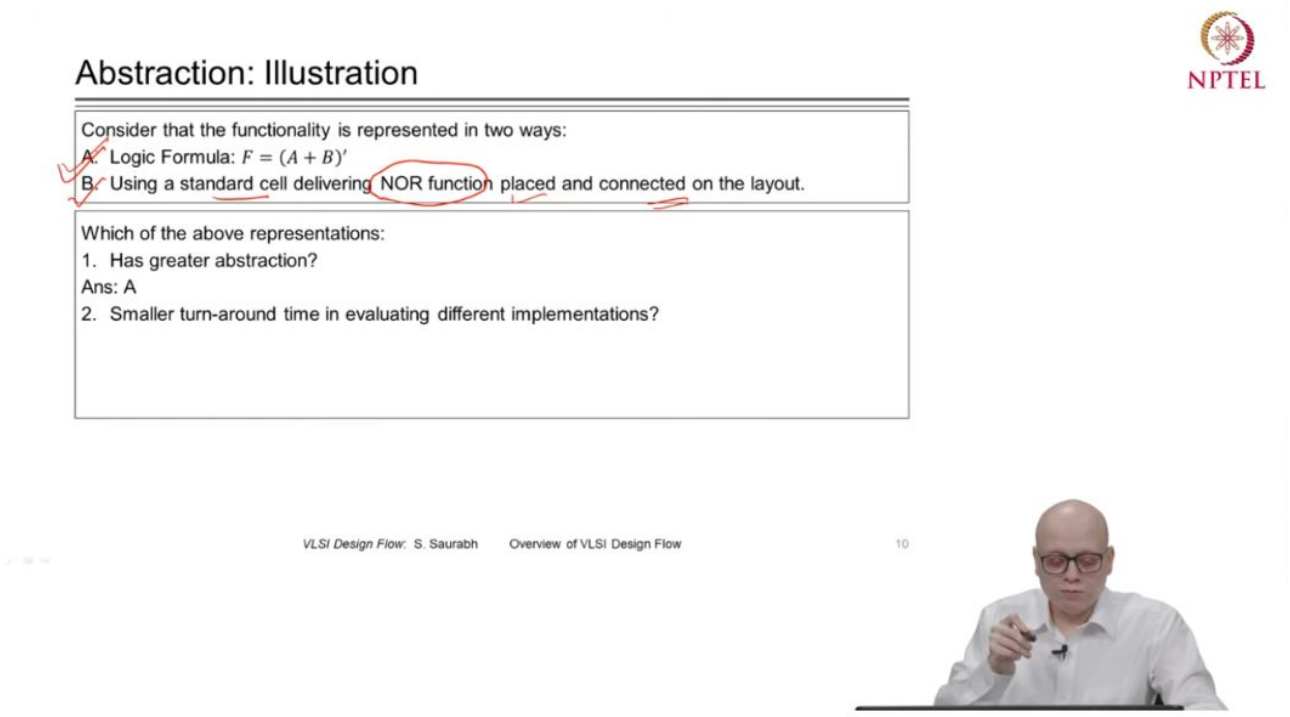

Figure 1.33. *Lecture: 14:10. The same NOR function described as an equation or a placed cell*

Abstraction is selective omission of detail so a representation exposes the properties needed for a particular decision. The Boolean equation $F = \overline{A + B}$ specifies the NOR truth function. It does not choose transistor sizes, a cell variant, a placement coordinate, or metal routes. A placed and connected NOR instance contains more implementation detail and supports more physically accurate analysis.

A high-level representation lets us compare architectural alternatives quickly. Replacing one algorithm or pipeline organization before layout may be inexpensive; making the equivalent change after placement and routing can require extensive reimplementation. **Turnaround time** is the time needed to complete an iteration. Lower abstraction often increases analysis cost, but it also exposes effects that the higher model hides. Neither level replaces the other: use a simple model to explore and a detailed model to validate.

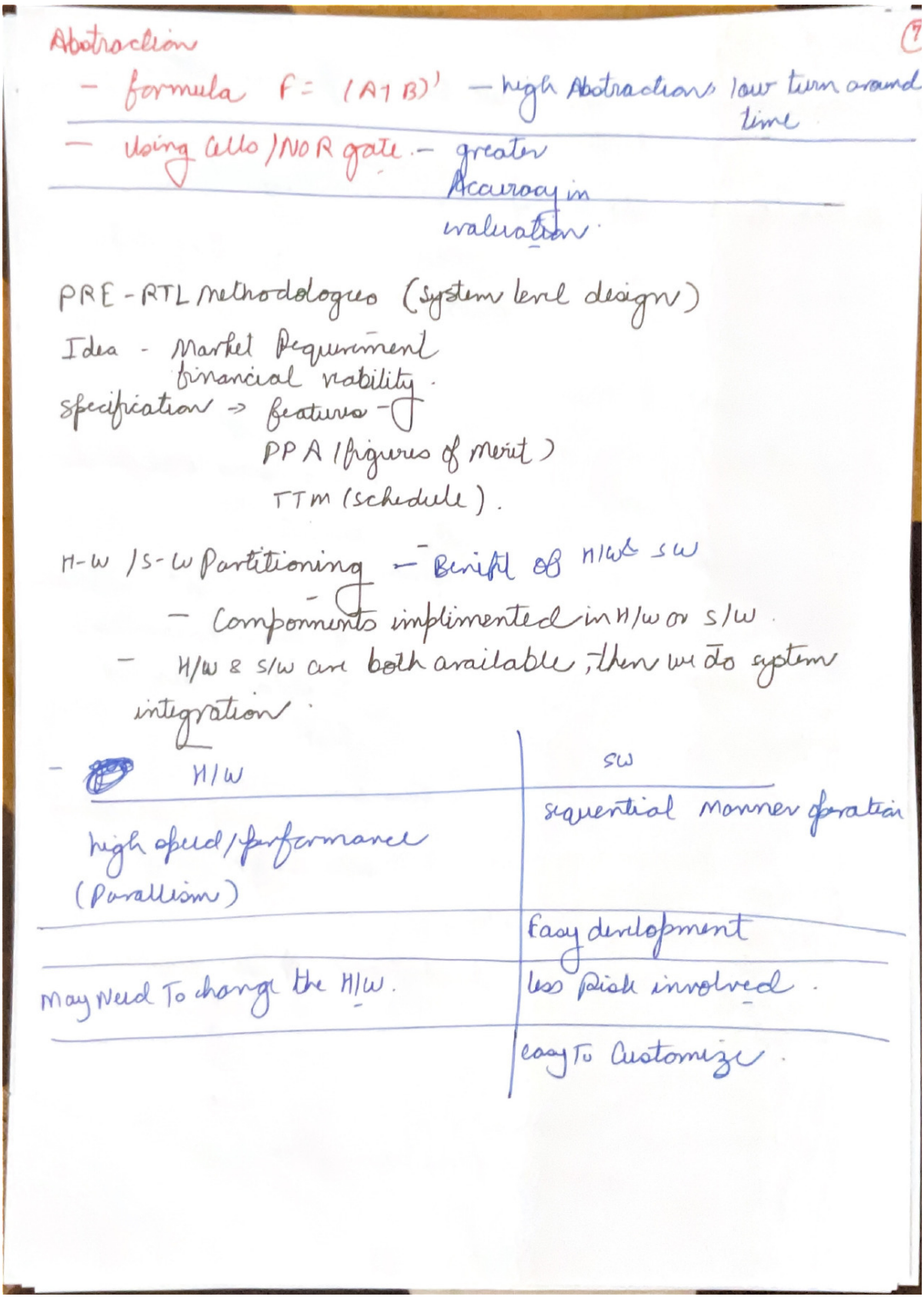

Figure 1.34. *Source: Part 1, PDF page 8.*

The NOR example correctly identifies the equation as the more abstract representation. The claim "hardware is parallel and software is sequential" is a useful introductory contrast, but it needs qualification. Hardware can contain serial dependencies and shared resources; software can use

multiple cores, vectors, and threads. The architectural question is which implementation gives the required behavior, cost, flexibility, and performance under the actual workload.

Before partitioning, write measurable requirements: input/output formats, supported functions, maximum response time, throughput, power budget, and operating conditions. Market and schedule constraints help decide whether a technically possible design is a useful product. A vague requirement such as “very fast” cannot guide a partitioning algorithm.

Hardware and software partitioning

Trace an accelerator request and calculate its communication cost

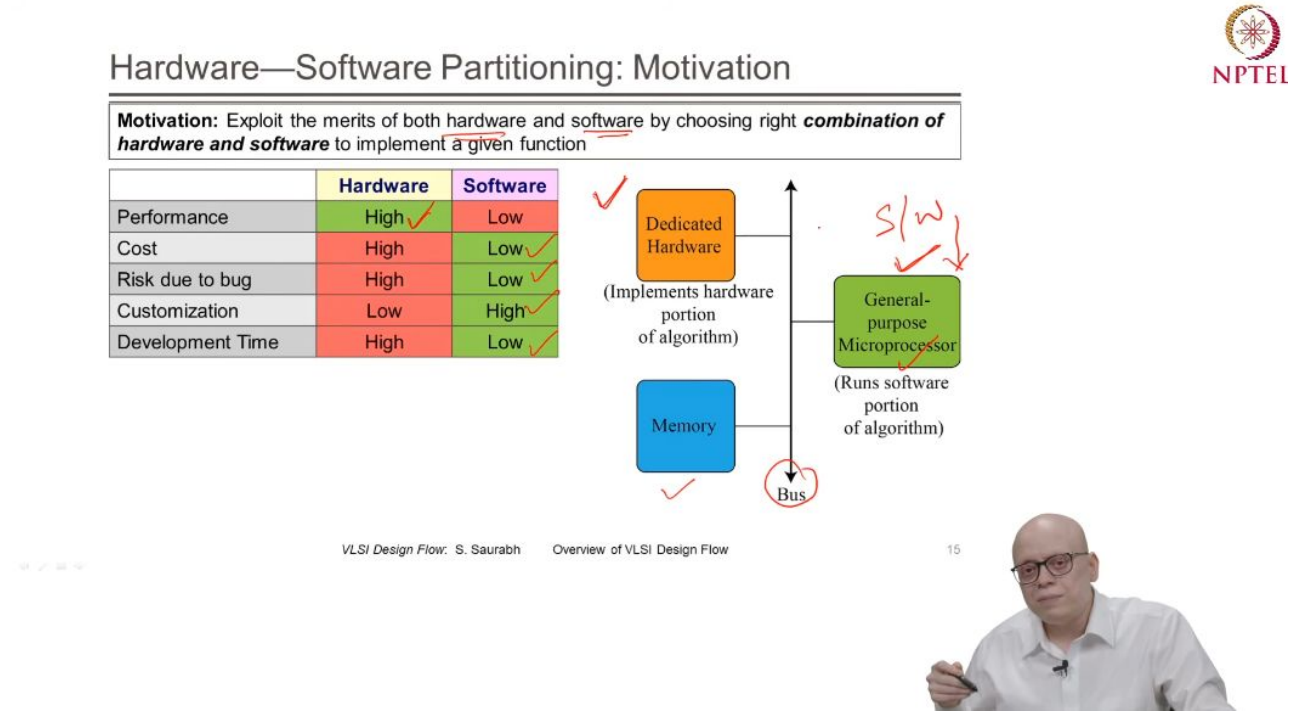

Figure 1.35. *Lecture: 26:20. Hardware/software partitioning: the processor, accelerator, memory, and bus.*

Read the three boxes as different roles. The **processor** runs instructions implementing the software portion. The **accelerator** implements a selected operation using dedicated hardware. **Memory** holds instructions, input data, intermediate values, and results as required by the architecture. The **interconnect** carries transactions between these components. A bus drawn as one line is a connectivity abstraction; it does not imply unlimited bandwidth or zero communication delay.

An accelerator request needs more than arithmetic. Software identifies the input and operation, makes data available, starts the work, detects completion, and consumes the result. A **control register** can hold a command or mode; a **status register** can report readiness or completion. These are examples of interfaces that implement the diagram's arrows, not a particular bus protocol specified by the slide. A result must belong to the intended request, so synchronization is part of correctness as well as performance.

The comparison table summarizes common tradeoffs: fixed hardware can be expensive to change after fabrication, while processor software can often be revised without changing silicon. Its “high” and “low” labels are qualitative comparisons under that context, not universal numerical properties of every ASIC, FPGA, or software implementation.

Suppose one operation takes 100 µs in software. An accelerator computes it in 5 µs, needs 3 µs of control overhead, and transfers 64 KiB of input plus 16 KiB of output. One KiB is 1,024 bytes, giving 81,920 bytes total. Assume a single request, serial transfers and computation, and sustained usable bandwidth B:

$$T_{\text{accelerated}} = T_{\text{control}} + \frac{N_{\text{in}} + N_{\text{out}}}{B} + T_{\text{compute}}.$$

Sustained bandwidth	Transfer time	Total request time	Whole-request speedup
2.0 GB/s	40.96 µs	48.96 µs	$100/48.96 \approx 2.04$
0.5 GB/s	163.84 µs	171.84 µs	$100/171.84 \approx 0.582$; slower than software

Here GB/s uses 10^9 bytes/s. Although computation alone is 20 times faster, communication can erase that gain. Under these assumptions, acceleration helps only if $B > 81920/(92 \times 10^{-6})$, approximately 0.890 GB/s. This follows by requiring total request time below 100 µs. Overlapping transfers with computation, reusing nearby data, and batching requests can change the model; their benefit depends on the actual protocol and workload. Use sustained bandwidth, not an unjustified advertised peak.

The next frame shows how the course uses repeated measurement to choose functions for hardware. Read the handwritten architecture below with these communication costs in mind.

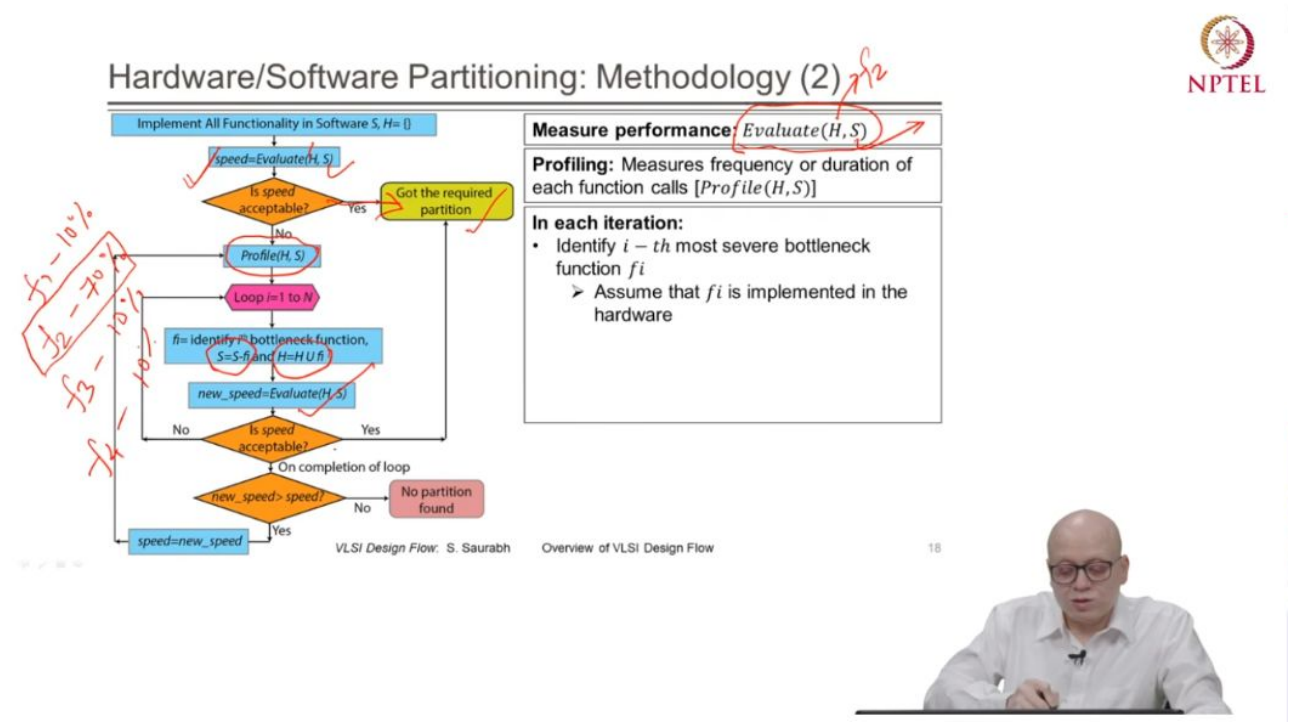

Figure 1.36. *Lecture: 37:48. Profile bottlenecks, move functions into hardware, and evaluate again*

Hardware/software partitioning assigns system functions to dedicated hardware or programmable software. A common architecture combines a processor, memory, an interconnect, and one or more accelerators. The accelerator can exploit specialized datapaths and parallelism; software handles control, changing policies, or functions where dedicated hardware brings little benefit.

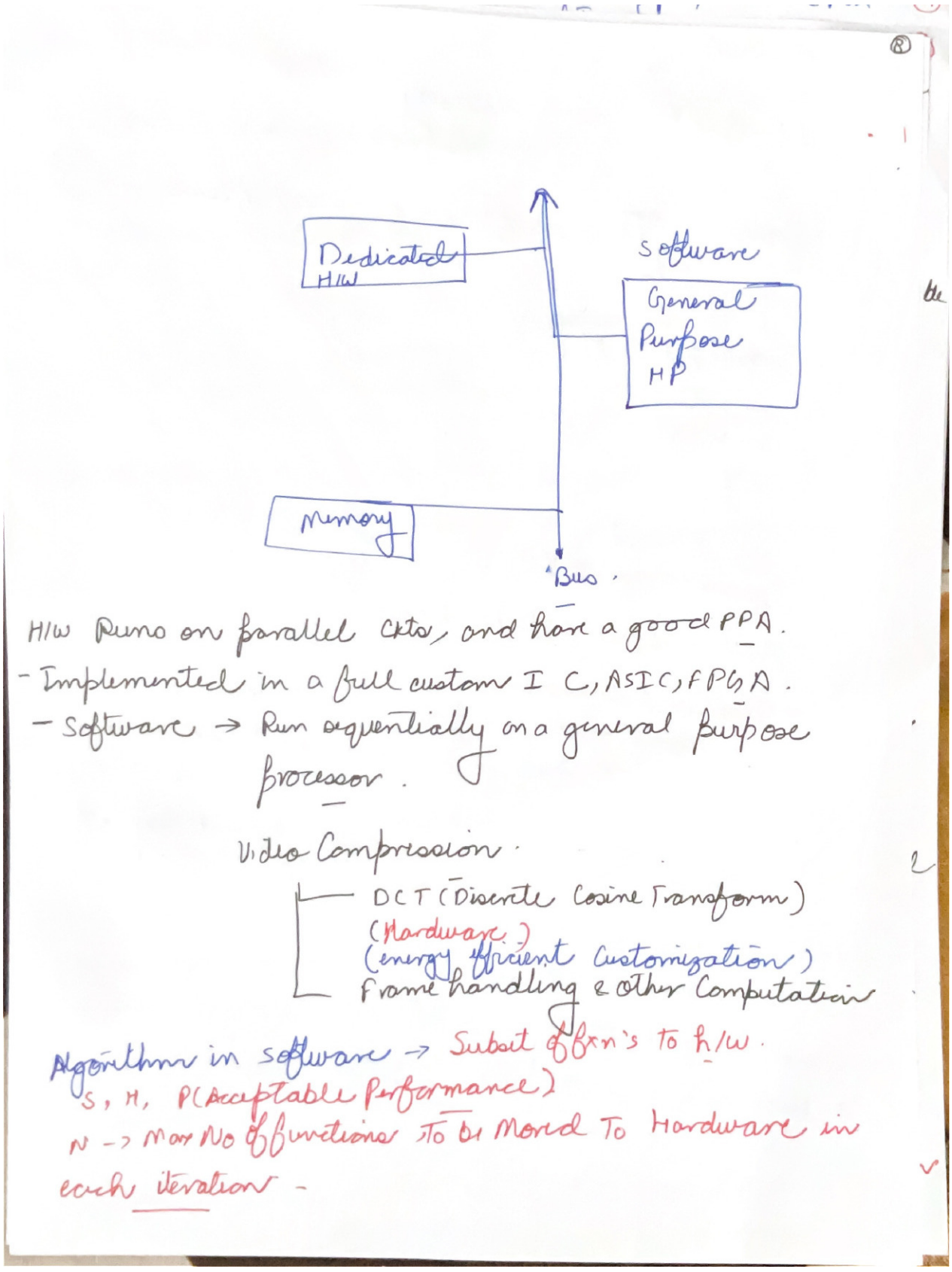

Figure 1.37. *Source: Part 1, PDF page 9.*

In the CPU-memory-accelerator drawing, communication is part of the computation. An accelerator needs input data, configuration, a start/ready protocol, and a way to return results. Shared memory bandwidth and bus contention may become bottlenecks. Replacing an expensive software function with fast hardware does not remove the cost of moving data into and out of that hardware.

The lecture's DCT example illustrates a function that dominates execution time. To quantify the limit, let fraction p of original execution time be accelerated by factor s, with normalized extra communication/control overhead o. A simple extension of Amdahl's argument is

$$\text{speedup} = \frac{1}{(1-p) + p/s + o}.$$

If $p = 0.8$, $s = 1000$, and $o = 0$, total speedup is $1/(0.2 + 0.0008) \approx 4.98$, not 1000. The unaccelerated 20% caps the ideal benefit near 5. Adding overhead of 0.05 reduces speedup to about 3.99. These calculations extend the lecture's example and show why whole-system evaluation matters.

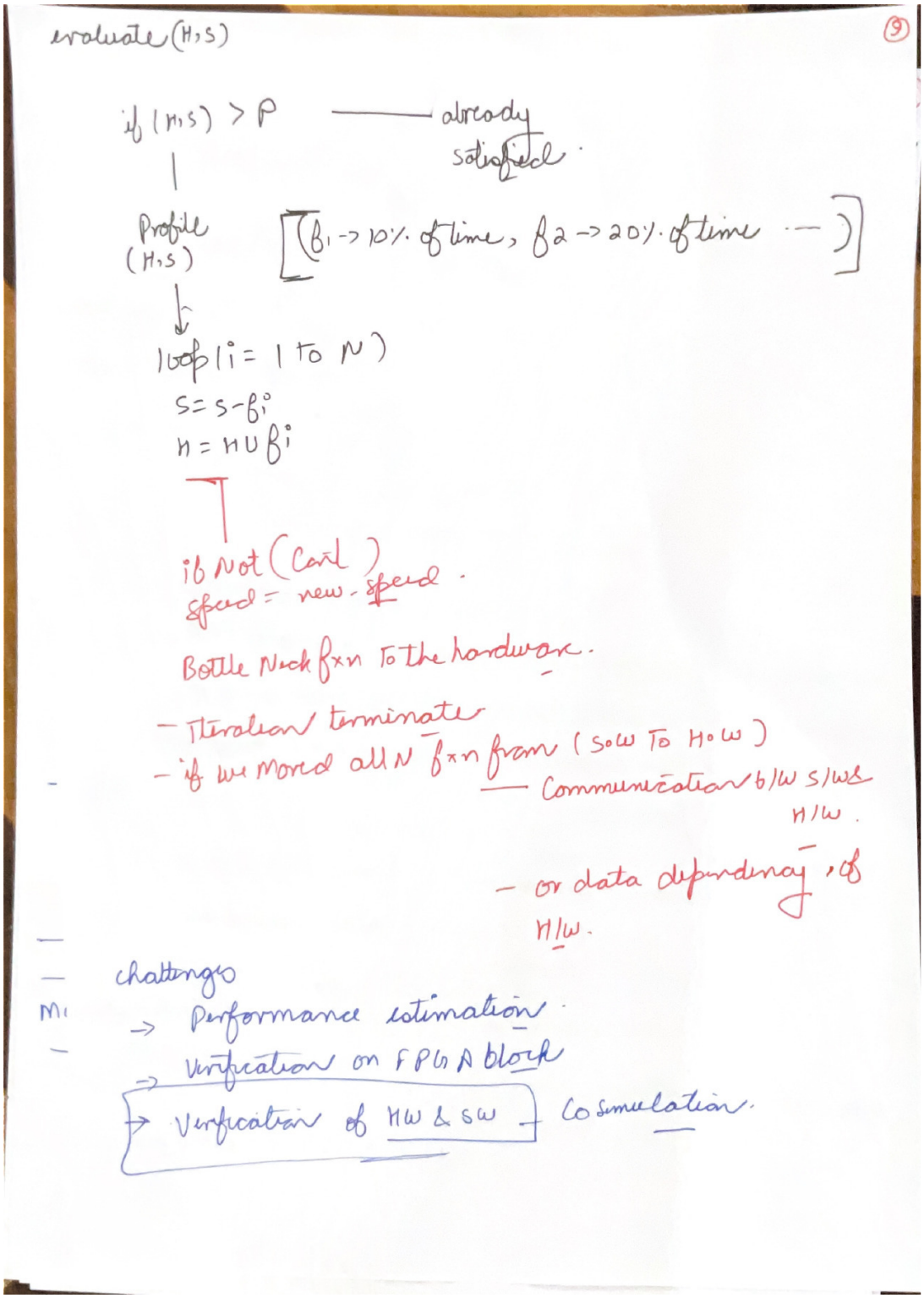

Figure 1.38. *Source: Part 1, PDF page 10.*

Read the algorithm as a heuristic search:

1. Start with hardware-function set $H = \varnothing$ and software set S containing all candidate functions.
2. Evaluate the current system. If it meets the performance requirement, retain that partition.
3. Profile execution to identify dominant functions or communication costs.
4. Move selected candidate functions from S to H, up to the chosen search limit, and evaluate again.
5. Continue if the changes improve the objective. Stop successfully when requirements are met; otherwise report that this search did not find an acceptable partition.

In a set update, $H \leftarrow H \cup \{f_i\}$ and $S \leftarrow S \setminus \{f_i\}$. The braces matter: a function is being moved as an element of a set. In a throughput objective, larger measured performance is better; in a latency objective, smaller is better. The inequality in a stopping condition must match the chosen metric.

"No partition found" is not proof that no feasible partition exists. A greedy choice can miss a beneficial combination or a different architecture. Reprofiling matters because removing one bottleneck exposes another. Moving functions can also create new communication and synchronization costs.

Estimating hardware before it exists

An early performance estimate can come from an analytical model, high-level simulation, FPGA prototyping, or a rapid synthesis/implementation experiment. Each has limits. An FPGA prototype exercises hardware/software interaction but its clock, memory, and routing characteristics do not automatically predict ASIC timing. A high-level model can evaluate workloads quickly but must model communication and contention well enough for the decision.

Co-simulation connects models of hardware and software so their interaction can be checked. It helps expose protocol and sequencing mistakes, but its assurance depends on the models, stimuli, and properties checked. An early model is valuable when its assumptions and uncertainty are explicit.

Recall checks: Why might the most frequently called function be a poor accelerator candidate? Why can an accelerator with a short arithmetic delay still reduce total system performance? Why should the stopping condition distinguish feasibility from global optimality?

Definitions and mechanisms in Lesson 03

Design representation and abstraction. A representation records selected properties of a design: an algorithm expresses a computation, RTL identifies state and clock-related behavior, a netlist identifies instances and connections, and layout identifies geometry and layers. Abstraction deliberately omits details that are unnecessary for the current question. A Boolean expression can establish a truth function without establishing timing or routability. Moving to a more detailed representation provides evidence about additional properties; it does not make the earlier requirement disappear.

RTL and GDSII. Register-transfer level describes stored state and the operations that transform values between storage boundaries. GDSII is a hierarchical geometry interchange format used for layout data. RTL can express a mux and a clocked register without choosing their coordinates. Layout data records physical shapes without independently stating the complete system specification. "RTL

to GDS" names a series of constrained transformations and checks between those representations. Producing a GDSII file alone is not evidence that every signoff requirement was met.

Hardware and software partitioning. Partitioning assigns functions to dedicated hardware or to instruction execution on programmable processors. It is an architectural choice involving computation, communication and control. Moving a function into hardware creates or changes an interface: inputs must be transferred or made accessible, work must be started, completion must be recognized, and outputs must be associated with the correct request. The relevant benefit is the whole-system result after these costs, not just the accelerator's internal arithmetic speed.

Profiling and a bottleneck. Profiling measures how execution time or another resource is distributed over a representative workload. A bottleneck limits the system's current performance. The most frequently called function need not consume the most time, and the most expensive arithmetic need not dominate after communication is included. Changing one component can expose a different limit. Reprofiling is therefore part of partitioning: it tests whether the assumed bottleneck still determines the result after an architectural change.

Bandwidth and transaction overhead. Bandwidth is data transferred per unit time, with the byte convention and measurement conditions stated. Latency and setup overhead determine how long a particular request waits or takes to begin. A high-bandwidth interface can still have expensive small transactions. The accelerator example uses sustained usable bandwidth and assumes serial transfers and computation. If transfers overlap computation, the timing model changes. A bus line in a block diagram supplies connectivity information; its bandwidth and protocol costs require additional assumptions or measurements.

Heuristic search and optimality. A heuristic chooses promising alternatives without exhaustively proving the best result over every possibility. Moving a dominant software function into hardware is one plausible step, but combinations, ordering and communication can affect the outcome. "This partition satisfies the requirement" is supported by a correctly scoped evaluation. "No feasible partition exists" requires stronger evidence than the failure of one greedy search. Use the conclusion that the actual search and model justify.

Application and validation in Lesson 03

An accelerator request is measured from available input to usable output. Transfer bytes, sustained bandwidth, control overhead and computation time are included under an explicit overlap assumption. The communication calculation explains why a fast arithmetic block can fail to improve the application.

Deriving the limiting speedup. Normalize the original execution time to one. If a fraction $p = 0.9$ is accelerated by a factor s, with no added overhead, total time becomes $(1 - p) + p/s$. Even as s grows without bound, the remaining time approaches 0.1, so overall speedup cannot exceed ten. With $s = 100$, time is 0.109 and speedup is approximately 9.17. If communication adds 0.05 of the original execution time per request, speedup falls to $1/0.159 \approx 6.29$. The overhead is added to elapsed time; it is not divided by the arithmetic speedup.

Complete original notebook pages for Lesson 03

Part 1 original page 08

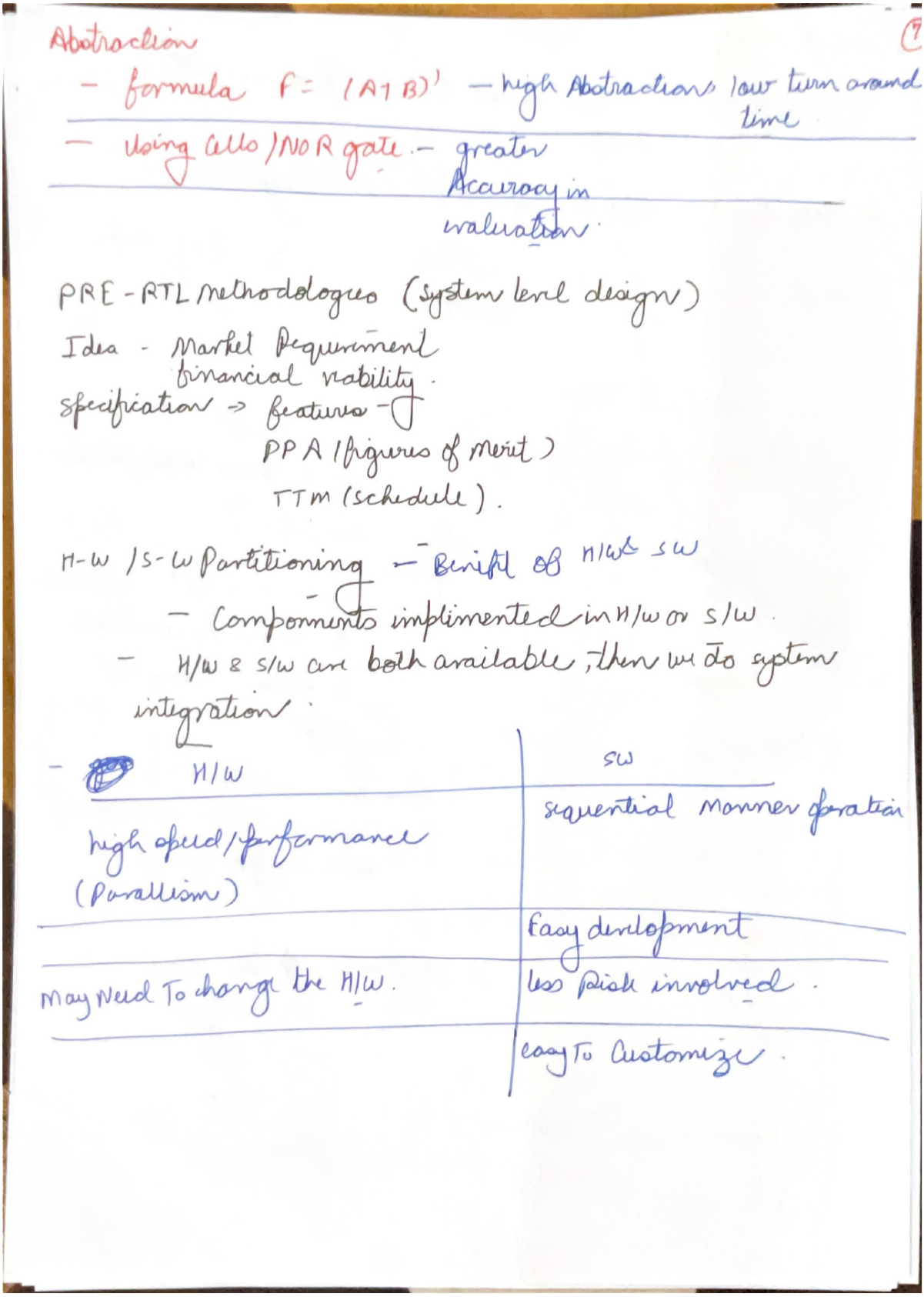

Figure 1.39. *Source: Part 1, PDF page 8. NOR abstraction, specification, hardware/software partition.*

Explanation and worked examples: Day 1 · Lesson 03.

Part 1 original page 09

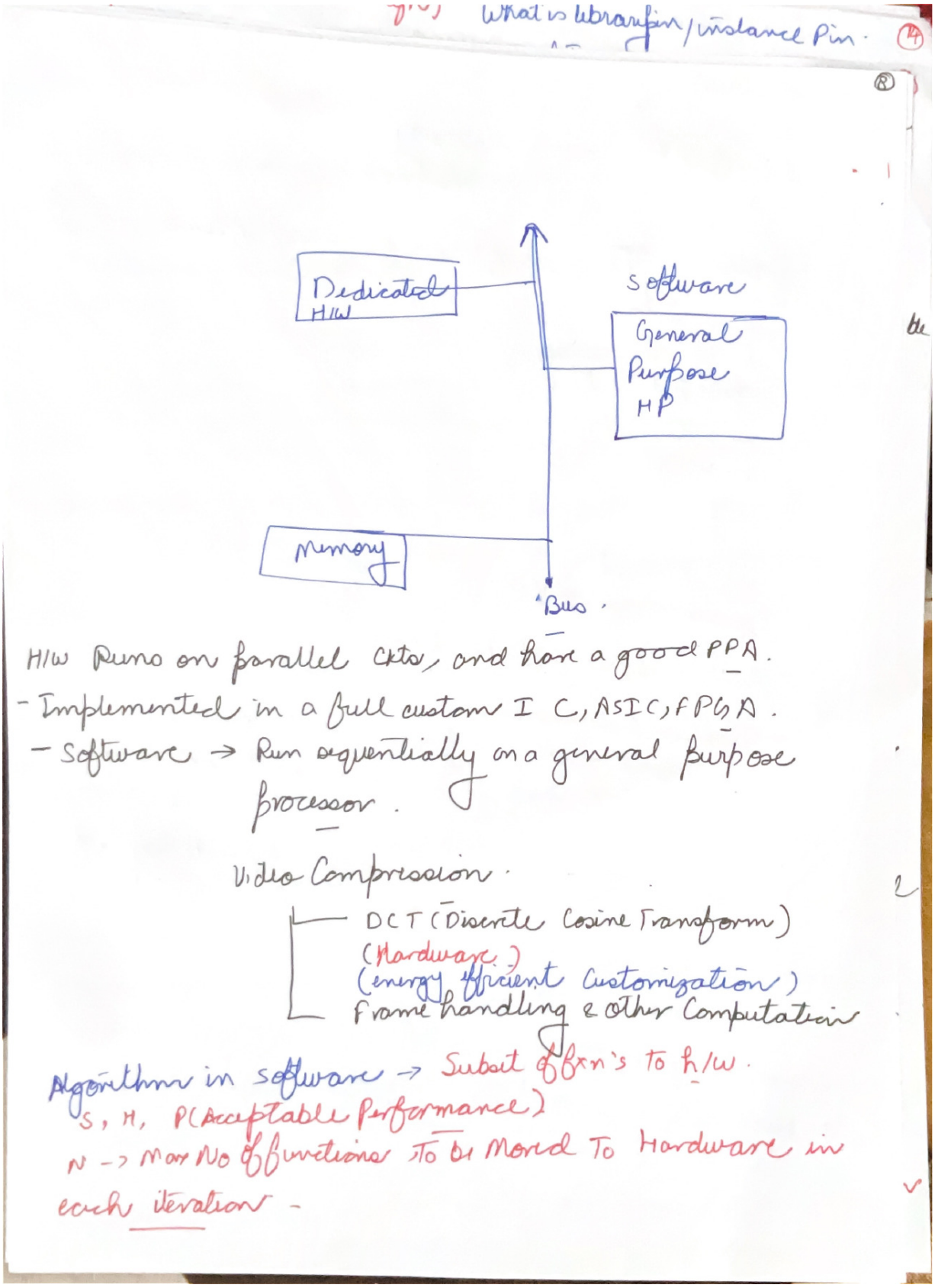

Figure 1.40. *Source: Part 1, PDF page 9. Processor/accelerator/memory and partition estimates.*

Explanation and worked examples: Day 1 · Lesson 03.

Part 1 original page 10

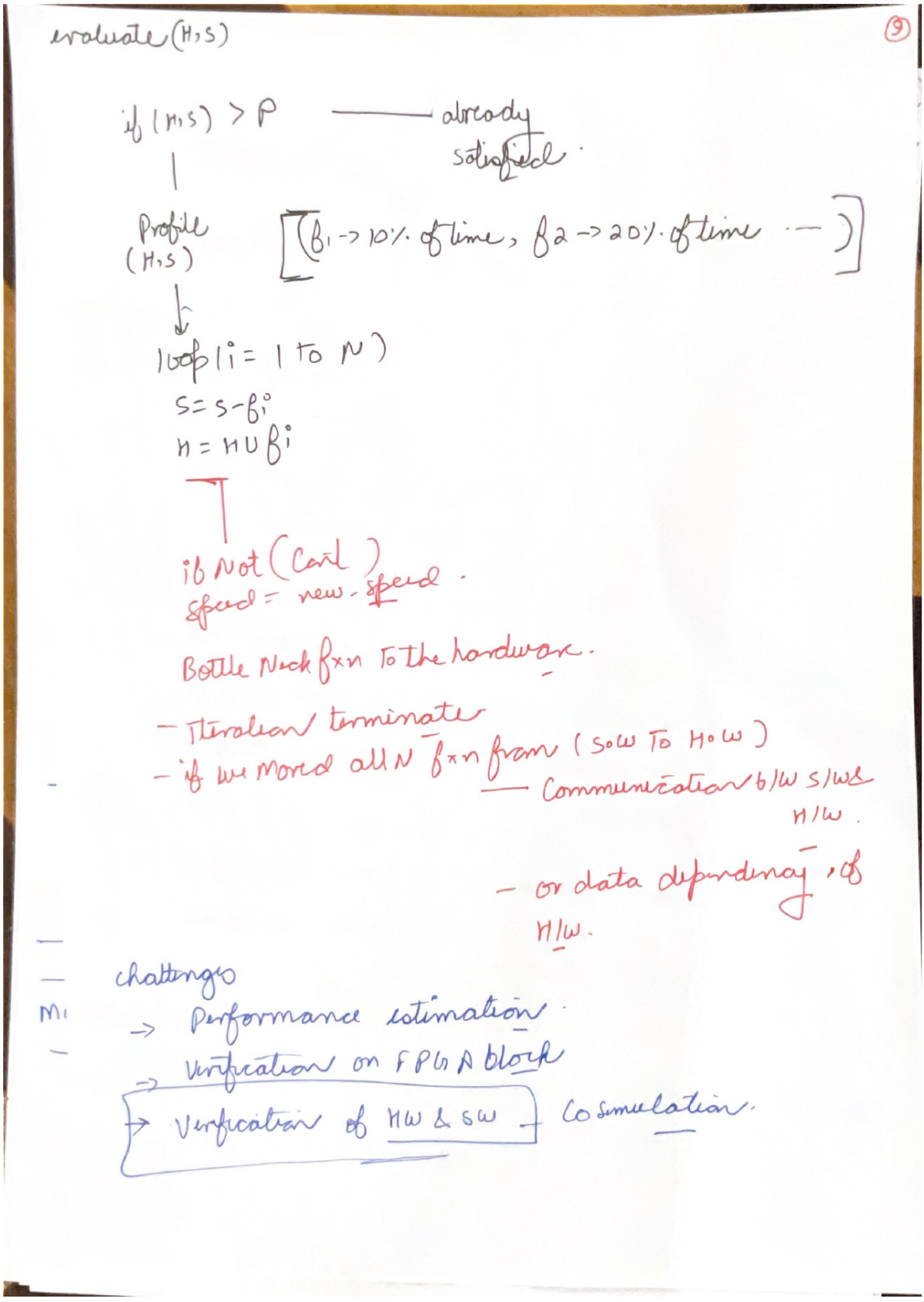

Figure 1.41. *Source: Part 1, PDF page 10. Partition heuristic, communication cost, prototyping/co-simulation.*

Explanation and worked examples: Day 1 · Lesson 03.

Lesson 04 High level synthesis and timing

Week 1 · Lecture video

The implementation gap and IP reuse

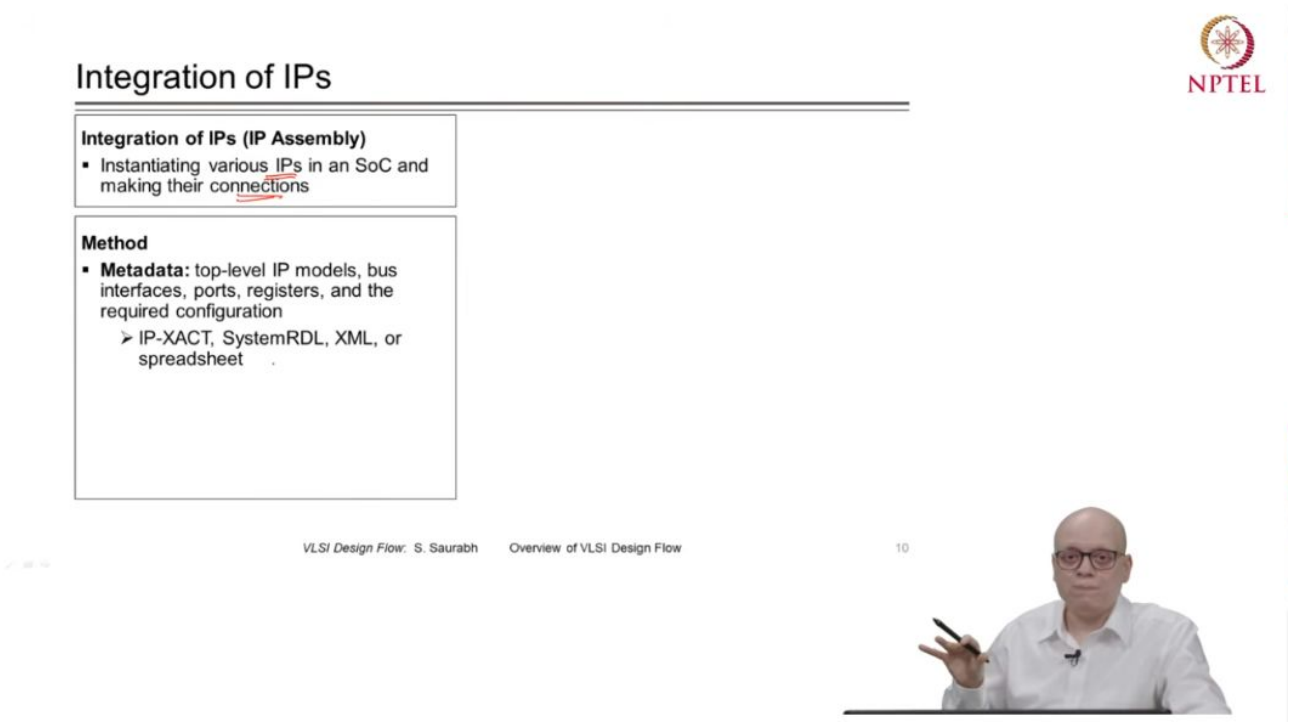

Figure 1.42. *Lecture: 17:48. Metadata supports integration of reusable IP blocks*

A functional specification says **what** computation must occur. RTL also commits to **when** operations occur relative to clock events and how state is stored. The difference is the **implementation gap**. Translating an untimed expression into RTL requires architectural choices: how many arithmetic units exist, which operations share them, where registers are placed, how inputs are accepted, and when outputs become valid.

The **datapath** contains arithmetic, logic, multiplexers, and data registers. The **control path** generates enables, selects, and state transitions that govern datapath operation. Both run as hardware. Writing lines one after another in a file does not automatically mean that the hardware executes them in separate clock cycles.

Three routes bridge the gap: manually write RTL, integrate existing IP, or use behavioral/high-level synthesis. A **system on chip (SoC)** integrates substantial system components such as processors, memory, accelerators, peripherals, and sometimes analog/RF blocks. A **reusable IP block** packages a design capability and the information needed to use it. “Pre-verified” does not remove the need to verify its configuration and integration in the new system.

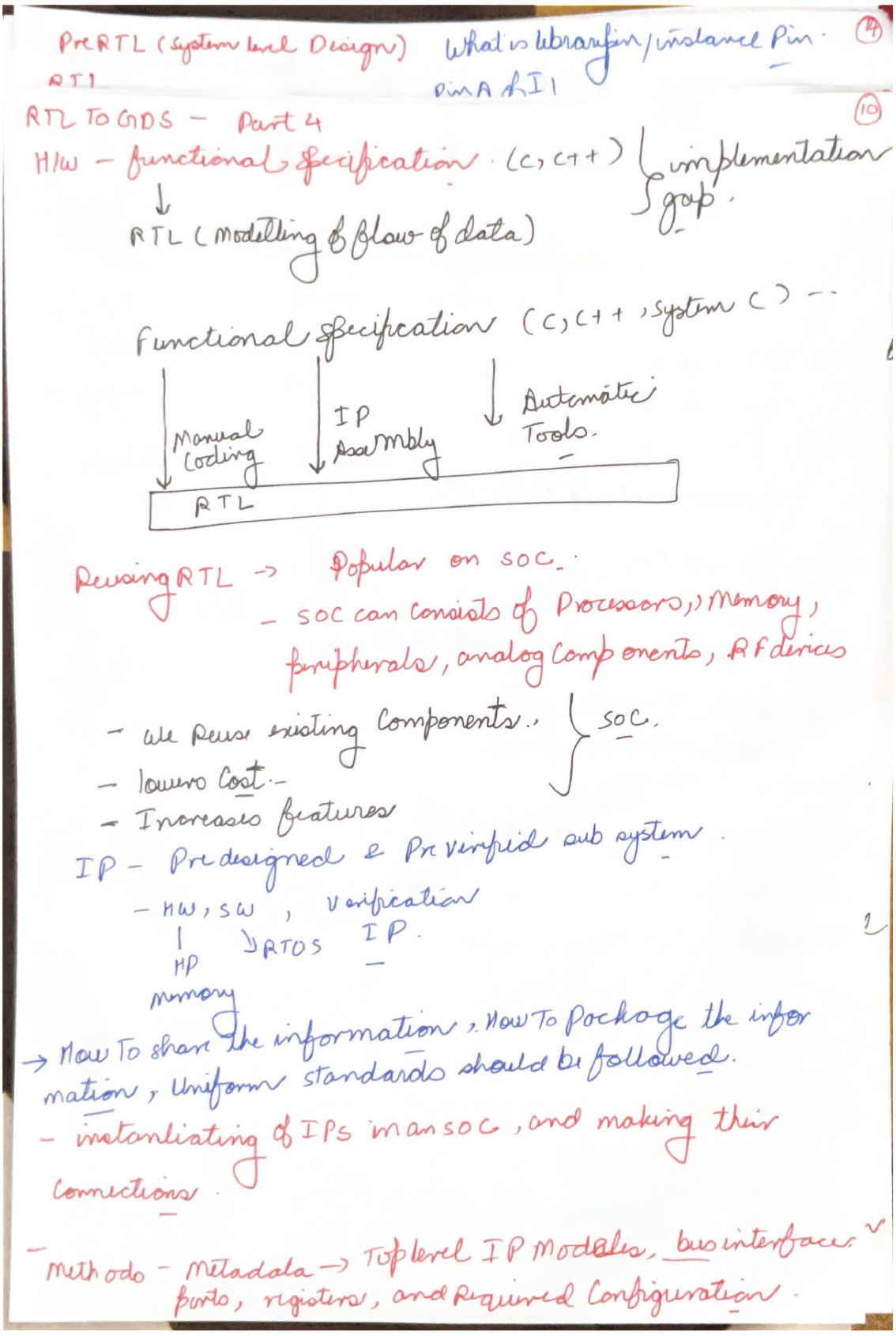

Figure 1.43. *Source: Part 1, PDF page 11.*

The hardware/software/verification-IP categories describe different reusable deliverables. Hardware IP can implement a processor or interface; software IP can provide drivers; verification IP can generate protocol transactions and check responses. A driver is not a physical gate block, but it can be essential to operating the hardware correctly.

Metadata is structured information about the design: module identities, parameter values, ports, bus interfaces, address maps, registers, and configuration. A generator can use it to create connections, wrappers, register descriptions, or a verification environment. The lecture mentions IP-XACT, SystemRDL, XML, and spreadsheets. They do not all express the same information or offer identical guarantees.

Integration must resolve width, protocol, clock, reset, and power-domain compatibility. An 8-bit interface connected to a 16-bit interface may require a width adapter and a defined ordering of transfers. A **network on chip (NoC)** provides structured on-chip communication. It helps organize complex systems but adds its own latency, arbitration, buffering, and verification concerns; not every SoC requires a NoC.

Behavioral synthesis and its cost measures

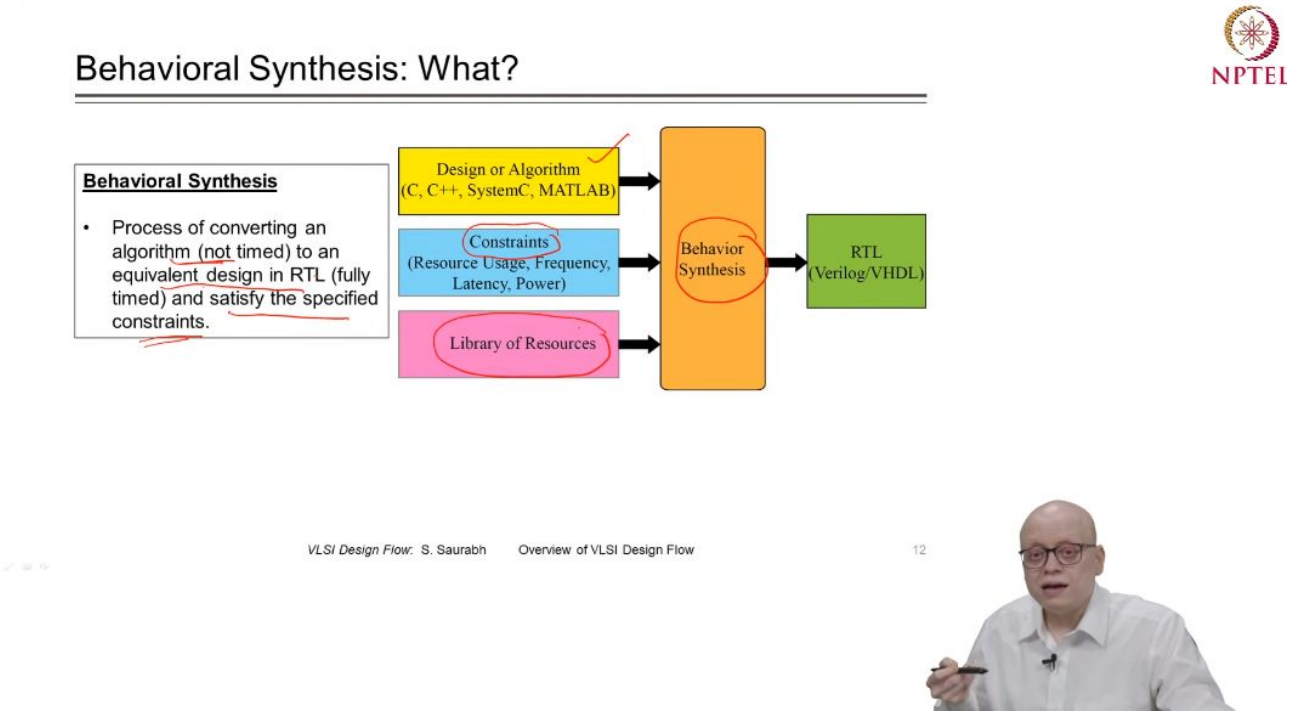

Figure 1.44. *Lecture: 23:46. The three inputs to behavioral synthesis and its RTL output.*

The yellow box specifies **what to compute**. The blue box specifies requirements such as frequency, latency, and resource use. The pink box describes resources the implementation can use. The output adds an architecture: operations happen in particular cycles, values are stored, resources are selected, and control coordinates execution. An algorithm alone does not uniquely determine those choices. “Untimed” does not remove data dependencies: an addition consuming a product still needs that product to exist first.

Behavioral synthesis, also called **high-level synthesis (HLS)**, converts an algorithmic description into a timed RTL architecture under constraints. Its input is more than the algorithm: it also needs resource and implementation models, target clock requirements, and constraints on area, latency, throughput, or power.

Scheduling assigns operations to control steps or cycles. **Allocation** chooses how many resources are available. **Binding** maps operations and stored values to particular resources and registers. These decisions interact: sharing one multiplier can reduce arithmetic area but introduce multiplexers, control, and longer scheduling intervals.

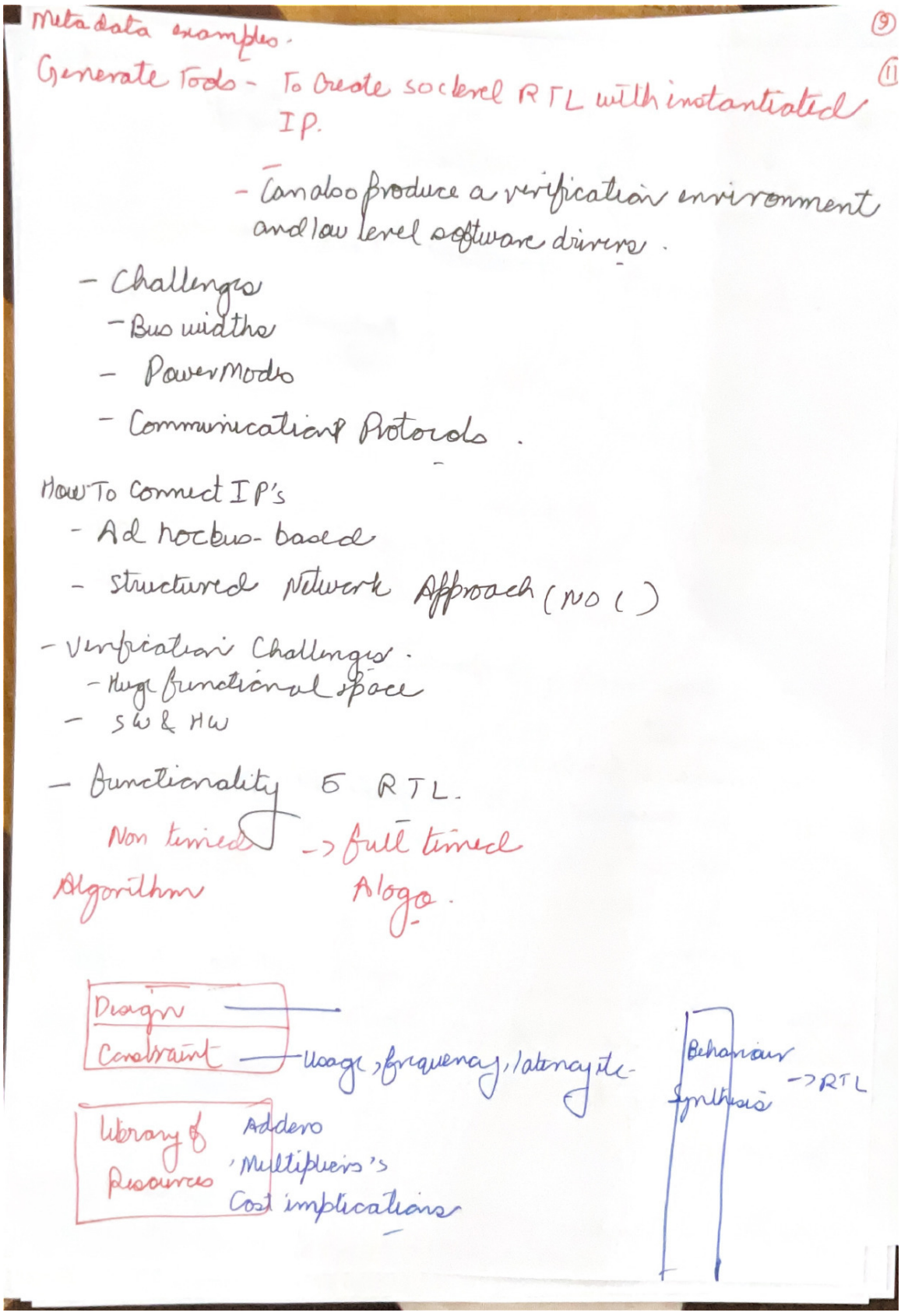

Figure 1.45. *Source: Part 1, PDF page 12.*

The diagram captures the algorithm + constraints + resource-library → RTL relationship. Separate three metrics carefully:

Metric	Definition	Example
Latency	Time from an accepted input transaction to its valid result	2 cycles at 5 ns per cycle = 10 ns
Initiation interval	Cycles between accepted independent transactions	An interval of 2 allows one new transaction every other cycle
Throughput	Results completed per unit time in steady state	With interval 2 at 200 MHz, at most 100 million results/s

Register count and arithmetic-unit count provide an early area estimate. Physical wire length, congestion, buffering, and clocking remain uncertain until implementation. A resource-sharing solution can look small at RTL but collect so many signals around one unit that physical implementation becomes difficult. Generated RTL also needs verification against the original algorithm, including finite widths, overflow, signedness, and timing of inputs and outputs.

Build a schedule from dependencies and resource limits

Consider $y = ab + cd$ for one accepted transaction. The multiplications are independent; the addition depends on both products. Suppose each multiplier operation takes one cycle, an addition takes one cycle, and the selected clock period permits those assumptions. Results are stored at cycle boundaries. This illustrates scheduling; it is not a measured implementation result.

Cycle	Two multipliers and one adder	One shared multiplier and one adder
1	Compute $p = ab$ and $q = cd$ in parallel	Compute and store $p = ab$
2	Compute $y = p + q$	Compute and store $q = cd$
3	This transaction has finished	Compute $y = p + q$

Scheduling decides the cycle, allocation decides the resource count, and binding assigns each operation to a particular resource. Sharing requires operand selection and control. Storing p preserves information needed after the second multiplication. Clock period and operation delay constrain whether dependent operations can be chained into one cycle; writing them on one source line does not establish that they fit. AMD's HLS scheduling guide explains scheduling after predecessor operations and the role of resource latency.

At 5 ns per cycle, the illustrated latencies are 10 ns and 15 ns. Throughput requires a separate analysis. Each transaction needs two multiplications. If a multiplier accepts one operation per cycle, one multiplier imposes a resource lower bound of **two cycles per transaction** in steady state. The independent adder may finish an older transaction while the multiplier starts a newer one, so three-cycle latency does not automatically mean initiation interval three. Two multipliers can potentially support interval one. Dependencies, storage ports, interfaces, and stalls can make the achieved interval larger.

Widths also affect correctness. Unsigned four-bit inputs range from 0 to 15. Each product can reach 225 and needs eight bits; the sum can reach 450 and needs nine bits. An eight-bit result loses

information if the specification requires the full sum. Resource sharing must preserve the specified numeric behavior and transaction pairing.

Reproduce it: label two transactions with different letters and overlap their schedules. Identify the registers holding each product before the adder reads them. A schedule is incomplete if it computes the right expression but combines products from different requests.

A feedback dependency can limit throughput despite spare hardware

The resource bound above concerns independent requests. A feedback computation adds a different restriction: an operation can need the result of a preceding iteration. Consider the illustrative running sum $s_{n+1} = s_n + x_n$, initialized with $s_0 = 0$. Assume the selected adder accepts independent operands every cycle but returns each sum three cycles after acceptance. The next update needs that returned state, so this particular recurrence cannot start its next addition after only one cycle.

Update	State consumed	New input	Earliest start cycle	Result available at cycle	Updated state
0	0	2	0	3	2
1	2	3	3	6	5
2	5	4	6	9	9

These are cycle-boundary assumptions for a teaching schedule, with no overflow. Although the adder can accept independent work in cycles 1 and 2, the state required for update 1 is unavailable until cycle 3. A second adder does not manufacture the missing state. Starting update 1 with the old zero would compute 3 rather than the required cumulative value 5.

If a recurrence produces a value after L cycles and its consumer is d iterations later, starts separated by an initiation interval II must satisfy $d \cdot II \geq L$. Hence its dependency bound is $II \geq \lceil L/d \rceil$. Here $L = 3$ and $d = 1$, giving a lower bound of three. Resource capacity, dependencies and interfaces all impose bounds; satisfying one does not establish the achieved interval. AMD's loop-dependency guide explains why a later iteration may have to wait for an earlier computation.

This distinction also clarifies the feedback adder on Part 1 page 14. Identify when a stored partial result is valid before assigning the next operation to a cycle. Reorganizing an algorithm can sometimes change a dependence, but merely ignoring a genuine dependence changes the computation.

Paths and the clock-period budget

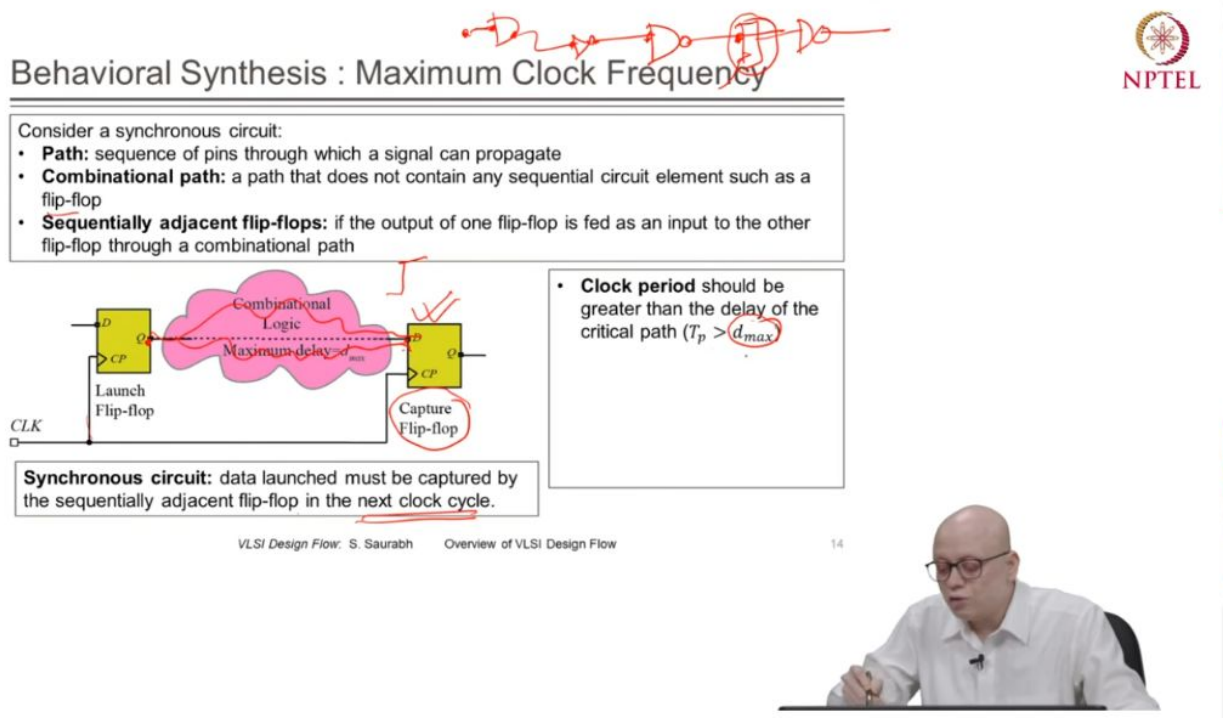

Figure 1.46. *Lecture: 32:14. Launch and capture flip-flops connected by combinational logic*

A **path** is an ordered sequence of pins and connections along which a signal can propagate. A **combinational path** does not cross a state-holding element. Two flip-flops are **sequentially adjacent** when one can send data to the other through only combinational logic and interconnect. This is a connectivity relationship, not a statement that they sit next to one another on the die.

For a conventional single-cycle register-to-register path, the launch edge causes the first register's output to change. Data traverses logic and wire delay and must settle before the next capture edge by at least the receiving register's setup time. The lecture first approximates this using only maximum combinational delay. A more useful zero-skew setup budget is

$$T_{\text{clk}} \geq t_{\text{cq,max}} + t_{\text{comb,max}} + t_{\text{setup}} + t_{\text{uncertainty}}.$$

Here $t_{\text{cq,max}}$ is maximum clock-to-Q delay, $t_{\text{comb,max}}$ includes the logic and wires, and uncertainty reserves margin for specified clock effects. If capture clock arrival is later than launch clock arrival by t_{skew}, using that sign convention adds t_{skew} to the available setup budget. Real STA also handles multiple clocks, exceptions, corners, and other checks.

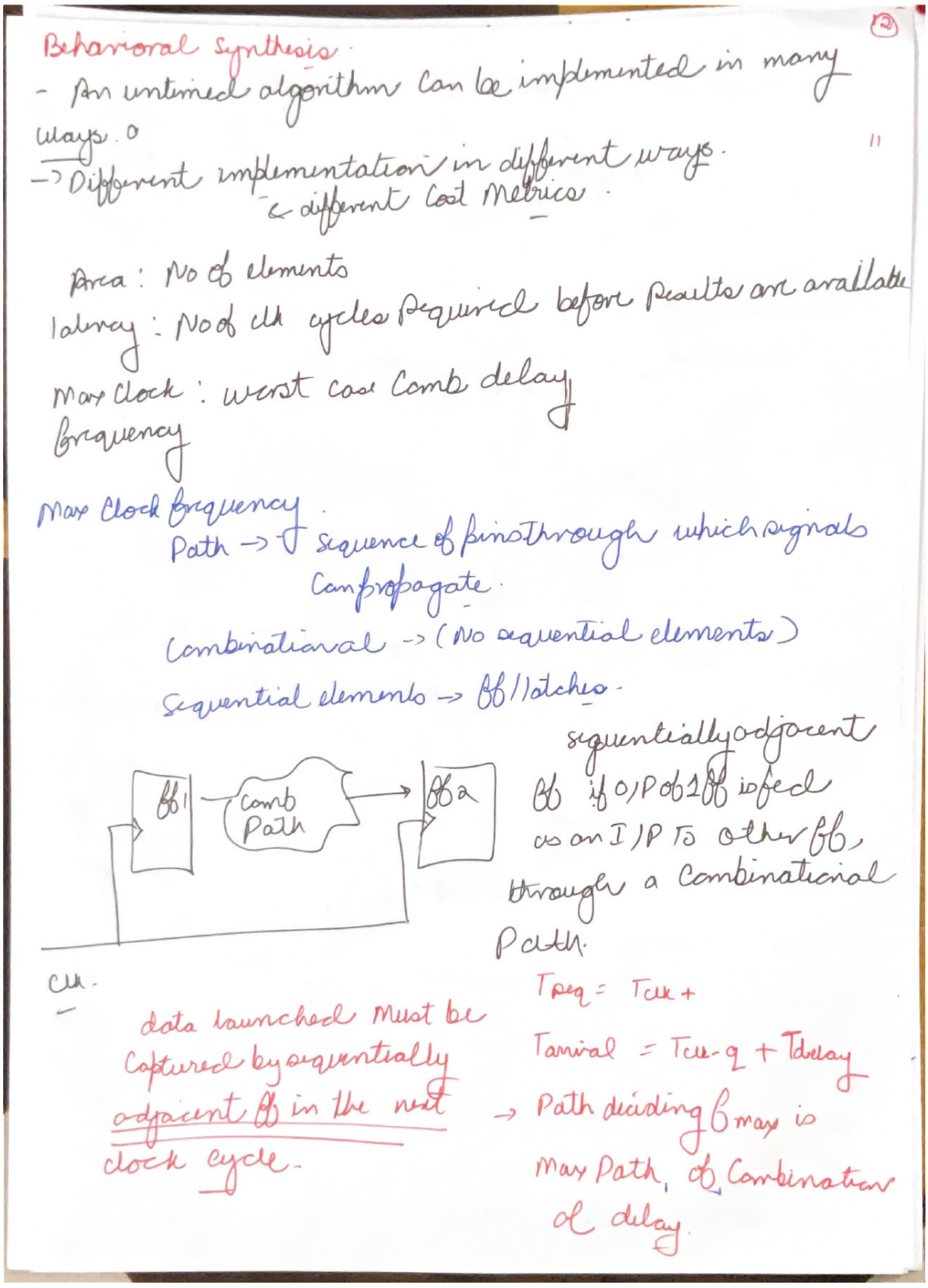

Figure 1.47. *Source: Part 1, PDF page 13.*

The arrival-time expression is pointing in the right direction but needs a time reference. Measured from a launch clock event at time zero, data arrival is $t_{cq} + t_{comb}$. Required arrival is the capture-edge time **minus** setup time and applicable uncertainty. Setup slack is required arrival minus actual arrival; nonnegative slack meets that modeled check.

For example, with $t_{cq} = 0.08$ ns, combinational delay 0.62 ns, setup 0.10 ns, and uncertainty 0.05 ns, the minimum zero-skew period is 0.85 ns, corresponding to approximately 1.176 GHz. These are illustrative values. Increasing the clock period helps this setup check; it does not generally fix a hold violation, which concerns data changing too soon around the same capture edge. A common zero-skew hold condition is $t_{cq,min} + t_{comb,min} \geq t_{hold}$ before adding the relevant margins.

The "next cycle" rule assumes an ordinary single-cycle path. Valid multicycle protocols require explicitly justified constraints; they cannot be assumed merely because a path is slow.

Why clock skew can help setup and hurt hold

Setup time requires the value being captured to be stable before the receiving clock edge. **Hold time** requires that value to remain stable after the edge. Thus setup limits how late the intended data may arrive, while hold limits how early the following data may replace it. These are separate checks on maximum and minimum delays.

Define skew as capture-clock arrival minus launch-clock arrival for corresponding clock edges. Positive skew means the capture clock arrives later. For this single-cycle, equal-period model, with nonnegative setup and hold uncertainty margins U_s and U_h:

$$S_{setup} = T + t_{skew} - t_{setup} - U_s - (t_{cq,max} + t_{comb,max}).$$

$$S_{hold} = t_{cq,min} + t_{comb,min} - (t_{skew} + t_{hold} + U_h).$$

Both slacks must be nonnegative. Setup slack subtracts arrival from the latest allowed arrival. Hold slack subtracts the earliest allowed change from the actual earliest change. The opposite subtraction order reflects what each check protects.

Use the setup numbers above, a 1.00 ns period, and $U_s = 0.05$ ns. Separately, suppose minimum clock-to-Q is 0.04 ns, minimum combinational delay is 0.03 ns, hold time is 0.04 ns, and $U_h = 0$ for this example:

Clock skew	Setup slack	Hold slack	Meaning
0.00 ns	$1.00 - 0.10 - 0.05 - 0.70 = 0.15$ ns	$0.07 - 0.04 = 0.03$ ns	Both modeled checks pass
+0.05 ns	$1.05 - 0.10 - 0.05 - 0.70 = 0.20$ ns	$0.07 - 0.09 = -0.02$ ns	Setup improves, but hold fails

The later capture clock gives incoming setup data more time, but requires the previous value to survive longer against the earliest new data. Increasing the period does not change the second equation. Adding data-path delay may repair hold while consuming setup margin, so both checks must be repeated. Real STA selects the applicable edges, early/late delays, corners, and uncertainties rather than applying one nominal skew to every check.

Three implementations of a plus b plus c

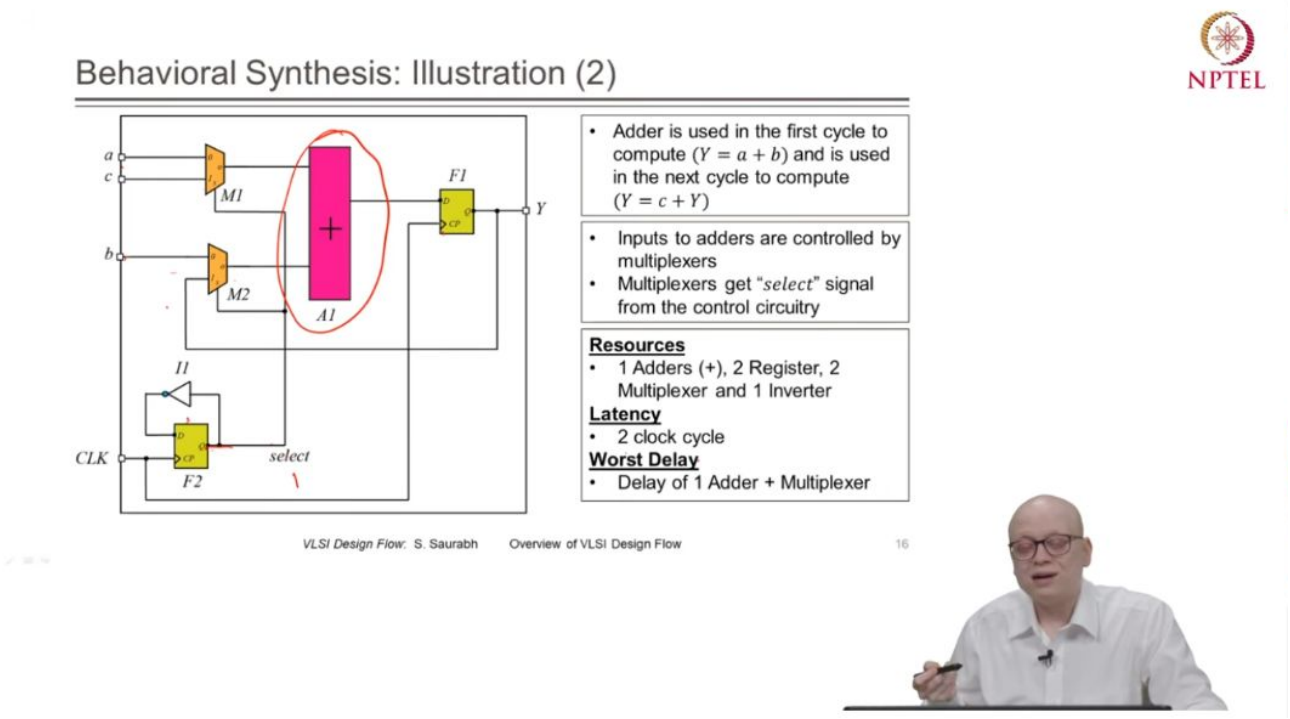

Figure 1.48. *Lecture: 39:00. One adder reused over two cycles with multiplexers and feedback*

The expression $y = a + b + c$ permits several architectures. For a fair comparison, define the bit widths, arithmetic behavior, and input protocol consistently.

Architecture	Arithmetic resources	Result latency	Long combinational portion	New-input interval
Two cascaded adders and output register	2 adders	1 cycle	Approximately 2 adder delays	1 cycle
Pipelined adders	2 adders, additional data registers	2 cycles	Approximately 1 adder delay per stage	1 cycle after filling
One shared adder	1 adder, muxes, data/control storage	2 cycles	Approximately mux + adder	2 cycles for this simple design

The pipelined version must delay c appropriately so the second stage combines values belonging to the same transaction. That alignment register may be omitted in a simplified drawing whose inputs are held stable. The table describes the protocol explicitly rather than assuming that any two-register drawing is a correct streaming pipeline.

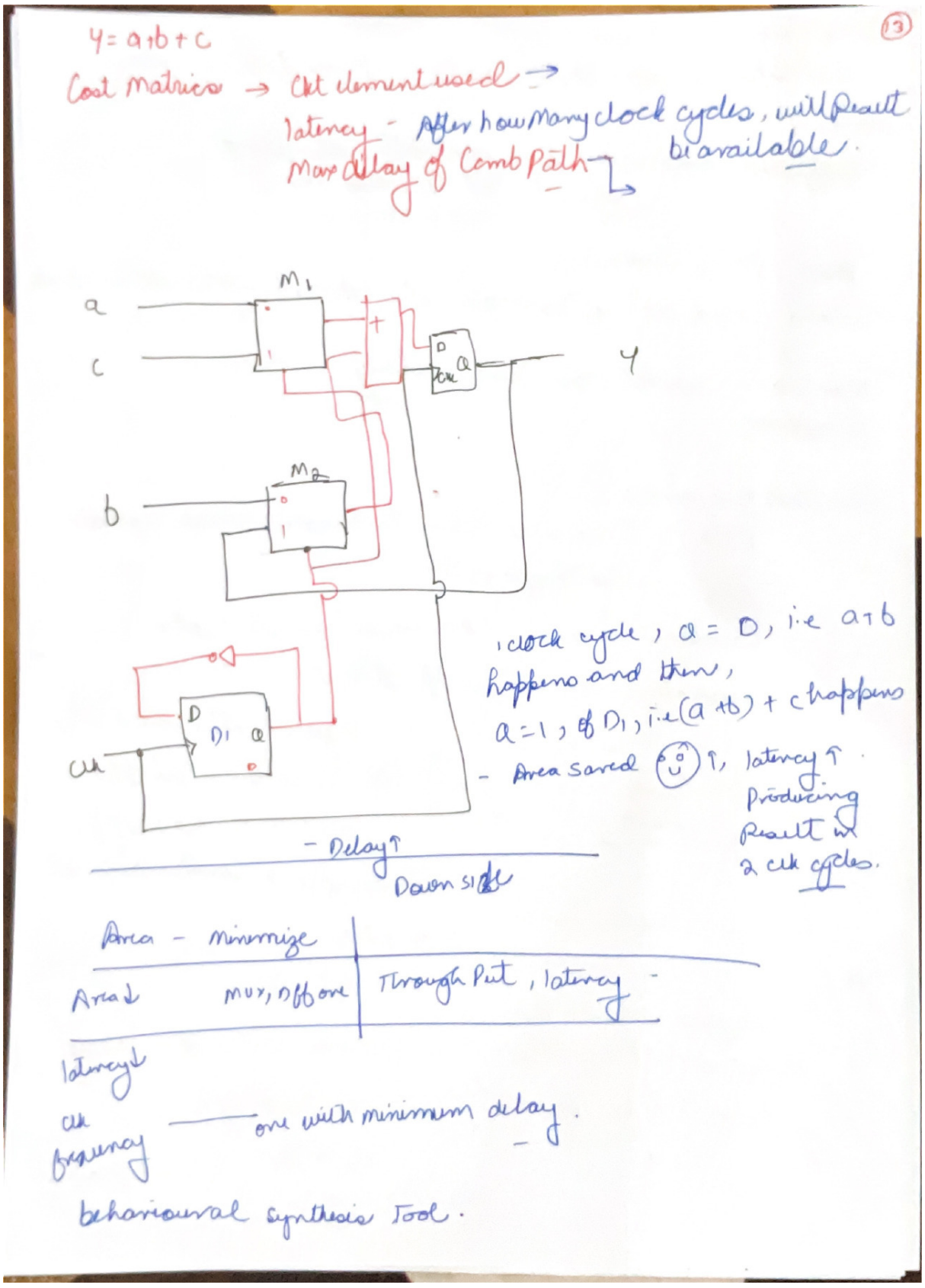

Figure 1.49. *Source: Part 1, PDF page 14.*

The feedback drawing reuses one physical adder. In phase 0, the muxes select a and b, and the result register captures their sum. In phase 1, they select c and the saved sum, and the register captures the final result. With a=2, b=3, and c=4, the register becomes 5 after the first computing edge and 9 after

the second. The first value is an intermediate result; a consumer must not mistake it for the completed transaction.

The toggle flip-flop creates alternating phases only if its initial phase is known. A practical block needs a reset or other defined initialization, a clear input-acceptance rule, and an output-valid indication. Inputs must remain valid when consumed or be captured into local registers. Without these details, the diagram illustrates resource sharing but is not a complete interface specification.

Area savings are conditional: the removed adder must save more than the added muxes, control, and registers cost. A shorter combinational path may permit a higher clock, but reduced initiation rate can still lower throughput. HLS chooses among these tradeoffs according to constraints and its available implementation models. The final design still needs synthesis and physical validation.

Follow transaction ownership through the shared adder

Part 1 page 14 draws arithmetic and phase control; the missing interface question is which transaction owns each operand. Suppose transaction A is (2,3,4) and transaction B is (7,11,13). A's first computing edge stores 5. At its second edge the adder must use A's c=4 and return 9. If the external input has already advanced to B's c=13, a design that did not save c instead returns 18. Both additions are individually correct, but the transaction is wrong.

Computing edge	Accepted tuple	Operation on the shared adder	Result status
1	A: (2,3,4)	2 + 3 = 5; save A's c	Intermediate
2	None	5 + saved 4 = 9	A valid
3	B: (7,11,13)	7 + 11 = 18; save B's c	Intermediate
4	None	18 + saved 13 = 31	B valid

One solution captures the tuple when a request is accepted; another requires the sender to hold the relevant operands until consumed. Either needs an explicit handshake or fixed schedule. Reset must establish the first phase, and output-valid must identify the second phase. For streaming data, a pipelined two-adder design similarly delays c alongside a+b. Resource sharing changes the acceptance schedule as well as the gate count; verification should compare results indexed by accepted transactions.

Definitions and mechanisms in Lesson 04

Datapath and control path. A datapath stores and transforms data through registers, arithmetic units, logic and selection networks. A control path determines which operation occurs, which operands are selected, when state updates and when a result is valid. Reusing one adder requires control because the same hardware must perform different additions at different times. These are cooperating parts of one circuit. Calling control “software” because it sequences operations confuses an implementation role with the hardware/software partition.

Scheduling, allocation and binding. Scheduling assigns operations to time steps while respecting dependencies and timing. Allocation chooses the available resources. Binding associates operations with particular resources and values with storage. With two products followed by a sum, the products can be scheduled together only if the chosen resources support that concurrency. A shared

multiplier changes the schedule and requires operand selection and intermediate storage. The three decisions jointly define an architecture; none is merely a different name for translating syntax.

Dependency and transaction alignment. A data dependency exists when an operation needs a value produced by another operation. A transaction identifies which input values belong to the same requested computation. Correct scheduling respects both. In a pipelined computation of a plus b plus c, delaying only the partial sum can accidentally pair an old sum with a newer c. Delay or store c so the final addition uses operands from the same transaction. Arithmetic correctness for isolated constant inputs does not prove correct streaming behavior.

Initiation interval. The initiation interval is the number of cycles between successive accepted independent transactions under the stated schedule. It describes how often new work can enter; latency describes how long one item remains in the computation. A resource that accepts one multiplication per cycle cannot support more than one multiplication per cycle simply because its surrounding circuit is pipelined. Two multiplications per transaction impose a resource bound of at least two cycles per transaction for one such multiplier, before other restrictions are considered.

Arrival time, required time and slack. Arrival time states when a signal reaches a point relative to a chosen reference. Required time states the relevant latest or earliest acceptable boundary. Setup slack compares actual arrival with the latest safe arrival; hold slack compares the earliest new-data change with the earliest safe change. The sign convention must be stated because the protected inequalities differ. A positive slack means that particular modeled check has margin; it does not certify unrelated paths, modes or operating conditions.

Clock skew. Skew is a difference between clock arrival times at specified points. With capture arrival minus launch arrival as the sign convention, positive skew can increase a single-cycle setup budget and reduce hold margin. The same shift gives old data more time to arrive for the next capture and requires it to remain unchanged longer around the current capture. This causal explanation is more useful than remembering that skew is always bad or that a slower clock fixes every timing problem.

Application and validation in Lesson 04

Scheduling assigns operations to clock steps while respecting dependencies and resource availability. Binding determines which physical operator executes each operation. Storage must retain each intermediate until its consumer uses it; transaction labels distinguish values belonging to different requests.

Deriving a resource bound. A request containing two additions requires two adder uses. If one nonpipelined adder accepts at most one operation each cycle, N completed requests require at least $2N$ occupied adder cycles. The long-run initiation interval is therefore at least two cycles, independent of whether one request's latency is two or three cycles. A second adder can remove this resource bound, but dependency and clock-period constraints still apply. For a two-stage pipeline at a 10 ns period, a request can have 20 ns latency while new requests begin every 10 ns after the pipeline fills; latency and initiation interval describe different quantities.

Complete original notebook pages for Lesson 04

Part 1 original page 11

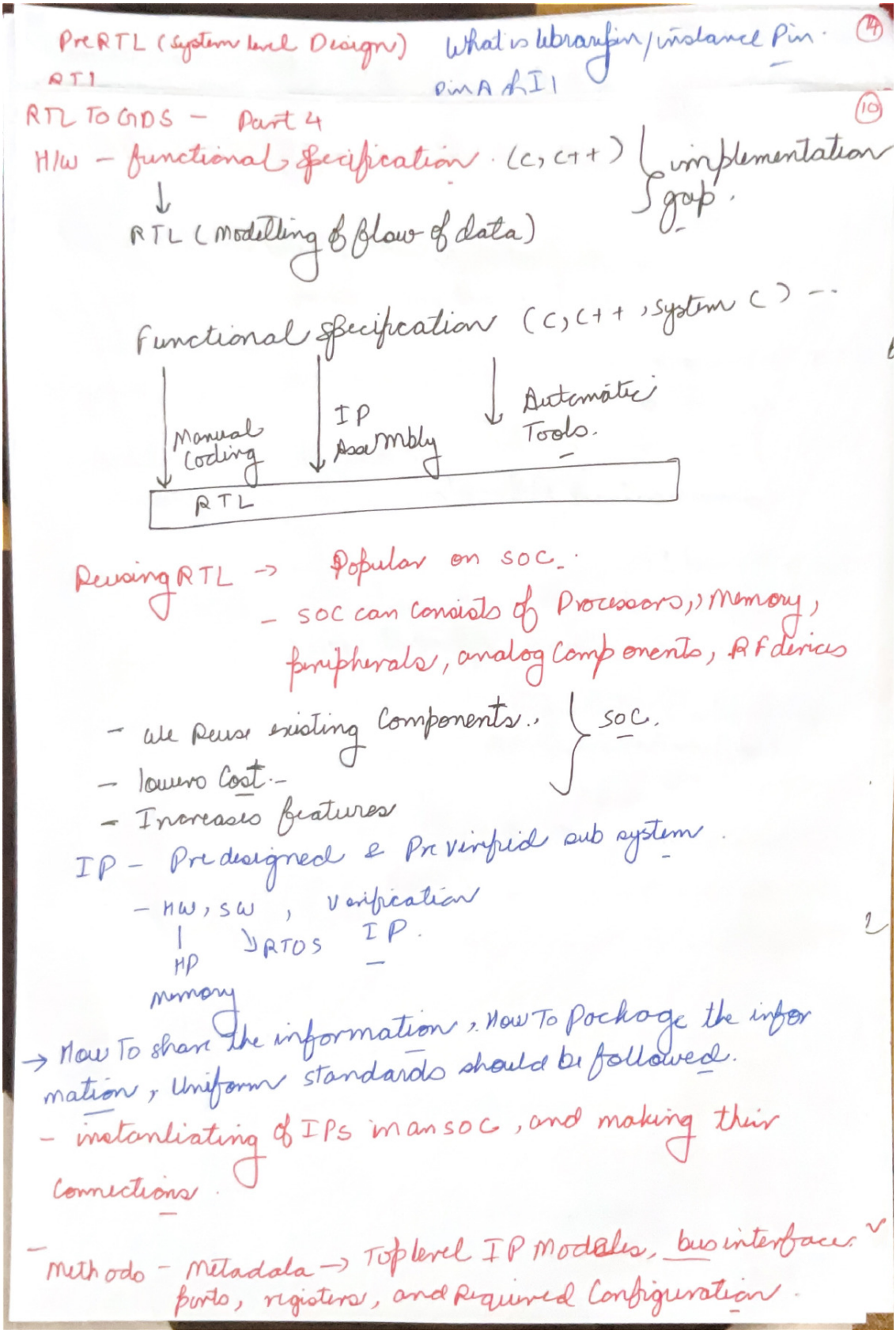

Figure 1.50. *Source: Part 1, PDF page 11. Implementation gap, SoC integration, IP contents.*

Explanation and worked examples: Day 1 · Lesson 04.

Part 1 original page 12

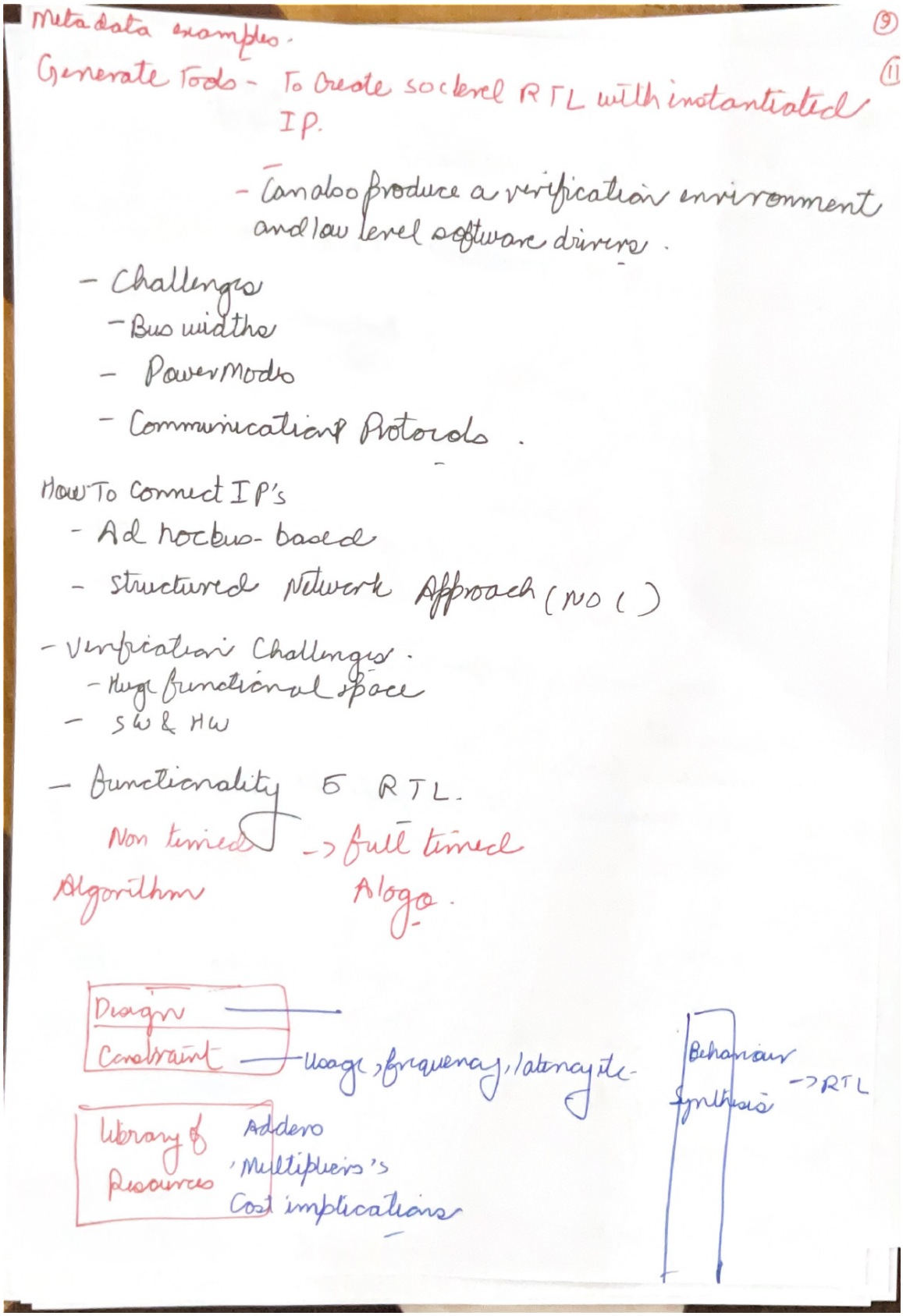

Figure 1.51. *Source: Part 1, PDF page 12. IP metadata/configuration, interconnect, HLS inputs.*

Explanation and worked examples: Day 1 · Lesson 04.

Part 1 original page 13

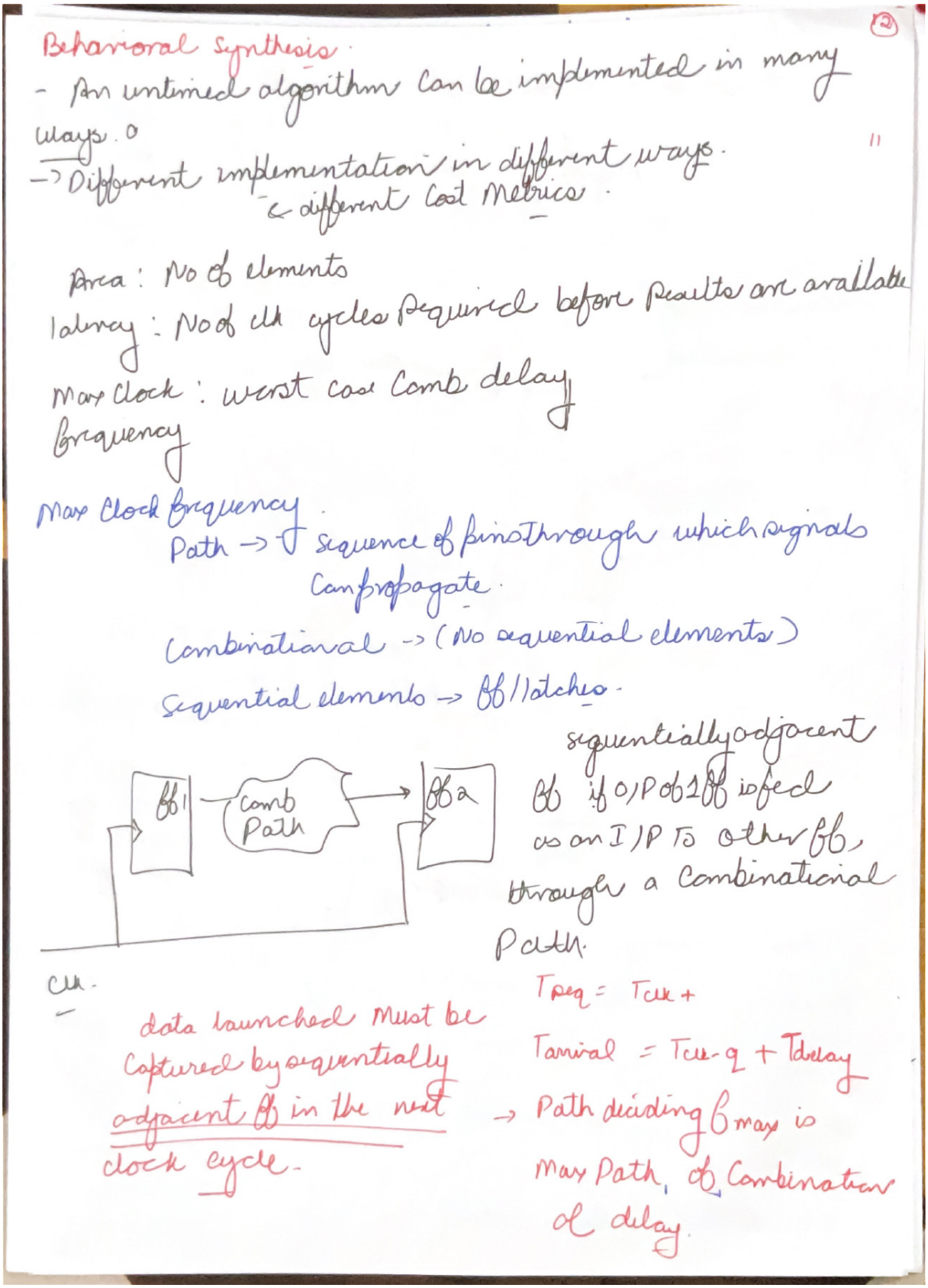

Figure 1.52. *Source: Part 1, PDF page 13. Area/latency/frequency, register paths, arrival/required time.*

Explanation and worked examples: Day 1 · Lesson 04.

Part 1 original page 14

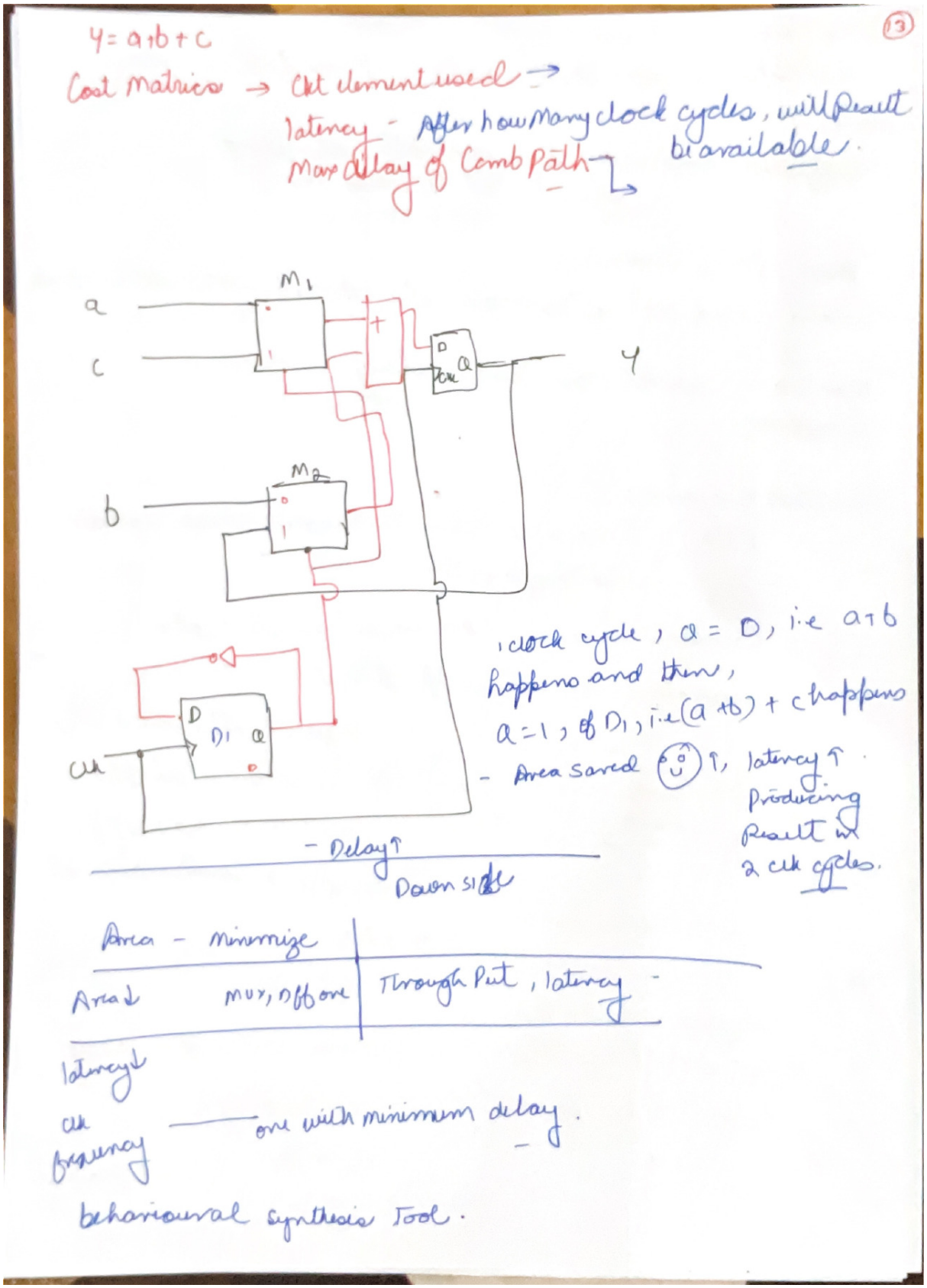

Figure 1.53. *Source: Part 1, PDF page 14. Shared adder, feedback, multiplexer, control and schedule.*

Explanation and worked examples: Day 1 · Lesson 04.

Lesson 05 Unix foundations for EDA

Week 1 · Lecture video

A shell is the working interface

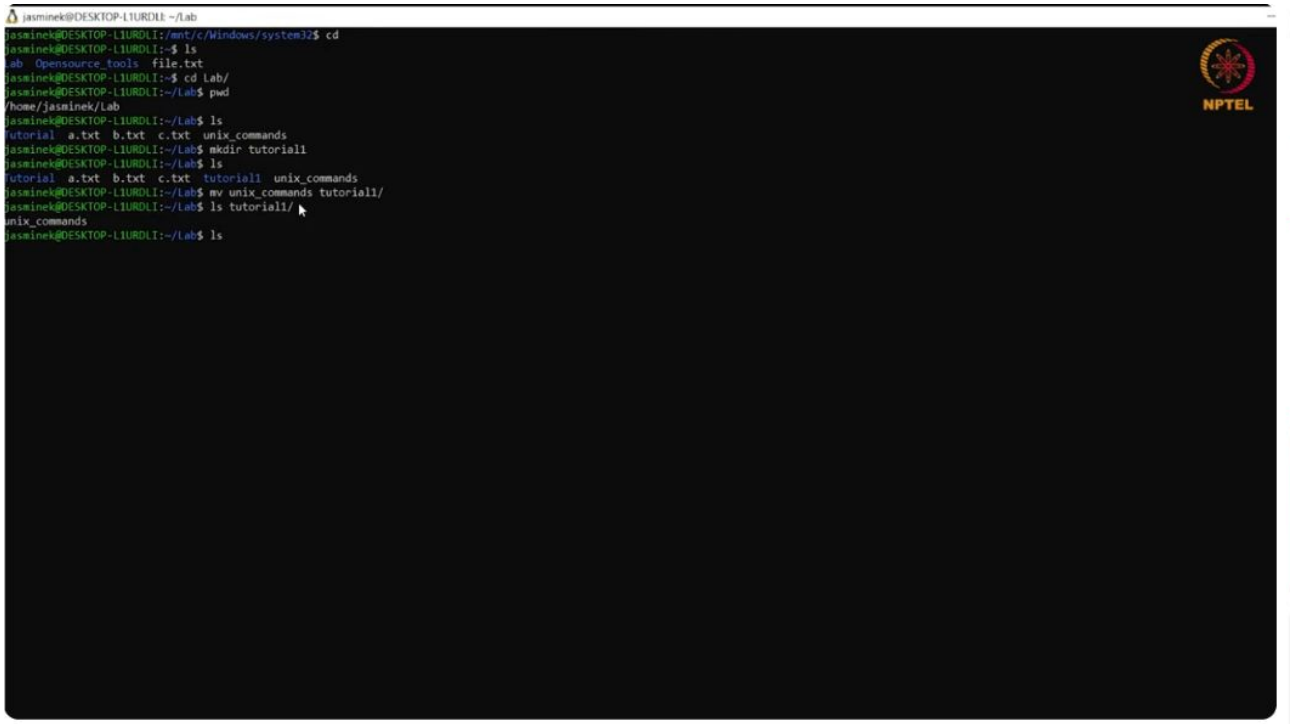

Figure 1.54. *Lecture: 4:18. The tutorial demonstrates file and directory commands in a Linux shell*

This tutorial is presented by **Jasmine Kaur**, the course teaching assistant. It introduces Unix-style command-line work used to launch tools, manage design files, inspect logs, and control long-running jobs. A **shell** interprets commands; a **terminal** provides the shell's text interface. The **working directory** is the directory against which relative paths are resolved.

On Windows, WSL provides a Linux environment. The tutorial demonstrates `wsl --install` from administrator PowerShell, a restart, and initial Linux-user setup. The exact installation path depends on the existing Windows/WSL state; Microsoft's current installation instructions describe the prerequisites and supported command.

There is no dedicated Unix page in the uploaded handwriting. The Tcl note belongs to Lesson 10, because a Tcl interpreter and a Unix shell are different command environments.

Navigation and file operations

Command	Meaning	Detail that prevents a common mistake
`pwd`	Print working directory	Check it before interpreting a relative path
`ls`	List directory entries	`ls -a` also includes hidden names beginning with `.`
`cd path`	Change working directory	`cd ..` goes to the parent; `cd` normally goes home
`mkdir tutorial1`	Create a directory	The directory name is an argument, not a command
`mv a.txt tutorial1/`	Move a file	The original pathname no longer refers to it after success
`cp a.txt a_copy.txt`	Copy a file	The source remains; an existing destination can be overwritten
`touch d.txt`	Update timestamps, creating a file if absent	It does not erase an existing file's contents
`rm d.txt`	Remove a file	Ordinary shell removal does not provide a desktop recycle-bin workflow
`cat a.txt`	Write file contents to standard output	Best suited to text; large logs are easier to page or search

An **absolute path** starts from the filesystem root, such as `/home/student/lab`; a **relative path** is interpreted from the current directory, such as `rtl/top.v`. Quote paths containing spaces. Linux filenames are generally case-sensitive: `Top.v` and `top.v` can name different files.

This small practice sequence creates a fresh directory and copies a file without deleting any study material:

```bash
mkdir tutorial1_practice
cd tutorial1_practice
pwd
touch example.txt
cp example.txt example_copy.txt
ls
```

Expected result: both filenames appear in the new directory. `touch` creates an empty file here because the name did not exist. Running the sequence again requires accounting for the already-created directory; shell commands change persistent filesystem state.

Help, permissions, disk space, and processes

Command	What it answers
`which cat`	Which executable is found through the search path, for this external command?
`man ls`	What does the local manual say about `ls` and its options?
`sudo command`	Run an authorized command with elevated or another user's privileges
`du -h directory`	How much allocated disk space do these directory contents use?
`df -h`	How much space is used and available on mounted filesystems?
`ps`	What processes are selected by this invocation and its options?
`top`	How are processes and system resources changing over time?
`history`	What commands are recorded by the shell's history feature?
`whoami`	What is the current effective username?

`du` and `df` answer different questions and need not show the same number. A small `du` result for one project does not prove the filesystem has free space. Plain `ps` usually selects only a subset of processes; "all processes" requires appropriate options such as `ps -e` on common Linux systems.

In the tutorial, `sudo apt-get update` refreshes package metadata. It does not by itself upgrade every installed package. `sudo` is a privilege boundary, not a prefix to add automatically whenever an ordinary command fails. Shell built-ins such as `cd` also differ from external executables; `type cd` can identify a built-in where `which` is less informative.

Foreground and background jobs

The tutorial runs `sleep 100`, suspends it with Ctrl+Z, resumes it with `bg %1`, and returns it to the foreground with `fg %1`. `sleep` delays that process; it does not put the whole computer into sleep mode.

A process ID (PID) identifies an operating-system process. **A job number** such as `%1` identifies a job tracked by the current shell. They are not the same identifier. Ctrl+Z ordinarily suspends a foreground job; `bg` resumes a stopped job in the background; `fg` brings a job into the foreground. `jobs` lists the shell's jobs. A trailing `&` starts a command asynchronously. These distinctions are described in the GNU Bash manual.

For EDA, this matters when a simulation or implementation run takes a long time. A background process still consumes resources and can still write to files. Backgrounding is not the same as making a job survive logout or a terminal shutdown.

Recall checks: Why does `cd` have to affect the shell itself? What is the difference between `%1` and PID 1? Why can a failed `cp` command result from a wrong working directory rather than missing privileges?

Definitions and mechanisms in Lesson 05

Terminal and shell. A terminal is the interaction surface for text input and output. A shell is the program that interprets command syntax, performs its defined expansions and launches or controls commands. Several shells can be used through the same terminal application. A command's behavior therefore depends on the interpreter, not the appearance of the window. Bash syntax, Tcl syntax and PowerShell syntax are different even when all three appear as text at a prompt.

Working directory and path resolution. A process has a current working directory used to interpret relative paths. An absolute path identifies a location from the relevant filesystem root; a relative path requires that current-directory context. A missing-file error may result from using the correct relative name in the wrong directory. Printing the working directory and checking the expected file are concrete diagnostic steps. The directory containing a script and the process's working directory are not automatically the same.

Process and shell job. A process is an operating-system execution instance with a PID and resources. A job is a shell's grouping and tracking of commands for job control; it can involve a pipeline containing multiple processes. A job identifier such as percent one belongs to that shell's job table, while a PID belongs to the operating system's process namespace. Foreground and background describe interaction with the shell and terminal, not whether computation is happening or whether files can be changed.

Suspension and background execution. A suspended job has stopped executing until resumed. A background job can continue running while the shell accepts more input. Ctrl+Z normally requests suspension of the foreground job; bg resumes a stopped job in the background; fg brings a job to the foreground. A long EDA run can still consume CPU, memory and disk bandwidth in the background. Background execution also does not by itself guarantee survival after logout or terminal closure.

Executable search and privileges. The shell resolves command names using built-ins, functions and its configured executable search rules. A missing command may mean the tool is absent or its directory is outside PATH. Permission failure is a different diagnosis. Elevation changes execution privileges; it does not correct a misspelled filename, a wrong working directory or syntax intended for a different shell. Identify the failure before deciding what action addresses it.

Application and validation in Lesson 05

A reproducible tool run records the working directory, complete input paths, invocation, completion status and output paths. Relative paths are resolved against the process's working directory, not against the source file's location. Copying preserves the original pathname; moving relocates it.

Diagnosing an incomplete run. In a shell command `tool design.v > run.log`, standard output is redirected while standard error may still appear in the terminal. `tool design.v > run.log 2>&1` sends both streams to the log in a conventional Unix shell. A log file's existence only establishes that redirection opened a file, not that the tool succeeded. The shell's exit status must be inspected after the tool completes, and the expected output must be checked for existence and relevant content. Background launch returns control before that completion evidence is available.

Lesson 06 Logic synthesis and technology mapping

Week 2 · Lecture video

RTL plus libraries plus constraints

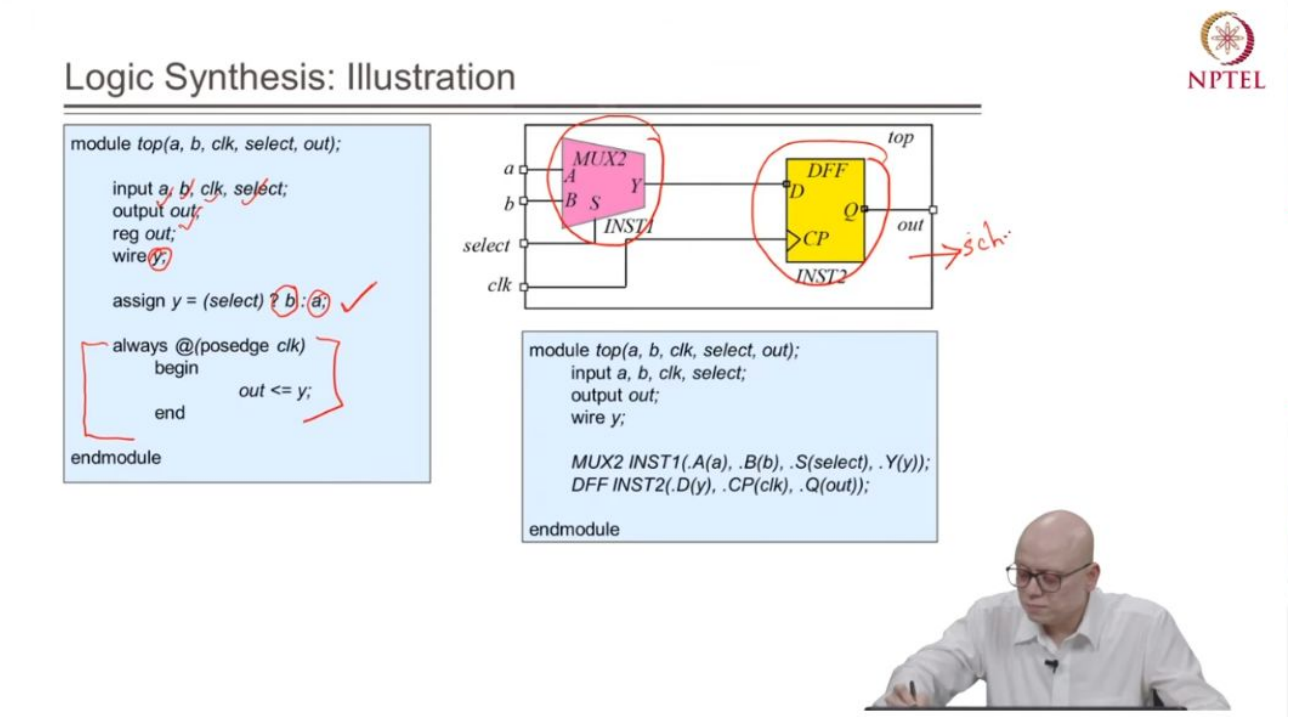

Figure 1.55. *Lecture: 9:38. RTL selection and storage mapped into a mux and flip-flop*

Logic synthesis converts a synthesizable RTL description into a functionally equivalent network of available implementation cells while optimizing against constraints. It does not fabricate silicon or choose all final wire geometries. Its main output is a **netlist** describing instances and connectivity.

Input	Role	Common representation
RTL	Intended state and combinational behavior	Verilog/VHDL source
Technology library	Available cell functions and characterized properties	Liberty `.lib`
Constraints	Clock requirements, external timing, loads, and other design intent	SDC and tool commands

The same file extension `.v` may contain behavioral RTL or a structural gate-level netlist. Inspect the contents to determine its abstraction. The synthesis goal is not merely to produce syntactically legal Verilog: the chosen cells must implement the intended behavior and meet the modeled constraints.

The lecture uses a mux feeding a flip-flop. This complete equivalent teaching version keeps the behavior clear:

```
Verilog
module select_register (
    input  wire a,
    input  wire b,
    input  wire select,
    input  wire clk,
    output reg  out
);
    wire y;
    assign y = select ? b : a;
    always @(posedge clk)
        out <= y;
endmodule
```

`assign` describes the mux continuously. The edge-sensitive process describes output storage. There is no reset in this example, so simulation starts with `out` unknown until a defined value is captured. A technology library might provide a mux cell and a D flip-flop, or the mapper might implement the mux using an equivalent combination of gates.

Library pins and instance pins

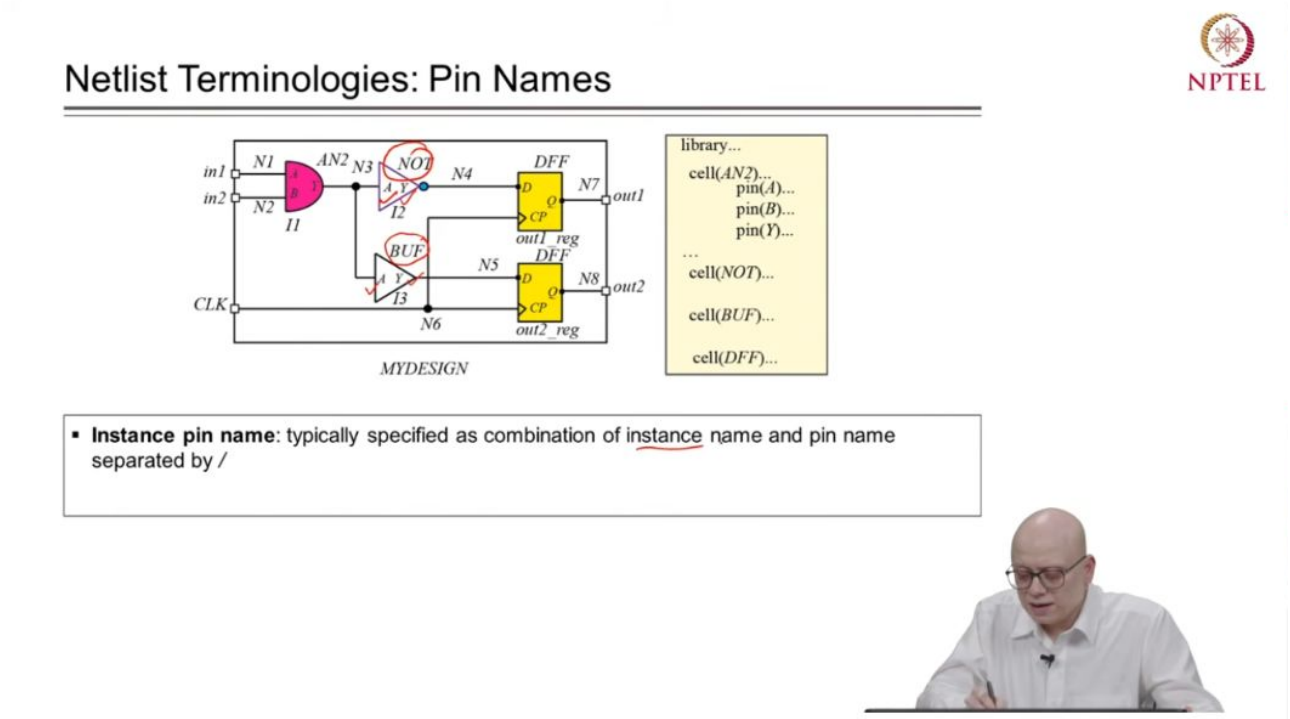

Figure 1.56. *Lecture: 25:48. A cell definition can have multiple independently named instances*

A library describes reusable cell types. A netlist contains particular instances of those types and nets connecting their pins. Read the figure by first identifying each cell type, then its instance name, and finally the pin being connected. This distinction prevents two pins named `A` on different gates from being mistaken for one connection.

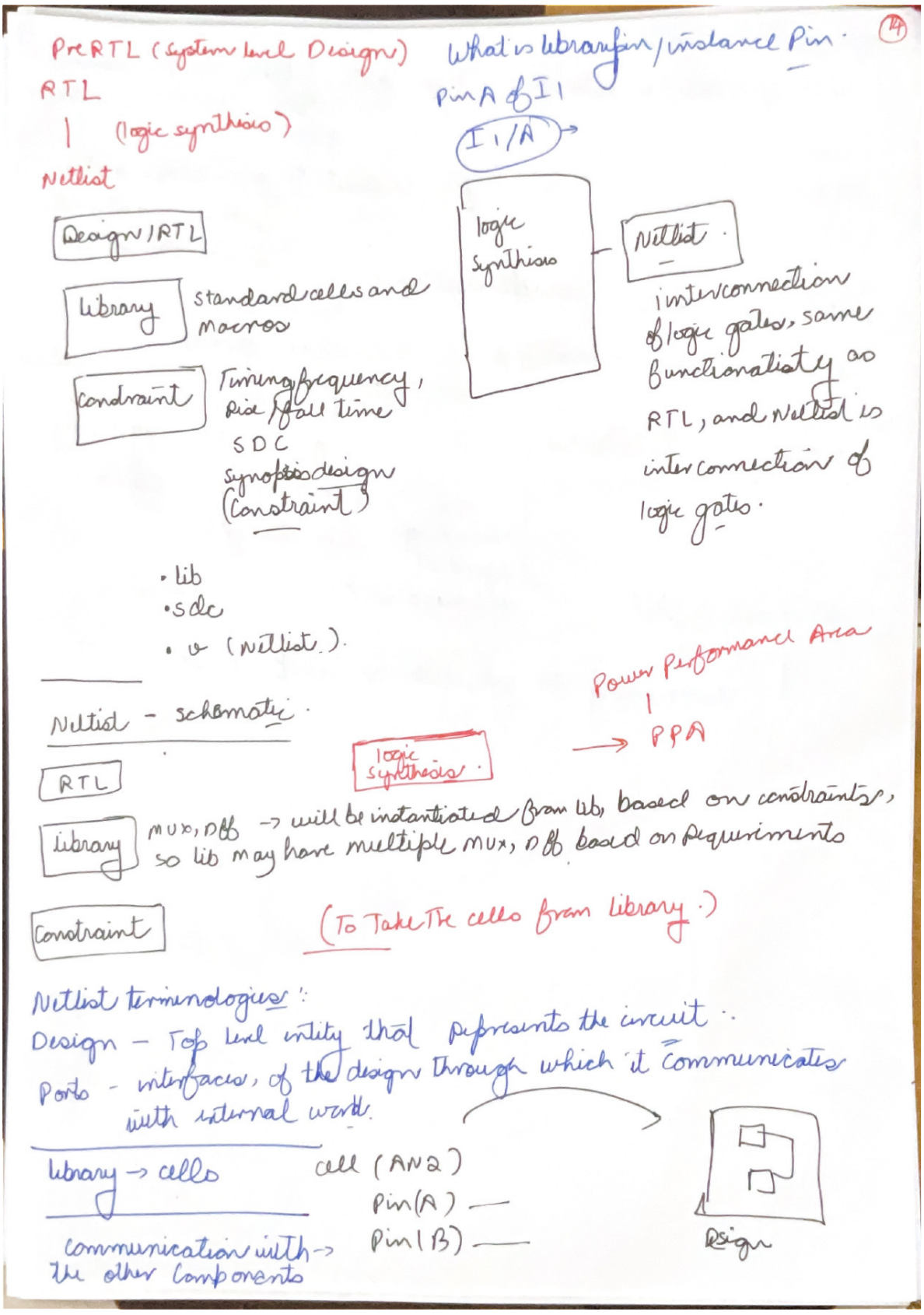

Figure 1.57. *Source: Part 1, PDF page 15.*

What is a library pin versus an instance pin?

A **library cell** is a reusable definition, such as a two-input AND gate called `AN2`. Its **library pins** `A`, `B`, and `Y` describe the interfaces and associated properties of that cell type. An **instance** is one use of that cell in a design. If `AN2` is instantiated as `I1` and `I2`, then `I1/A` and `I2/A` are different physical/logical connection points, although both refer to the library definition's `A` pin.

Name	Meaning
`AN2`	Library cell type
`AN2/A`	Pin A of the library definition, using explanatory notation
`I1`	One instance of AN2
`I1/A`	Pin A on that particular instance
`in1`	A top-level input port, if declared on the design boundary
`n1`	A net connecting a set of pins and/or ports

The slash is the course's naming convention; a tool may use another separator or a hierarchical path. Instance names are unique within their scope, while a complete hierarchical path identifies an instance across the design. Repeated local pin names such as `A` therefore need instance context.

A **port** belongs to a module/design boundary. A **pin** in this synthesis vocabulary belongs to a cell or instance. A **net** is a connection, not a gate. A net can have one driver and several loads, giving fanout. Direction is defined relative to the owner: an input port brings a signal into the top design, while an output pin of an internal cell can drive a net inside that design.

The `.lib`, `.sdc`, and `.v` triangle is a useful reminder: changing constraints or the cell library can change the result even when the RTL is unchanged. Relaxing timing may allow smaller/slower cells; tightening timing can require faster cells or a different logic structure. Unachievable constraints do not force the tool to create impossible hardware.

Timing arcs slew load and operating corners

A **timing arc** describes a timing relationship between specified pins of a cell. For an AND gate, an `A`-to-`Y` propagation arc describes the output response to a relevant input transition, under conditions that permit that transition to propagate. The `B`-to-`Y` arc is a separate relationship. For a flip-flop, clock-to-Q is a propagation relationship; setup and hold are timing-check relationships between data and clock. A timing arc therefore need not mean that data flows directly between its two named pins.

Slew, or transition time, measures how long a signal takes to cross specified voltage thresholds. It is different from **propagation delay**, which measures the time between reference crossings at the input and output. A slow input edge and a late output edge are related effects, but they are not the same measurement.

Capacitive load is the capacitance the output must drive, including connected input pins and interconnect. Many cell models tabulate output delay and transition against input slew and output load, with separate rise/fall behavior. Consequently, “this gate takes 50 ps” is incomplete without its arc, input transition, loading, and operating conditions. Increasing load commonly slows an output; a stronger cell may drive it faster while presenting more input capacitance to the preceding stage.

For an actual example, the SKY130 medium-speed library's table templates name `input_net_transition` and `total_output_net_capacitance` as delay-table variables. The same file distinguishes 50% propagation-delay reference thresholds from 20%-80% slew thresholds. Those values belong to that library; inspect the chosen library before interpreting another timing report.

An operating **PVT corner** specifies process assumptions, supply voltage, and temperature. Different corners can produce different delays and leakage. Which corner is limiting depends on the check and technology; “hot is always worst for every check” is not a general rule.

For instance, `tt_025C_1v80` in the documented SKY130 configuration means typical NMOS/PMOS process assumptions, 25 °C, and 1.80 V. The process label describes a characterization assumption, not the manufacturing date or a physical corner of the die. LibreLane's timing-corner guide also distinguishes these cell PVT corners from interconnect corners used to model wire parasitics.

Keep the information sources separate:

Data	Question it helps answer
Liberty `.lib`	What function and characterized timing/power behavior does this cell have?
Netlist `.v`	Which instances exist, and how are their pins connected?
SDC constraints	What clocks, external timing, and timing intent must the implementation satisfy?
LEF physical abstract	What placement dimensions, pin shapes, and routing obstructions must physical tools respect?
SPEF parasitics	What modeled interconnect resistance and capacitance affect the implemented nets?

For concrete format context, see OpenSTA's accepted timing inputs and SKY130's file-type descriptions. A LEF pin shape alone does not provide a timing arc, and a logic netlist alone does not provide extracted wire parasitics.

Generic logic and technology mapping

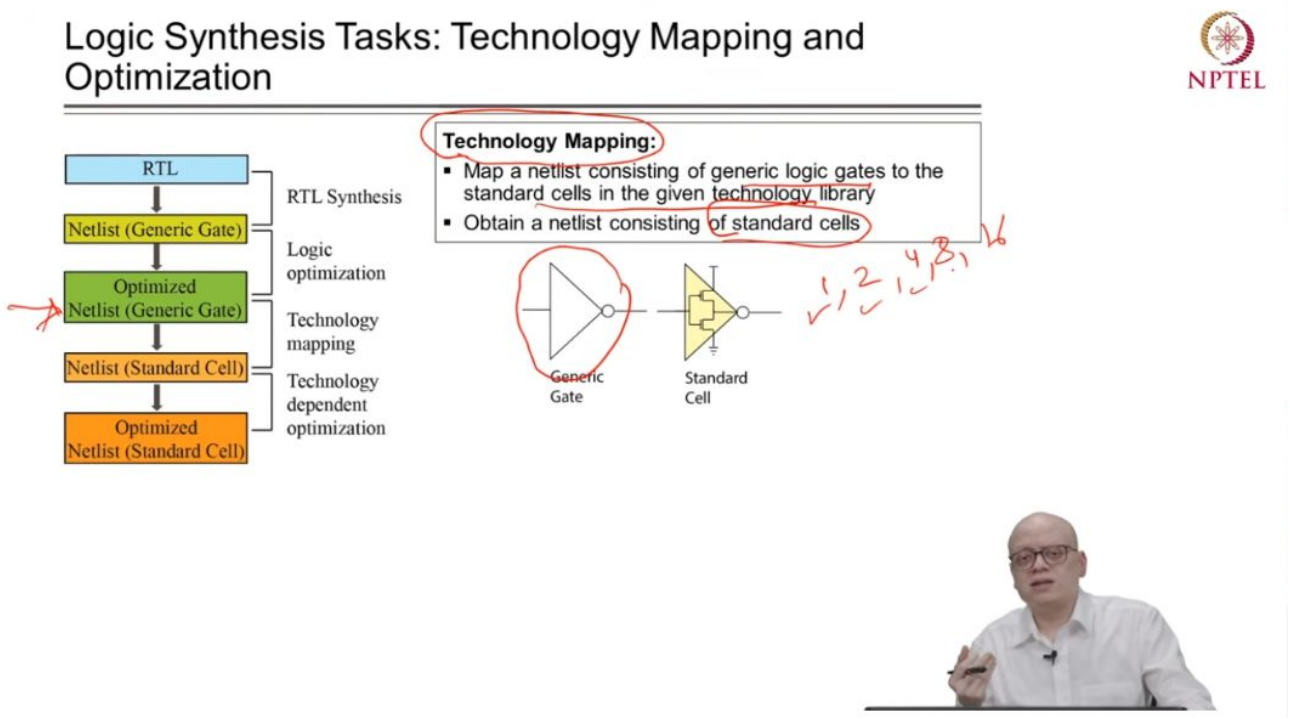

Figure 1.58. *Lecture: 38:43. Technology mapping chooses real library cells and drive strengths*

Technology mapping replaces technology-independent logic with implementations available in the selected library. The available cells have real area, timing, drive, and power properties. Mapping and subsequent optimization therefore choose more than a Boolean symbol: they choose realizations that must meet the supplied constraints.

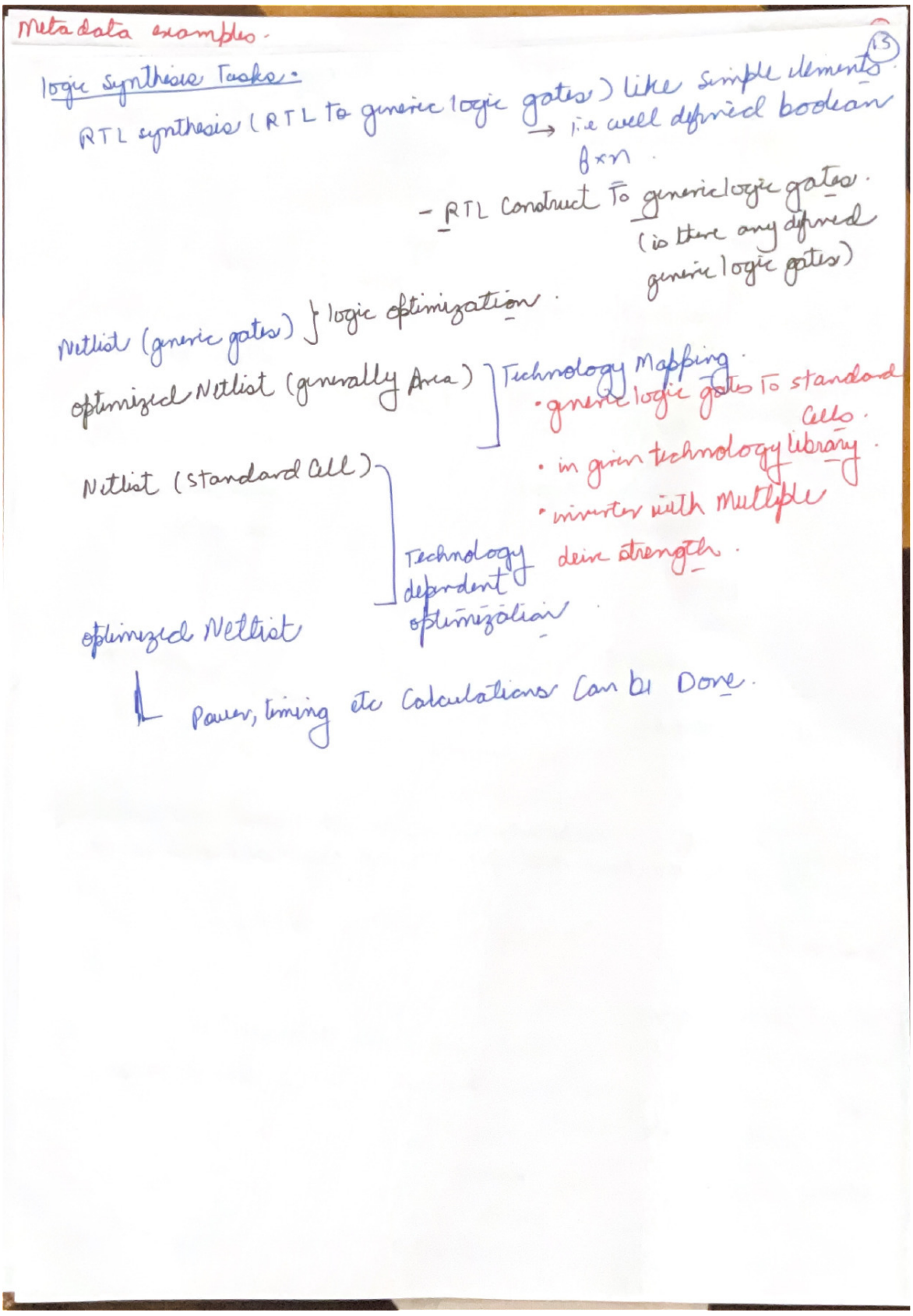

Figure 1.59. *Source: Part 1, PDF page 16.*

Is there a defined set of generic logic gates?

There is no single universal internal gate set required of every synthesis tool. A tool may represent intermediate logic using Boolean operators, muxes, arithmetic operators, flip-flops, or a more specialized internal graph. “Generic” means that the representation has not yet committed to a particular characterized cell implementation. A generic inverter specifies logical inversion; a library inverter adds a concrete implementation and characterized properties.

The lecture's synthesis sequence is:

1. **Parse and elaborate RTL:** interpret syntax, parameters, hierarchy, widths, and connectivity; derive hardware behavior.
2. **Create and simplify generic logic:** infer operators and storage, propagate constants, remove unused logic, and simplify Boolean structure.
3. **Technology map:** cover the required logic with cells available in the target library.
4. **Optimize the mapped implementation:** resize, restructure, buffer, or otherwise improve timing, area, and power while maintaining the required function.

For instance, $ab + ac = a(b + c)$ is a Boolean factoring opportunity. Whether the factored form is physically better depends on available complex gates, loading, fanout, and timing. The smallest number of drawn generic gates need not map to the smallest or fastest real implementation.

A stronger inverter can drive a larger load with less output delay, but its input may load the preceding stage more heavily. The mapper must consider the path and network, not a cell in isolation. Generic gate count provides a rough structural cost; it is not an accurate physical area or power number. Mapped cell models improve estimates, but pre-route wire estimates still differ from extracted post-route parasitics.

Equivalence checking verifies that intended behavior survives transformation. Timing analysis checks the modeled temporal constraints. Neither check replaces the other. The resulting netlist proceeds into physical implementation, with test-related transformations included where the flow requires them.

Preserve polarity when mapping a Boolean function

For the fully specified binary function $F = ab + cd$, define two NAND outputs $n_1 = \overline{ab}$ and $n_2 = \overline{cd}$. A final NAND implements:

$$F = \overline{n_1 n_2} = \overline{\overline{ab} \cdot \overline{cd}} = ab + cd$$

The final inversion is part of the equivalence. Connecting the first two NAND outputs to an OR instead would give $\overline{ab} + \overline{cd}$, a different function. At $a = b = c = d = 0$, the required output is zero, whereas that incorrect connection produces one. This single counterexample exposes a polarity error that a plausible gate drawing can hide.

Two ANDs followed by an OR and three NANDs are both legal three-cell descriptions if those cells exist in the library. A compound cell implementing the same function may provide another covering choice. The Boolean proof establishes functional legality, while characterized arcs, input capacitances, output load and wiring determine the physical cost. A mapper cannot conclude that the NAND covering is faster merely from its Boolean identity or its cell count. Apply this two-step

reasoning to the generic-gate question on Part 1 page 16: first establish the function, then compare its available implementations under the same constraints.

Definitions and mechanisms in Lesson 06

Logic synthesis. Logic synthesis interprets supported RTL behavior and constructs an implementation using available cells under constraints. It includes elaboration, inference, simplification, mapping and optimization. Its output is a netlist whose instances and connections implement the required behavior. The output is still a design representation: synthesis does not manufacture transistors or determine every routed wire. Correct syntax, logical equivalence and timing feasibility are different properties and require their respective checks.

Cell definition, instance, pin, port and net. A cell definition specifies a reusable building block. An instance is one occurrence of that block. A library pin belongs to the definition; an instance pin belongs to a particular occurrence. A port is a connection on a module boundary, and a net connects pins or ports. Two NAND instances can both have a pin called A without those pins being electrically connected. Use the complete instance context when identifying a connection point.

Timing arc. A timing arc is a modeled timing relationship between named cell pins under specified conditions. A combinational arc can describe propagation from an input to an output. A clock-to-Q arc describes a sequential output response, while setup and hold relationships constrain data relative to clock. The word arc therefore covers more than a wire inside a cell. Interpreting a timing report requires knowing which relationship is being measured and the transition, load and operating condition used.

Slew, delay and load. Slew measures the duration of a signal transition between defined voltage thresholds. Propagation delay measures separation between defined input and output reference crossings. Load is the electrical burden an output drives, commonly modeled with pin and wire capacitance. These quantities interact: a slower input or larger output load can change cell delay and output transition. A stronger cell may improve its own output transition while increasing the load of the stage driving its input.

Technology mapping and optimization. Mapping selects available cell implementations for technology-independent logic. Optimization then changes choices or structure to improve the specified measures while preserving behavior. A library's complex gate may implement a function with fewer or better stages than a literal translation into basic gates. Conversely, sharing a logical term can create a high-fanout net. Boolean simplification is one input to physical quality, not a direct measurement of area, delay or power.

Constraint and characterization corner. A constraint communicates design intent, such as clock timing and external delays. A characterization corner provides modeled cell behavior for process, voltage and temperature assumptions. The tool evaluates the design using both. A timing report with the wrong clock or corner may answer a different question from the one intended. State the scenario when discussing a result; “the circuit meets timing” is incomplete if the analyzed conditions are unspecified.

Application and validation in Lesson 06

Synthesis interpretation starts with the selected RTL top, elaborated parameters, compatible library and constraints. In the mux/register example, continuous selection describes combinational logic and the rising-edge assignment stores its selected value. The absent reset leaves the initial register value unspecified in the simulation model.

Why the smallest cell can miss timing. Assume one critical arc has an 80 ps budget after other path contributions are accounted for. Under the same input slew and load, a small implementation has area 1.0 units and delay 95 ps; a stronger implementation has area 1.4 units and delay 70 ps. The smaller option has slack $80 - 95 = -15$ ps; the larger has $80 - 70 = 10$ ps. Selecting the larger option satisfies this isolated arc budget, but its increased input capacitance can slow the preceding stage. The complete path and surrounding electrical checks must therefore be reevaluated; the local 25 ps improvement is not automatically a 25 ps path improvement.

Complete original notebook pages for Lesson 06

Part 1 original page 15

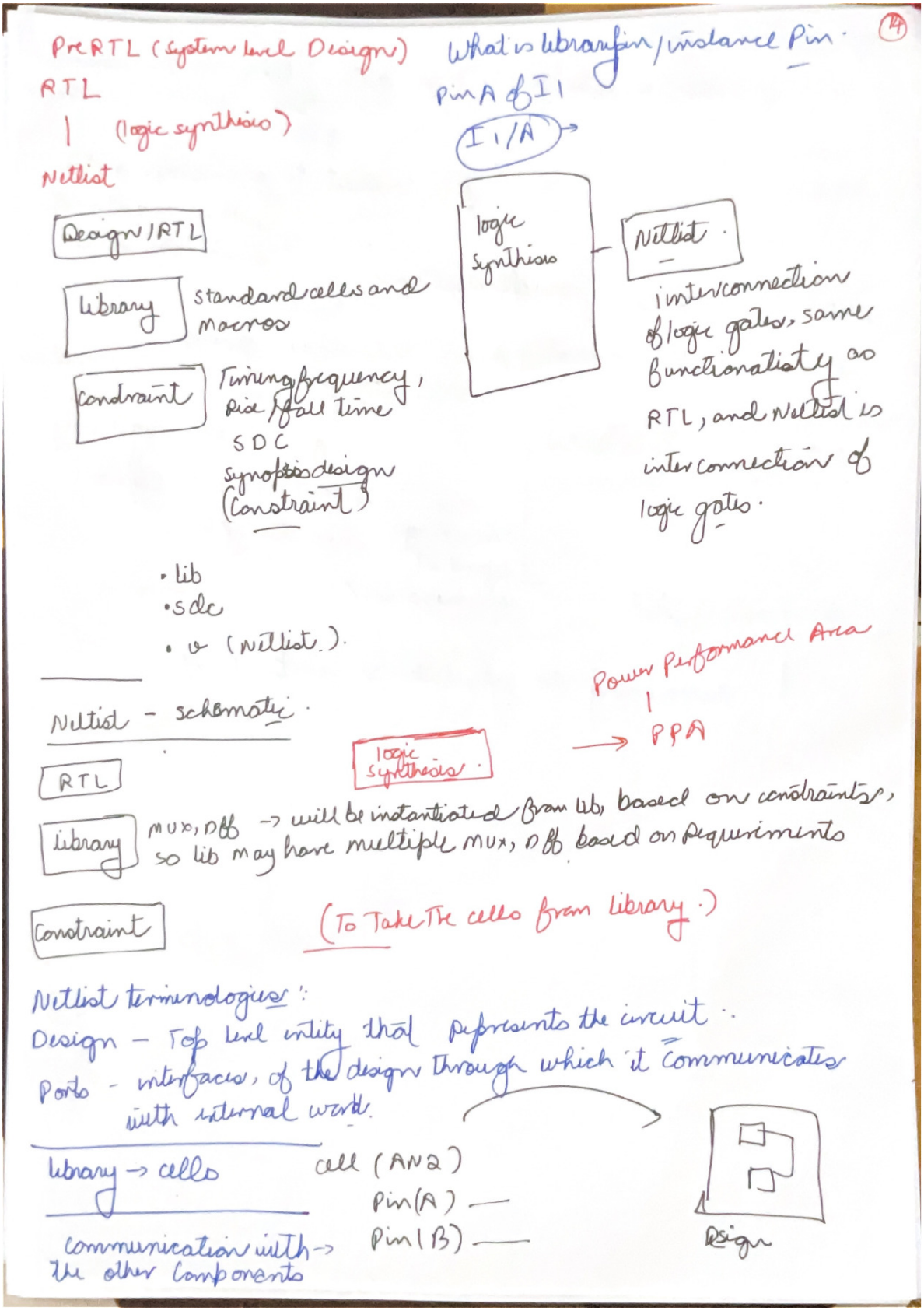

Figure 1.60. *Source: Part 1, PDF page 15. Synthesis inputs, library pin versus instance pin.*

Explanation and worked examples: Day 1 · Lesson 06.

Part 1 original page 16

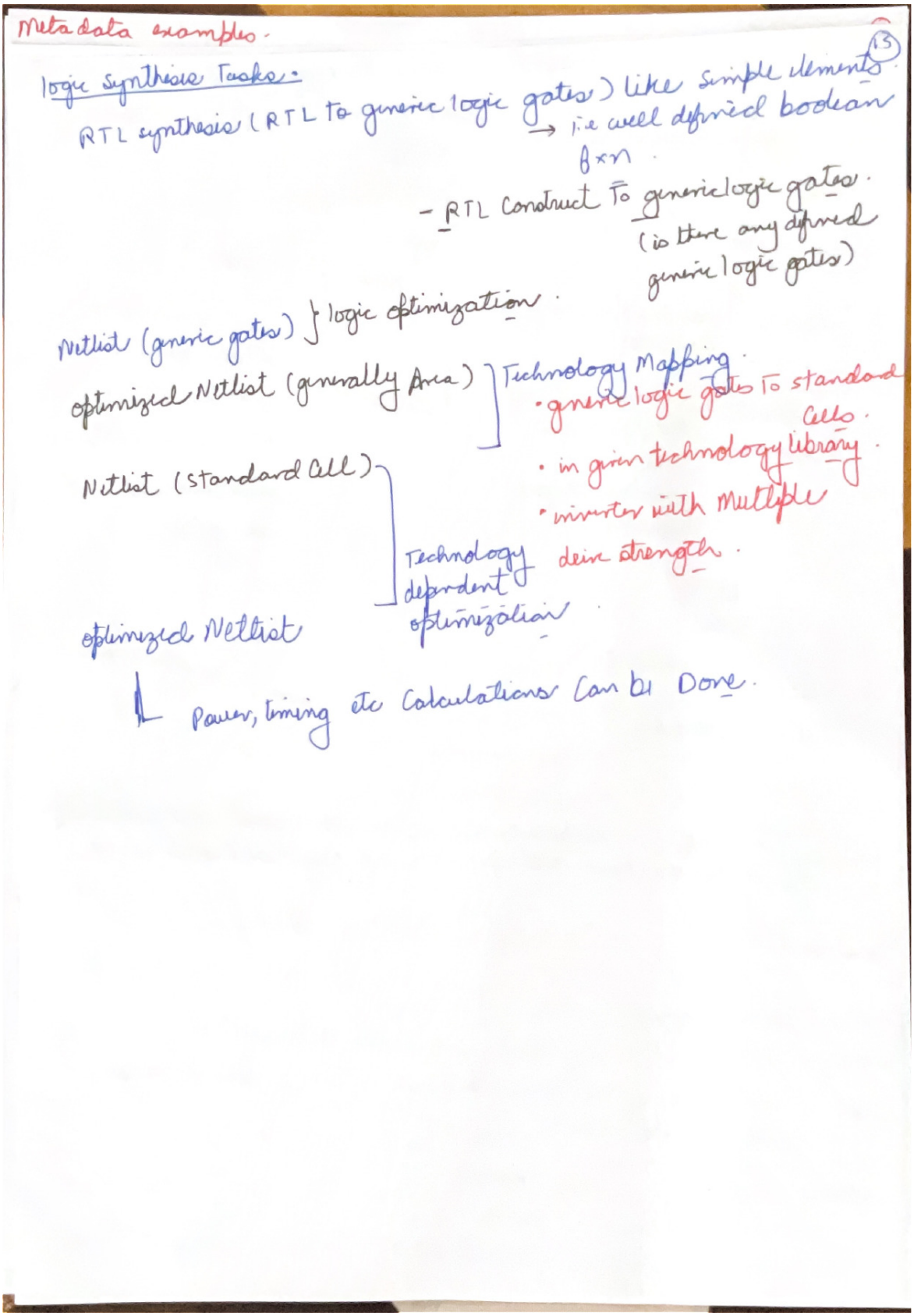

Figure 1.61. *Source: Part 1, PDF page 16. Synthesis stages and the set of generic gates.*

Explanation and worked examples: Day 1 · Lesson 06.

Revision checks and explained answers

Try each question before reading its answer. Revisit the linked lesson and its original handwritten page when an assumption or step is unclear.

Check 01: Manufacturing yield has a denominator

Question. A teaching wafer has 500 gross dies and 400 electrically good dies. What is die yield, and what happens if only 350 packaged devices pass final test?

Explained answer. Die yield is $400/500 = 0.8$, or 80%. Packaging/final-test yield relative to good dies is $350/400 = 0.875$, or 87.5%. The overall shipped fraction relative to gross dies is $350/500 = 0.7$, or 70%. Multiplying the two conditional yields gives the same 70%. Keep these populations separate; a transistor count or an RTL simulation result does not supply either yield.

Check 02: Recover the fixed investment

Question. An ASIC costs 9,000,000 more in fixed investment than an alternative. Its shipped-unit cost is 20, versus 35 for the alternative. Find break-even volume; repeat when the ASIC unit cost becomes 29.

Explained answer. The saving per shipped unit is $35 - 20 = 15$, giving $9{,}000{,}000/15 = 600{,}000$ units. At cost 29 the saving is only 6, so break-even becomes 1,500,000 units. Equal fixed investment has not changed; lower per-unit savings require 2.5 times as many sales. If ASIC unit cost reaches 35, these assumptions give no positive-volume crossover.

Check 03: Check the accelerator's interface

Question. A 32-bit bus transfers one word per cycle at 100 MHz. Each item needs 80 transferred bytes, and the required rate is three million items per second. Is raw bandwidth sufficient?

Explained answer. Ideal bus capacity is $4 \times 100 = 400$ MB/s in decimal units. The workload needs $80 \times 3 = 240$ MB/s, or 60% of that ideal capacity. This arithmetic establishes headroom only before protocol, arbitration and memory stalls. If usable efficiency is below 60%, the interface misses the requirement even if the accelerator arithmetic is fast enough.

Check 04: Separate latency from throughput

Question. Each adder takes 2 ns. Clock-to-Q plus setup totals 0.5 ns per registered stage; ignore wires and skew. Compare two serial combinational adders, one reused adder, and a two-stage pipeline for a+b+c.

Explained answer. Two serial adders need a period of at least 4.5 ns. One reused adder can use a 2.5 ns period but needs two dependent operations: 5 ns latency and, without overlapping independent work, one result per 5 ns. Two pipelined adders also have 5 ns latency, but independent inputs can produce a result every 2.5 ns once filled. Resource count, latency and initiation interval are different quantities.

Check 05: Know what a shell command changes

Question. After entering a project directory, what do pwd, ls and cd .. each do? Why does the working directory matter for an EDA script?

Explained answer. pwd reports the current directory; ls lists entries; cd .. changes the shell's directory to its parent. A relative path in a tool command is interpreted against a working-directory context, which may differ from the script's own location. Record or establish that context so the same RTL, library and constraint paths resolve on a repeat run. Listing a file does not execute or validate it.

Check 06: Count library and instance pins

Question. Two instances U1 and U2 use the same XOR2 cell with library pins A, B and Y. Count cell instances, library pin definitions and instance pins.

Explained answer. There are two cell instances and three pin definitions on the shared library-cell type. Each instance has its own A, B and Y connection points, so there are six instance pins. U1/A and U2/A refer to different connections, even though both are modeled by the same library pin A. Timing arcs belong to the cell model; connectivity belongs to particular instances.

Handwritten page finder

Every entry opens the complete original page embedded in this PDF, beside its explanation. Original scan page numbers and this PDF's page numbers are different.

Original handwritten page	Open in this PDF
Part 1, page 1	Figure 1.14
Part 1, page 2	Figure 1.15
Part 1, page 3	Figure 1.16
Part 1, page 4	Figure 1.27
Part 1, page 5	Figure 1.28
Part 1, page 6	Figure 1.29
Part 1, page 7	Figure 1.30
Part 1, page 8	Figure 1.39
Part 1, page 9	Figure 1.40
Part 1, page 10	Figure 1.41
Part 1, page 11	Figure 1.50
Part 1, page 12	Figure 1.51
Part 1, page 13	Figure 1.52
Part 1, page 14	Figure 1.53
Part 1, page 15	Figure 1.60
Part 1, page 16	Figure 1.61

Sources and references

Primary course: **VLSI Design Flow: RTL to GDS**, Prof. Sneh Saurabh, IIIT Delhi, NPTEL. Unix and Tcl tutorials: Jasmine Kaur. Lecture captions provide video timestamps. Additional derivations and practice examples are study annotations.

Open the course video playlist.

Original handwritten sources: **Part 1 PDF pages 1 to 16**. Source identifiers are separate from the page numbers of this document.

Primary references for the technical clarifications

1. SKY130 - Assumptions

2. ASML - Lithography principles

3. ASML - Semiconductor manufacturing steps

4. SKY130 - File types

5. Intel - Inside Intel Core Microarchitecture, page 5

6. AMD - HLS scheduling principles

7. AMD HLS loop dependencies

8. Microsoft - Install WSL

9. GNU Bash - Job control

10. SKY130 - Medium-speed library table templates

11. LibreLane - Timing corners

12. OpenSTA - Timing analyzer inputs

Numerical examples use stated teaching assumptions. Technology-specific library values and manufacturing rules require their corresponding source models. Verilog examples use traditional Verilog semantics unless otherwise stated.